

Enhancing Scholarly Document Accessibility and Analysis

Frameworks for Semantically Rich PDFs, Efficient Transformation, and Advanced Bibliometric Applications

Zur Erlangung des Grades eines Doktor der Philosophie (Dr. phil.)

Vorgelegte Dissertation im Fachbereich Gesellschafts- und Geschichtswissenschaften
von Changxu Duan

Tag der Einreichung: 23. Dezember 2025, Tag der Prüfung: 2025-MM-DD

1. Gutachten: Prof. Dr. Andrea Rapp

2. Gutachten: Dr. Sabine Bartsch

Darmstadt, Technische Universität Darmstadt



TECHNISCHE
UNIVERSITÄT
DARMSTADT

History and Social Sciences
Department

Institute of Linguistics and
Literary Studies

Enhancing Scholarly Document Accessibility and Analysis
Frameworks for Semantically Rich PDFs, Efficient Transformation, and Advanced Biblio-
metric Applications

Submitted doctoral thesis in the department of History and Social Sciences by Changxu
Duan

Date of submission: December 23, 2025

Date of thesis defense: 2025-MM-DD

Darmstadt, Technische Universität Darmstadt

Erklärungen laut Promotionsordnung

§ 8 Abs. 1 lit. d PromO

Ich versichere hiermit, dass zu einem vorherigen Zeitpunkt noch keine Promotion versucht wurde. In diesem Fall sind nähere Angaben über Zeitpunkt, Hochschule, Dissertationsthema und Ergebnis dieses Versuchs mitzuteilen.

§ 9 Abs. 1 PromO

Ich versichere hiermit, dass die vorliegende Dissertation – abgesehen von den in ihr ausdrücklich genannten Hilfen – selbstständig verfasst wurde und dass die „Grundsätze zur Sicherung guter wissenschaftlicher Praxis an der Technischen Universität Darmstadt“ und die „Leitlinien zum Umgang mit digitalen Forschungsdaten an der TU Darmstadt“ in den jeweils aktuellen Versionen bei der Verfassung der Dissertation beachtet wurden.

§ 9 Abs. 2 PromO

Die Arbeit hat bisher noch nicht zu Prüfungszwecken gedient.

Darmstadt, 23. Dezember 2025

Changxu Duan

Abstract

PDF is the dominant format for scholarly documents, but most PDFs lack the structural information needed for accessible reading or reliable automated processing. This dissertation presents three contributions that address this problem at different stages of the document lifecycle.

First, `LaTeX Rainbow` generates accessible PDFs from LaTeX sources. It uses a color-coding mechanism to preserve semantic structure through the compilation process, producing PDFs that are closer to PDF/UA compliance.

Second, `EditTrans` and `CopyLookup` provide efficient methods for converting existing PDFs to structured formats. These hybrid approaches achieve 1.5 to 1.7 times speedup over end-to-end vision-language models by copying plain text directly and generating only complex elements like formulas and tables.

Third, `SOFT` framework demonstrates the value of structured input for downstream analysis. It introduces a semantically orthogonal approach to citation classification that separates citation intent from cited content type, improving annotation consistency and cross-domain transfer.

Together, these contributions form an integrated framework: preventing structural deficiencies in new documents, remediating existing documents, and enabling better downstream analysis. The research validates through peer-reviewed publications that document accessibility benefits both human users and automated systems.



Zusammenfassung

the german abstract will be here...

Contents

1	Introduction	1
1.1	Background: Evolution of Scholarly Communication	2
1.2	Success and Limitations of PDF	4
1.3	Technical Analysis of Structural Challenges	5
1.4	Research Contributions	8
1.5	Thesis Structure	10
2	LaTeX Rainbow: Generating Inherently Accessible Scholarly PDFs	11
2.1	Thesis Integration: Outputs, Dataset, and Reuse	11
2.1.1	What This Chapter Produces	11
2.1.2	A Copyright-Unrestricted Fine-Grained arXiv Dataset	12
2.1.3	The Reuse of These Outputs	13
2.2	Publication Context and Author Contributions	13
2.3	The LaTeX Accessibility Landscape	13
2.3.1	Progress and Prospects of the LaTeX Tagged PDF Project	14
2.3.2	The Persistence of pdfTeX	15
2.3.3	Alternative Approaches and Their Trade-offs	16
2.4	Technical Foundations of PDF Accessibility	17
2.4.1	Compliance Target Under Practical Engine Constraints	17
2.4.2	The “Double-Tree” Architecture of Tagged PDFs	17
2.4.3	PDF/UA: The Accessibility Standard	18
2.5	The LaTeX Rainbow Framework	19
2.5.1	Design Inspiration: Lessons from Document Layout Analysis	19
2.5.2	The Color-Coding Alignment Mechanism	20
2.5.3	Mathematical Content Handling	21
2.6	Implementation	21
2.7	Evaluation	22
2.8	Discussion and Contributions	23
2.8.1	Positioning and Complementarity	23

2.8.2	Limitations	23
2.8.3	Implications for Scholarly Communication	24
3	Efficient PDF-to-Structured-Markdown Transformation using Hybrid Approaches	25
3.1	The Need for Scalable PDF Conversion	25
3.2	Technical Foundations	26
3.2.1	The Evolution of Vision-Language Models	26
3.2.2	From OCR to Neural Document Conversion	27
3.2.3	Speculative Decoding	28
3.2.4	Text Editing Models	29
3.3	CopyLookup: Accelerating VLM Decoding via PDF-Sourced Drafts	30
3.3.1	Design Rationale	30
3.3.2	Methodology	31
3.3.3	Experimental Validation	31
3.4	EditTrans: Hybrid Copy-Generate for PDF-to-Markdown	32
3.4.1	Design Rationale	32
3.4.2	Methodology	32
3.4.3	Experimental Validation	33
3.5	Comparative Analysis and Discussion	33
3.6	Conclusions	34
4	SOFT: A Semantically Orthogonal Framework for Nuanced Citation Analysis	37
4.1	The Evolution of Citation Analysis	37
4.1.1	Origins: From Citation Counts to Indices	37
4.1.2	Early Classification Frameworks: Recognizing Heterogeneity	38
4.1.3	Sociological Critiques: The Limits of Citation Analysis	38
4.1.4	The NLP Era: Automated Classification	39
4.1.5	Summary: Progress and Persistent Gaps	40
4.2	Limitations of Existing Citation Classification Schemas	41
4.2.1	A Linguistic Analysis of Schema Design	42
4.2.2	The Subject-Verb-Object Paradigm	44
4.3	The SOFT Framework: Design and Rationale	46
4.4	Methodology: Applying SOFT to Structured Document Inputs	46
4.5	Evaluation	48
4.6	Discussion: Enhancing Bibliometrics and Library Science Applications	49

5	Discussion	51
5.1	An Integrated Document Lifecycle Framework	51
5.1.1	From Creation to Analysis	51
5.2	Positioning Within the Broader Ecosystem	52
5.2.1	Relationship to the LaTeX Tagged PDF Project	52
5.2.2	Implications for Digital Libraries and Publishers	53
5.3	Limitations	53
5.3.1	Scope Limitations	53
5.3.2	Technical Limitations	54
5.3.3	Evaluation Limitations	54
5.4	Future Research Directions	54
5.4.1	Toward Accessible Document Intelligence	55
5.4.2	Specific Research Directions	56
6	Conclusion	59
6.1	Summary of Contributions	59
6.2	The Research Journey	59
6.3	Methodological Reflections	60
6.4	Closing Remarks	61
	Publications	81

1 Introduction

Every year, researchers produce millions of scholarly documents. These documents encode scientific findings, form the basis for citation networks, and serve as inputs for search engines, machine learning models, and bibliometric analyses. The dominant format for this exchange is PDF.

PDF became dominant because it guarantees consistent rendering: a PDF file displays identically regardless of operating system, installed fonts, or viewer application. This visual fidelity was valuable in the 1990s when document interchange formats lacked standardization. However, this design prioritizes appearance over structure. PDF stores text as a sequence of drawing instructions that position glyphs at specific coordinates. It does not inherently encode which glyphs form a paragraph, which paragraph belongs to a section, or what the logical reading order should be.

This lack of structure creates two distinct problems. For people who use assistive technologies such as screen readers, unstructured PDFs are often partially or entirely unreadable. For computer systems that process documents automatically, unstructured PDFs require error-prone inference to recover the information that was discarded during document creation. Both problems have been recognized for decades. Accessibility standards like PDF/UA have existed since 2014 [50], and text extraction from PDF has been an active research area throughout [83, 8]. Yet recent empirical studies show that the vast majority of scholarly PDFs still lack the structural information needed for accessible reading or reliable automated processing [61, 101].

This dissertation addresses these problems through technical interventions at different stages of the document lifecycle. The goal is to make PDF documents more structured, enabling both accessible reading for people using assistive technologies and reliable processing by automated systems.

1.1 Background: Evolution of Scholarly Communication

Origins and Functions of the Scholarly Journal The modern system of scholarly communication began in 1665 with the launch of two journals: the *Journal des Sçavans* in Paris and the *Philosophical Transactions of the Royal Society* in London [38, 14]. These publications emerged as a solution to a practical problem that had long troubled natural philosophers: how to establish priority for a discovery without keeping the discovery itself secret.

Before journals, scientists faced a dilemma. If they openly shared a discovery, someone else might claim the idea as their own. If they kept it secret, they could not prove they had made the discovery first. A common workaround was the encrypted anagram: a scientist would write a Latin description of the discovery, scramble the letters into a meaningless string, and send this string to a trusted third party with a dated seal. Later, when ready to reveal the discovery, the scientist could unscramble the anagram. The dated record proved that the discovery had been made by that date, while the scrambled form prevented others from stealing the idea in the interim. Galileo, for example, used anagrams in 1610 to announce his observations of Saturn and Venus without revealing the details.

This approach protected priority but had serious drawbacks: others could not read, verify, or build upon the work. Knowledge remained locked away. The scholarly journal solved this problem by providing a trusted, dated, public record. Once a finding was published, the publication date established priority, and the content was immediately available for others to examine and extend. Scientists no longer needed secrecy to protect their claims.

Journals served three functions that remain central today. First, registration: establishing a dated public record of a discovery or contribution. Second, archiving: creating a permanent, cumulative record of knowledge. Third, certification: providing quality control through editorial oversight and, later, peer review. These functions define what scholarly documents are supposed to do, and they provide criteria for evaluating whether modern document formats support or hinder scholarly communication.

Digital Transformation The transition from print to digital began in the 1990s. The internet made it possible to distribute publications globally at low cost. Publishers digitized their workflows, converting paper-based processes to electronic ones.

This transition was largely conservative. The fundamental unit of publication remained the article, formatted to resemble a printed page. Publishers adopted PDF because it

preserved their existing workflows. A PDF could be created from the same typesetting files used for print production, and it would look identical to the printed version.

Alternative approaches were possible. SGML and later XML offered formats that explicitly encoded document structure. The Journal Article Tag Suite (JATS), now an ANSI/NISO standard, provides a schema specifically designed for scholarly articles [92]. HTML, the native format of the web, is inherently structured and accessible.

However, PDF became the dominant format before these alternatives gained traction in scholarly publishing. The result is an example of path dependence: early adoption created momentum that was difficult to reverse. Publishers had invested in PDF workflows, authors expected PDF as the output format, and readers had tools to view PDFs. The format became locked in.

Contemporary Data Governance Frameworks Two developments in the 2010s changed expectations for digital documents.

First, the FAIR principles were published in 2016 [127]. FAIR stands for Findable, Accessible, Interoperable, and Reusable. Originally formulated for research data, these principles articulate what it means for digital objects to be usable by both humans and machines. Research funders, including the European Commission and major national funding agencies, now require that funded research produce FAIR-compliant outputs.

Second, accessibility requirements strengthened. The Web Content Accessibility Guidelines (WCAG) evolved through multiple versions [19]. PDF/UA (Universal Accessibility), standardized as ISO 14289, specifies requirements for accessible PDFs. The standard has two versions: PDF/UA-1 (2014, based on PDF 1.7) and PDF/UA-2 (2024, based on PDF 2.0, with enhanced support for mathematical content) [50, 51]. Chapter 2 discusses these standards and their implications for LaTeX workflows in detail.

Legislation in multiple jurisdictions has created accessibility obligations. In the United States, Section 508 of the Rehabilitation Act requires federal agencies to make electronic content accessible, which affects federally funded research. The Americans with Disabilities Act (ADA) has been interpreted to apply to university websites and digital content. In Europe, the Web Accessibility Directive requires public sector websites to meet accessibility standards, and the European Accessibility Act extends requirements to certain digital products and services. While these laws do not uniformly mandate accessible scholarly PDFs, they create growing pressure on institutions and publishers to address document accessibility.

These frameworks apply not only to datasets but also to documents. A scholarly article is itself a digital object. If it cannot be found, accessed, interoperated with other systems, or reused, it fails the FAIR test. If it cannot be read by someone using a screen reader, it fails accessibility requirements.

1.2 Success and Limitations of PDF

Design Goals and Widespread Adoption Adobe released the PDF specification in 1993. The format was later standardized as ISO 32000 [49]. The core design goal was to enable documents to be shared and printed with consistent visual appearance across different systems.

PDF achieved this goal and became ubiquitous. In government and policy, a bibliometric study of the Overton policy document database found that PDF is the majority format for governmental, intergovernmental, and think tank publications [116]. In academic publishing, major journals, conference proceedings, and preprint servers distribute articles primarily as PDF. In education and business, PDF serves as a common format for electronic textbooks [129], contracts, reports, and external communications. The format's widespread adoption is well established, though precise global statistics are difficult to verify.

Technical Characteristics To understand the challenges PDF presents, it helps to understand how the format works at a technical level.

A PDF file contains several types of objects. The most important for our purposes are content streams and, optionally, a structure tree. Content streams contain the instructions for rendering the document. These instructions position text at specific coordinates, select fonts, draw lines and shapes, and embed images. The structure tree, when present, provides a logical hierarchy of elements like sections, paragraphs, and headings.

The key point is that content streams store text as positioned glyphs, not as logical units. Consider a two-column layout. The content stream might contain instructions like:

```
Draw "Left column, line 1" at (50, 700)
Draw "Right column, line 1" at (300, 700)
Draw "Left column, line 2" at (50, 680)
Draw "Right column, line 2" at (300, 680)
```

A human viewer sees two columns and reads down each column in turn. But the content stream does not indicate that the first and third items form one column and the second and fourth form another. Extracting text in the correct reading order requires heuristically inferring the layout from coordinates.

The structure tree can solve this problem by explicitly specifying the logical order and hierarchy. However, the structure tree is optional. Most PDF creation tools do not generate it by default. When it is absent, any consumer of the PDF must infer structure from visual layout cues.

The Current State Empirical studies have measured how often PDFs include proper structural information.

A 2024 study by Wang and Kumar analyzed accessibility in approximately 20,000 research papers from major publishers. They found that only 12.6% of papers were tagged in any meaningful way, and just 3.2% passed a basic six-point accessibility check [61].

A separate 2024 study by Pierrès and colleagues examined 8,000 papers from four large academic publishers. They found that approximately 88% of PDFs had more than 80% of their content completely untagged. Tables and mathematical formulas were almost never correctly tagged [101].

Studies of screen reader users confirm the practical impact of inaccessible documents. Research has shown that users of assistive technologies frequently encounter content that is partially or entirely unreadable, significantly affecting their ability to extract information accurately and efficiently [107].

These findings indicate that the problem is not theoretical. Most PDFs in circulation lack the structure needed for accessible reading or reliable automated processing.

1.3 Technical Analysis of Structural Challenges

Challenges for Assistive Technology Screen readers convert visual content into speech or braille output. To do this effectively, they need to know the reading order of content on each page, the role of each element (heading, paragraph, list item, table cell), alternative text descriptions for images and figures, the structure of tables including which cells are headers, and the content of mathematical formulas in a readable form.

When a PDF lacks a structure tree, screen readers must guess. They may read text in the order it appears in the content stream, which can differ from the intended reading order. They cannot distinguish headings from body text because both are stored as positioned glyphs. Tables may be read as a sequence of unrelated text fragments. Users report concrete problems with inaccessible documents, including content read out of order, inability to navigate by headings, and inaccessible tables. These barriers significantly increase the time and effort required to complete tasks that would be straightforward with properly structured documents.

Challenges for Automated Processing Computer systems that process PDFs face related but distinct challenges. We can categorize these at three levels.

Level 1: Text Extraction. The first task is simply to get the text out of the PDF. For scanned documents, this requires optical character recognition (OCR), and accuracy depends on scan quality. For native PDFs (those created digitally rather than scanned), text can usually be extracted directly, but problems remain. The extraction may lose word spacing if the PDF does not include explicit space characters. Ligatures (combined letter forms like "ff" or "fi") may be extracted as single characters or decomposed incorrectly. Reading order may not match logical order.

Level 2: Structure Recognition. Given extracted text, the next task is to identify structure: where paragraphs begin and end, which text is a heading, where tables and figures are located, and how they relate to surrounding text. Without a structure tree, these must be inferred from visual cues. This is the domain of document layout analysis, an active research area with imperfect solutions. Errors at this level propagate downstream.

Level 3: Semantic Understanding. The highest level involves understanding what the content means: extracting citations and linking them to referenced works, parsing mathematical formulas, interpreting tables, and identifying claims and evidence. Current approaches, including large language models and vision-language models, operate at this level but depend on accurate input from the lower levels.

Research has quantified these limitations. A benchmark study found that even state-of-the-art vision-language models like GPT-4 and Gemini achieve 15 to 25% character error rates on text extraction from document images [138]. Complex layouts cause particular problems: one evaluation found that Gemini 1.5 Pro achieved 95% edit similarity on single-column legal contracts but dropped to 60% on multi-column academic papers [98].

Downstream Impacts Structural deficiencies in source documents affect multiple downstream applications:

Information retrieval. Search engines index documents based on extracted text. If extraction is inaccurate or the structure is lost, search quality decreases. Degraded extraction affects both traditional keyword search and modern retrieval-augmented generation (RAG) systems that use document collections to ground language model outputs [138].

Bibliometrics. Citation analysis depends on correctly extracting and linking references. Studies have documented that extraction errors lead to undercounting of citations, which can bias metrics like the Impact Factor and h-index [33, 35].

Knowledge organization. Constructing knowledge graphs from scientific literature requires accurate entity and relation extraction. Errors in source document processing propagate into the knowledge graph as missing or incorrect relations.

Content reuse. Translating, summarizing, or reformatting documents requires reliable structural input. If the input is noisy, downstream processing is more difficult and less accurate.

Long-term preservation. Archival systems need structured metadata to ensure documents remain usable over time. Unstructured PDFs may become effectively inaccessible as rendering software evolves.

Root Causes These challenges are not inherent defects of PDF. The format supports structure trees, tagged content, and accessibility features. The problems arise from three sources. First, design priorities: PDF was designed primarily for visual fidelity, and structure was added later as an optional layer. Second, tooling defaults: most PDF creation tools, including common word processors and typesetting systems like pdfLaTeX, do not generate structure by default, requiring authors to take explicit action. Third, ecosystem inertia: changing defaults or workflows requires coordinated action across authoring tools, publishers, and readers, and this coordination has not occurred at scale.

This analysis suggests two intervention points. At the creation stage, authoring tools can be modified or augmented to produce structured output. At the processing stage, techniques can be developed to recover structure from unstructured PDFs more accurately and efficiently.

1.4 Research Contributions

This dissertation addresses both intervention points through three contributions that span the document lifecycle. Chapter 5 discusses how these contributions form an integrated framework.

Contribution 1: LaTeX Rainbow LaTeX is a widely used authoring tool in fields with mathematical content, including physics, mathematics, and a significant portion of computer science. LaTeX sources contain explicit semantic markup: authors use commands like `\section`, `\begin{theorem}`, and `\begin{equation}` that clearly indicate document structure.

However, when LaTeX is compiled to PDF using the standard pdfTeX engine, this structure is lost. The resulting PDF contains only positioned glyphs.

LaTeX Rainbow addresses this gap. It uses a color-coding mechanism to pass structural information through the compilation process. Each semantic element is assigned a unique color. After compilation, the color information in the PDF can be used to reconstruct the structure tree.

This approach works with unmodified pdfTeX, which is important because many major publishers and platforms, including Springer, IEEE, ACM, and arXiv, continue to use pdfTeX-based workflows. The system produces PDFs that are closer to PDF/UA compliance and that provide reliable structural information for downstream processing.

Chapter 2 presents this contribution in detail. The work has been published in [28, 25].

Contribution 2: EditTrans and CopyLookup For the vast archive of existing PDFs, the creation-stage solution does not apply. These documents must be processed as they are.

Recent work has shown that vision-language models can convert PDF page images directly to structured markup like Markdown or LaTeX. Models such as Nougat process a page image end-to-end and generate the corresponding text with formatting [10]. However, these models are slow because they generate every token autoregressively, even for plain text that could simply be copied from the source.

EditTrans and CopyLookup address this efficiency problem using hybrid approaches. CopyLookup uses a speculative decoding strategy: text extracted from the PDF serves as draft tokens that the language model verifies rather than generates. EditTrans goes

further by decomposing the task: plain text is copied directly from the PDF, and the language model generates only the complex elements like formulas and tables.

These methods achieve speedups of 1.5 to 1.7 times while maintaining or improving output quality. The efficiency gain matters for practical deployment: processing millions of documents is feasible only if per-document costs are low enough.

Chapter 3 presents these contributions. The work has been published in [24, 26].

Contribution 3: SOFT Framework The previous two contributions produce structured documents. The third contribution demonstrates what becomes possible with structured input.

Citation analysis studies how academic papers reference other works. Existing classification schemas categorize citations by function: does the citing paper use the method from the cited work, extend it, compare against it, or simply mention it as background?

These schemas have a problem. They conflate two distinct dimensions: the action the citing paper takes (use, extend, critique) and the type of content being cited (a method, a finding, a dataset). This conflation reduces annotation consistency and limits cross-domain generalizability.

The Semantically Orthogonal Framework for Citation Classification (SOFT) separates these dimensions. Citation intent (what the author does) and cited content type (what is being cited) are classified independently. This orthogonal design produces clearer annotations and better cross-domain transfer.

Importantly, SOFT performs best when the input text is clean and well-structured. The structured outputs from the first two contributions provide exactly this: reliable citation contexts without OCR errors or structural ambiguity.

Chapter 4 presents this contribution. The work has been published in [27].

Connections Between Contributions The three contributions are linked technically:

- LaTeX Rainbow produces PDFs with known ground-truth structure. This data can train and evaluate the models in EditTrans and CopyLookup.
- EditTrans and CopyLookup produce structured Markdown from legacy PDFs. This provides cleaner input for tasks like citation classification.

- SOFT shows that structured input improves classification performance.

Chapter 5 discusses how these technical connections form a larger framework. Each contribution also addresses part of making documents more FAIR:

FAIR Dimension	Related Contribution
Findable	SOFT (improved semantic analysis supports discovery)
Accessible	LaTeX Rainbow (generates accessible PDFs)
Interoperable	EditTrans/CopyLookup (converts to standard formats)
Reusable	All three (structured output is easier to reuse)

Table 1.1: Mapping of contributions to FAIR principles.

1.5 Thesis Structure

This dissertation is organized as follows:

Chapter 2 presents LaTeX Rainbow, a framework for generating accessible PDFs from LaTeX sources. It covers the technical approach (color-coding alignment), implementation, and evaluation against PDF/UA standards.

Chapter 3 presents EditTrans and CopyLookup, two methods for efficient PDF-to-Markdown conversion. It describes the hybrid copy-generate approach, experimental results on acceleration and quality, and comparisons with baseline methods.

Chapter 4 presents the SOFT framework for citation classification. It analyzes limitations of existing schemas, introduces the orthogonal decomposition, and evaluates the framework on benchmark datasets.

Chapter 5 discusses the contributions as an integrated system, examines their relationship to ongoing efforts like the LaTeX Tagged PDF project, and addresses limitations.

Chapter 6 summarizes the contributions, reflects on the research journey, and identifies directions for future work.

2 LaTeX Rainbow: Generating Inherently Accessible Scholarly PDFs

The previous chapter described the widespread absence of structural information in scholarly PDFs, which harms readers who rely on assistive technologies and limits computational document analysis. This chapter addresses the problem at the point of creation by introducing LaTeX Rainbow, a framework that was initially developed as a LaTeX-to-PDF annotation tool for document layout analysis. Subsequent work showed that the same alignment mechanism can support accessibility, and this chapter focuses on that connection by showing how LaTeX sources can produce “born-accessible” scholarly PDFs.

Before describing the framework, this chapter first reviews existing approaches and their limits (§2.3), then explains the technical foundations that guide the framework’s design (§2.4). It then presents the LaTeX Rainbow method (§2.5), implementation (§2.6), and evaluation (§2.7), concluding with a summary of contributions (§2.8). This chapter builds on published work [28, 25].

2.1 Thesis Integration: Outputs, Dataset, and Reuse

This chapter presents a tagging method and a set of reusable artifacts. Chapter 3 uses these artifacts in its downstream pipeline, and the tagging and alignment steps also provide supervision signals and evaluation targets that make large-scale PDF-to-structured-text experiments easier to run and audit.

2.1.1 What This Chapter Produces

LaTeX Rainbow produces four outputs:

-
- **A tagging pipeline for scholarly LaTeX sources** that compiles with standard pdfTeX (PDF 1.7) and outputs structurally annotated PDFs.
 - **A deterministic alignment signal** that links LaTeX semantics to PDF glyphs using controlled color injection.
 - **An open-source implementation** for reproducibility and extension by the community.
 - **A redistributable fine-grained arXiv supervision dataset (annotations only)** derived from the pipeline for training and evaluating downstream document transformation models.

2.1.2 A Copyright-Unrestricted Fine-Grained arXiv Dataset

This pipeline also yields a large fine-grained dataset. The dataset complies with copyright restrictions because it does not include paper PDFs or verbatim full text [25].

For each processed arXiv submission, the pipeline outputs:

- **Span/token-level structure labels** (e.g., paragraph, heading, list item, formula, caption) derived from LaTeX semantics via the injected color channel.
- **Layout and rendering features** (page index, bounding boxes, reading-order cues, font/style signals, and the color IDs) extracted from the compiled PDF.
- **Document identifiers** (arXiv IDs, version metadata, and license metadata where available) so users can fetch originals from arXiv if needed.

Redistribution principle. The released dataset contains annotations and layout features plus arXiv identifiers, but does not redistribute original paper PDFs or verbatim full text content.¹ This design supports open redistribution and long-term maintenance while still enabling model training and evaluation.

¹<https://arxiv.org/licenses/>

2.1.3 The Reuse of These Outputs

Chapter 3 transforms legacy PDFs into structured Markdown at scale. It uses the dataset from this chapter for **supervision and evaluation**:

- **Supervision for “copyable vs. generative” decisions:** labels and layout cues help train and evaluate modules that decide what to copy and what to generate (e.g., formulas, tables, captions).
- **Reading-order and structure evaluation:** structure labels and layout features support scalable metrics that go beyond surface string matching.
- **A shared benchmark across chapters:** a consistent arXiv-derived dataset gives a stable target, so changes in Chapter 3 are easier to interpret.

2.2 Publication Context and Author Contributions

This chapter integrates and extends two peer-reviewed papers, included in full in the appendix: WIESP 2023 [28] and TUG 2024 [25]. WIESP 2023 introduced LaTeX Rainbow as a semantic and layout annotation framework with the main goal of creating fine-grained supervision data for document understanding. TUG 2024 then focused on accessibility, showing that the same alignment mechanism can support PDF accessibility and cross-format reuse. This chapter adds more background on PDF tagging and the constraints of pdfTeX/PDF 1.7, while linking the artifacts (including the arXiv dataset) to the experiments in Chapter 3.

Author contributions. In both publications, the author was the main contributor, responsible for proposing the key ideas, implementing the system, designing and running experiments, preparing datasets, and leading the writing.

2.3 The LaTeX Accessibility Landscape

Chapter 1 introduced the broad problem of inaccessible PDFs; this section examines the LaTeX side in more detail. LaTeX is the standard tool for fields with heavy mathematics, such as physics and computer science, where more than 90% of math-heavy papers use

LaTeX. Progress in LaTeX accessibility can therefore improve a large share of the scientific record.

LaTeX is promising for accessibility because it encodes structure, yet challenging for the same reason. Authors mark logical elements with commands like `\section`, `\theorem`, and `\begin{equation}`, and in theory this semantic information could flow into the PDF structure tree, enabling screen readers to navigate content by structure rather than by visual layout. In practice, standard compilation still drops most of this information, as the next subsections demonstrate.

2.3.1 Progress and Prospects of the LaTeX Tagged PDF Project

The largest ongoing effort is the LaTeX Tagged PDF Project, started by the LaTeX Project Team in late 2019 and announced publicly at TUG 2020 [31]. The project aims to make tagging automated and easy to use in mainstream LaTeX.

The project uses phases because the work is complex. Phase I (2020 to 2021) built core infrastructure by introducing a new hook management system so packages can be patched for tagging and adding PDF object handling via the `l3pdf` module, with automated tests using veraPDF and PDF/A validators [84]. Phase II (2021 to 2022) added automated paragraph tagging and support for basic elements such as headings, lists, footnotes, and tables of contents. Phase III (2023) extended support to math environments, floats, and cross-references [31]. Future phases aim to improve tables, add index support, and integrate tagging into the LaTeX kernel.

A key interface is `\DocumentMetadata`, introduced in the 2022 LaTeX release, which authors place before `\documentclass` to turn on tagging support. By 2025, the interface moved from `testphase=phase-III` to a simpler `tagging=on` option [32], lowering the barrier for authors and template maintainers.

However, major problems remain, with package compatibility being the main one. As of late 2025, incompatible packages include `axessibility`, `floatrow`, `hyperxmp`, `mdframed`, `nath`, `pdfx`, and `tabu`.² The team provides temporary patches through the `firstaid` package, but full compatibility with the wider package ecosystem will still take years.

²For the latest compatibility status, see <https://latex3.github.io/tagging-project/tagging-status/>.

2.3.2 The Persistence of pdfTeX

Any accessibility solution also depends on the TeX engine. The main engines are pdfTeX, LuaTeX, and XeTeX, and they vary in how well they support PDF tagging. This matters because pdfTeX still dominates scholarly workflows and sets the constraints for LaTeX Rainbow.

pdfTeX is the most widely used engine but was designed for visual typesetting and has no native primitives for building PDF structure trees. Ross Moore showed that accessible PDFs from LaTeX are possible, but he had to modify the pdfTeX source code by adding primitives such as `\pdfstructelem` and `\pdfstartmarkedcontent` [86, 85]. The results were strong, including fully tagged PDFs with MathML for mathematical content, but the modified engine was not standard, limiting broad deployment. Moore also warns that small mistakes in numerical identifiers can break the structure tree; in some cases, Acrobat can crash even when the PDF still renders correctly [86].

LuaTeX offers better support for accessibility through Lua hooks that give access to the typesetting process that pdfTeX does not expose, and the `luamml` package can automatically convert math to MathML [32]. LuaTeX also supports many PDF 2.0 features that matter for advanced accessibility.

Still, **pdfLaTeX remains the dominant engine in scholarly publishing** with no strong trend toward replacement:

- Major publishers (Springer Nature, IEEE, ACM, Elsevier, ACL Anthology) require submissions to compile with `pdflatex`.
- The Overleaf platform defaults to pdfLaTeX.
- The arXiv.org repository compiles most submissions with pdfLaTeX.
- Most packages and templates target pdfLaTeX, causing author concern about compatibility.
- LuaTeX compiles slower and uses more memory, which can slow iterative writing.

This reality shapes accessibility work: a solution for wide adoption must work with standard, unmodified pdfLaTeX. LaTeX Rainbow treats this constraint as a design requirement.

2.3.3 Alternative Approaches and Their Trade-offs

Other tools also try to improve LaTeX accessibility, each with different trade-offs between automation, coverage, and deployment.

The `axessibility` package focuses on mathematics [13] by embedding the LaTeX source of each formula as an `/ActualText` attribute so screen readers can access the equation content. Authors only need to load the package, making adoption easy, but the package does not create a PDF/UA document. The documentation also notes that screen readers may first read formulas incorrectly and only then read the `/ActualText` [13], resulting in a weaker experience than with structured MathML.

Ross Moore’s system sits on the other end of the spectrum, combining modified pdf-Tex primitives, a custom Perl tool for MathML conversion (`texmljoin`), and extensive LaTeX macros [87]. This system produces fully tagged PDFs with complete MathML representation; documents pass all 32 Acrobat accessibility checks and validate against PDF/UA standards. However, the workflow is complex, requiring custom engines, multiple processing passes, and deep expertise, which has limited adoption beyond Moore’s own institution.

The `tagpdf` package provides low-level tagging primitives that package authors and document designers can use to build accessibility support [106]. This foundational work underpins the LaTeX Tagged PDF project but requires substantial expertise to use directly.

Table 2.1 summarizes these approaches.

Table 2.1: Approaches to LaTeX accessibility, illustrating trade-offs between automation, coverage, and deployment requirements.

Approach	Mechanism	Strengths	Constraints
LaTeX Tagged PDF	Kernel-level integration	Comprehensive; automated	In development; package conflicts
<code>axessibility</code>	<code>ActualText</code> for formulas	Simple to deploy	Math only; not PDF/UA
Moore’s system	Modified engine + toolchain	Full MathML; PDF/UA	Custom engine required
<code>tagpdf</code>	Low-level primitives	Flexible foundation	Requires deep expertise

This landscape shows a gap. No existing approach achieves all three goals at once: (a) comprehensive PDF/UA compliance, (b) compatibility with standard pdfLaTeX, and (c) high automation with minimal author burden. The LaTeX Rainbow framework takes a step toward closing this gap.

2.4 Technical Foundations of PDF Accessibility

This section explains the PDF structures that underpin accessibility, guiding the design of LaTeX Rainbow and explaining why post-hoc tagging and remediation are difficult.

2.4.1 Compliance Target Under Practical Engine Constraints

This dissertation targets **maximal practical accessibility under standard pdfTeX**, which produces PDF 1.7. The goal is:

Under PDF 1.7 output constraints, get as close as possible to the accessibility and machine-readability benefits typically associated with PDF 2.0-era practices.

This goal leads to two points for this chapter:

- **Near-term goal (this chapter):** generate structurally well-formed PDFs under PDF 1.7. The outputs should follow PDF/UA as closely as practical. The system should tag content, preserve reading order, and provide meaningful alternatives for non-linear content.
- **Future work:** convert the PDF 1.7 outputs to PDF 2.0. Then attach richer semantic payloads, such as MathML as associated content, without changing pdfTeX-based author workflows.

2.4.2 The “Double-Tree” Architecture of Tagged PDFs

A Tagged PDF contains what Ross Moore calls a “double-tree” structure [87]. The first tree is the Pages tree. It organizes content for rendering. It starts at the `/Catalog` and includes `/Pages` and `/Page` objects for each page. Each page points to one or more

/Contents streams. These streams hold low-level drawing instructions, such as font selection, coordinate positioning, and glyph drawing.

The second tree is the Structure tree. It adds semantics on top of the visual content. Its root is `/StructTreeRoot`. Its nodes are `/StructElem` objects that represent sections, paragraphs, headings, lists, figures, tables, and formulas. The leaves reference page content through marked-content identifiers (MCIDs).

Linking these trees is technically demanding. Content on one page can belong to multiple structure elements; for example, a paragraph may contain inline math that sits in a separate `Formula` element, and a table cell can also sit inside several structure contexts. The reverse is also common: a single structure element can span pages (such as a paragraph that crosses a page break), in which case the element must reference content streams on both pages while staying one logical unit. Helper structures such as the `ParentTree` and `IDTree` manage these links.

This architecture also explains why post-hoc PDF tagging is hard: visual layout alone does not fully determine semantics, as two blocks can look identical but mean different things, and one semantic element can appear in many visual forms. Layout-based tools can help, but they still rebuild information that authors had at compile time and then lost.

2.4.3 PDF/UA: The Accessibility Standard

PDF/UA defines requirements for universally accessible PDFs. Two versions exist. **PDF/UA-1** (ISO 14289-1:2014) builds on PDF 1.7 [50]. **PDF/UA-2** (ISO 14289-2:2024) builds on PDF 2.0 [51]. This chapter targets PDF/UA-1 because standard `pdfLaTeX` produces PDF 1.7. The two versions differ in several ways. PDF/UA-2 supports MathML as associated files for formulas. It also introduces new structure elements such as `Document` and `DocumentFragment`. It allows multiple namespaces and explicit accessibility layers. Both versions aim to ensure that assistive technologies can navigate and present content reliably.

Key requirements include:

- complete tagging of all real content in logical reading order (Section 7.1);
- Unicode mapping for all fonts so text extraction works (Section 7.21.7);
- embedding of all fonts used for rendering (Section 7.21.4);
- alternative text for graphical content (Section 7.3);

-
- explicit handling of math, where formulas sit in `Formula` tags and have `Alt` attributes (Section 7.7).

Math is especially important in scholarly PDFs. Under PDF 1.7, many workflows treat formulas like graphics and rely on alternative text for a linear reading. PDF 2.0 adds MathML integration. It can attach MathML as associated files or use MathML-derived structure tags. MathML integration enables richer accessibility. Screen readers can navigate subexpressions. Braille devices can also render math more precisely.

The Matterhorn Protocol, published by the PDF Association, turns PDF/UA into 31 checkpoints with 136 failure conditions [99]. About 87 to 89 conditions can be checked automatically, and tools such as PAC (PDF Accessibility Checker) implement these checks. The rest still requires human verification. Authors often struggle to inspect tags because common viewers rarely show tagging information, even when they use tags to improve copy & paste [31], so specialized tools are typically needed to validate accessibility.

2.5 The LaTeX Rainbow Framework

With the landscape and technical basics in place, this section describes the LaTeX Rainbow framework in its accessibility form. It starts from a simple question: How can we create PDF/UA-compliant documents from LaTeX using standard pdfTeX, without engine modifications or extensive manual tagging?

2.5.1 Design Inspiration: Lessons from Document Layout Analysis

The key idea behind LaTeX Rainbow came from document layout analysis datasets, many of which use weak supervision. DocBank shows that color annotation can link LaTeX semantics to PDF tokens [66].

DocBank inserts color commands around semantic units in LaTeX (such as abstracts, sections, equations, and figures) so that when the source is compiled, the colors appear in the PDF’s content stream. DocBank then extracts text together with color and coordinate information, yielding token-level labels without manual annotation. The approach produced 500,000 pages of token-level annotated data from arXiv papers [66].

LaTeX Rainbow started with the same data goal as DocBank, using color to generate fine-grained training data. Later work used the same alignment for accessibility. In both cases, **color is a clear channel that links document semantics and visual rendering**.

This choice has a practical benefit: standard pdfTeX already supports color well, and authors use color commands routinely for typesetting. LaTeX Rainbow can therefore pass semantic hints through compilation without new primitives or engine changes.

2.5.2 The Color-Coding Alignment Mechanism

The core mechanism of LaTeX Rainbow assigns one unique color to each semantic role. Roles include section headings, paragraphs, list items, equations, and figure captions. The mapping is one-to-one, so the system can invert it later. The colors also need enough precision to avoid collisions. With three decimal places per RGB channel, we get about 10^9 distinct values. That is more than enough for document structure labeling.

The process works as follows:

1. **Parsing.** The system parses the LaTeX source and identifies semantic constructs, such as section commands, paragraphs, lists, math, floats, and cross-references.
2. **Color injection.** It wraps each construct with a color command. It preserves the original LaTeX structure and adds a color channel.
3. **Compilation.** Standard pdfTeX compiles the modified source. The PDF now carries the color IDs in its content stream.
4. **Extraction.** The system parses the compiled PDF and extracts text segments with their colors, coordinates, and font information.
5. **Structure reconstruction.** The system maps colors back to semantic roles and constructs the PDF structure tree.

A key design constraint is **visual non-interference**: the injected colors must not change how the document looks. LaTeX Rainbow uses a dual-compilation strategy where the first compilation produces a colored PDF for coordinate extraction and encoding semantic roles, and the second compilation produces a standard black-text PDF for final output. This approach ensures visual non-interference while preserving alignment information.

2.5.3 Mathematical Content Handling

Math needs extra care. It is central to scientific communication and it is still hard for many assistive tools.

Practical constraint. Standard pdfTeX output is PDF 1.7. This limits which PDF-native mechanisms can be used reliably across viewers and validators.

Best-effort strategy under PDF 1.7. LaTeX Rainbow focuses on robust, widely supported mechanisms:

- **Structural containment:** each mathematical expression is wrapped in a `Formula` structure element in the structure tree.
- **Accessible alternatives:** math content is accompanied by a machine-readable alternative, such as a linearized representation, so assistive technologies can read it even when MathML attachment is not reliable under PDF 1.7.

Towards PDF 2.0. A natural next step is to convert the produced PDFs to PDF 2.0 and attach MathML as associated content. Tools such as LaTeXXML already convert LaTeX to HTML with MathML.³ This provides a path to richer PDF 2.0 accessibility. Screen readers could navigate subexpressions. Braille devices could render math precisely. The authoring workflow would remain unchanged.

2.6 Implementation

The framework is implemented as an open-source tool for reproducibility and reuse [28, 25].

The LaTeX parser covers constructs common in scholarly writing, supporting sectioning commands at all levels, list environments, figure and table floats with captions, and math environments from both core LaTeX and `amsmath`. The design is extensible, allowing new constructs to be added through rules without modifying core logic.

Color injection uses the `xcolor` package, which is available in modern LaTeX distributions. The injection step preserves nesting and document structure while avoiding coloring content that should stay unchanged, such as image includes.

³See <https://dlmf.nist.gov/LaTeXML/>.

Post-processing parses the compiled PDF and constructs the structure tree in accordance with PDF/UA requirements. Role mapping translates document-specific categories into standard PDF structure types, and the system links content across pages by using coordinates to keep one structure element intact when its content spans a page boundary.

2.7 Evaluation

The evaluation covers standards compliance and scalability.

Compliance testing uses PAC (PDF Accessibility Checker) and the automated checks from the Matterhorn Protocol. Generated documents pass most automated checks. Manual inspection also confirms correct reading order, appropriate alternatives for figures and equations, and a valid heading hierarchy. In contrast, PDFs produced by conventional LaTeX compilation fail the structure-related checks because they contain no structure tree to evaluate.

Scalability testing applies the framework to a large corpus of papers from arXiv [25]. The pipeline runs consistently across diverse document types and disciplinary conventions. The tests also surface edge cases, such as unusual package usage, deeply nested environments, and documents mixing multiple languages. Processing times are suitable for batch workflows in institutional repository contexts.

On comparing against partially maintained systems. A practical complication in this space is that several research prototypes and community tools are hard to execute today because many are no longer maintained or have become incompatible with modern TeX distributions. Some are also not end-to-end deployable for large-scale experiments. In these cases, this chapter emphasizes auditable standards-based evaluation (PAC/Matterhorn checkpoints plus targeted manual inspection) and capability-based comparison (what each approach can guarantee by design), rather than fragile runtime benchmarks. When direct execution of a baseline is infeasible, comparisons rely on published features, stated constraints (e.g., engine modifications), and the scope of supported structures.

Table 2.2 positions the framework relative to alternative approaches.

Table 2.2: Feature comparison of accessibility approaches for LaTeX documents.

Feature	Rainbow	Tagged PDF	axessibility	Moore	Standard
PDF/UA compliance	✓	✓	–	✓	–
Standard pdfTeX	✓	✓	✓	–	✓
Math accessibility	MathML	MathML	ActualText	MathML	–
Automation	High	High	High	Low	N/A
Current status	Available	Developing	Available	Prototype	Baseline

2.8 Discussion and Contributions

2.8.1 Positioning and Complementarity

LaTeX Rainbow fills a distinct role in the accessibility landscape. The LaTeX Tagged PDF project aims for deep kernel integration, but that work takes years. Moore’s system achieves full tagging but requires engine modifications. The `axessibility` package is easy to deploy but only improves math. LaTeX Rainbow provides an immediately usable approach that works with current LaTeX installations and targets broad PDF/UA-1 compliance.

These approaches are also complementary. The LaTeX Tagged PDF project is the long-term goal since it integrates accessibility into LaTeX itself, but the project still needs to resolve many package compatibility issues. LaTeX Rainbow serves as a bridge in the meantime: documents processed today can benefit from accessibility, and later migration to native tagging should be easier. For a broader ecosystem-level synthesis, see Chapter 5.

2.8.2 Limitations

Scope limitations. Highly customized document classes may interfere with parsing or color injection. Complex tables with merged cells still need heuristics. Indexes and glossaries are not yet supported. They involve non-linear referential structures. Future work will extend the parser to support more packages.

Threats to validity. Automated checkers cover only a subset of accessibility requirements. Passing PAC/Matterhorn does not guarantee the same user experience across assistive technologies. Manual inspection mitigates this risk, but it cannot cover all edge cases at

scale. The pipeline also reconstructs structure from rendered output, so unusual layouts and package-level side effects can change PDF content streams in unexpected ways.

Maintenance and ecosystem risks. The TeX ecosystem evolves quickly. Accessibility support depends on upstream classes and packages. The color-channel mechanism is robust under pdfLaTeX. Long-term sustainability still requires ongoing maintenance [128]. Chapter 5 provides a broader appraisal of long-term maintenance.

Visual non-interference. The color-based approach should not change the document’s appearance. In rare cases, a failed normalization step could leave visible artifacts. Future work can explore other information-carrying channels and further investigate post-processing from PDF 1.7 to PDF 2.0 as described in §2.4.1.

2.8.3 Implications for Scholarly Communication

LaTeX Rainbow shows that high-quality typesetting and accessibility can coexist: a PDF can be print-ready and born-accessible at the same time.

The structure that LaTeX Rainbow embeds in PDFs also helps beyond accessibility by making text extraction more reliable and improving semantic parsing and automated analysis. These structured outputs provide better inputs for downstream experiments, including the transformation and analysis tasks in later chapters.

The broader implication is that accessibility is not only a compliance requirement but also enables richer document intelligence. When documents are treated as semantic structures rather than visual images, both human readers and computational systems can work with them more effectively.

3 Efficient PDF-to-Structured-Markdown Transformation using Hybrid Approaches

The previous chapter addressed the creation of “born-accessible” documents, but a large archive of existing scholarly work remains available only as unstructured PDFs. This chapter addresses large-scale conversion of such legacy documents by introducing two hybrid approaches, CopyLookup and EditTrans, that reduce the computational cost of end-to-end document conversion models.

Both methods combine direct text extraction with selective generation. CopyLookup accelerates decoding by using PDF-extracted text as draft tokens for model verification. EditTrans copies plain text and invokes generation only for complex elements such as formulas and tables. The core content of this chapter draws upon the author’s published work in [24] and [26].

3.1 The Need for Scalable PDF Conversion

End-to-end Vision-Language Models (VLMs) such as Nougat [10] improve PDF-to-markup conversion by generating structured text directly from document images. Their main limitation for large-scale remediation is computational cost.

Current end-to-end models exhibit an “efficiency gap” because they treat every page as a pure image-to-text task and regenerate the entire content token by token. This approach is computationally expensive, leading to high latency: processing a single page with Nougat requires approximately 6 seconds on average on a high-end GPU, while larger models like Qwen2-VL [124] can require over two minutes per page. Such latency may be acceptable for a single document but becomes prohibitive at the scale of large archives such as arXiv. Full-text regeneration therefore limits practical throughput in large-scale conversion settings.

This inefficiency stems from a fundamental observation: the vast majority of content in a typical academic PDF is plain text paragraphs that could be directly copied rather than regenerated. Only complex elements such as mathematical formulas, tables, and figure captions require the generative capabilities of a VLM. This insight motivates the hybrid strategies presented in this chapter, which bridge the gap between the speed of traditional extraction pipelines and the quality of end-to-end neural models.

3.2 Technical Foundations

This section traces the relevant technological landscape, covering the evolution of Vision-Language Models, the progression from traditional OCR to neural document conversion, speculative decoding techniques, and text editing models.

3.2.1 The Evolution of Vision-Language Models

The integration of visual and textual understanding has been a long-standing goal in artificial intelligence. The modern era of VLMs began with large-scale contrastive pre-training: CLIP [104] and ALIGN [52] demonstrated that training on hundreds of millions of image-text pairs enabled zero-shot transfer to numerous downstream tasks, while ViLT [58] simplified architectures by processing raw image patches directly with transformers, building on the Vision Transformer (ViT) paradigm [23].

The generative paradigm was advanced by BLIP [65] (bootstrapped caption filtering), Flamingo [2] (few-shot learning via interleaved sequences), and LLaVA [72] (instruction-following through vision-language projection). These models differ in how they integrate visual information: LLaVA uses cross-attention to inject image encoder outputs, while Qwen-VL directly concatenates image patch tokens with text tokens. Open-source multilingual models including Qwen [6, 124, 7, 134] and Llama 3 [29] have democratized access to powerful vision-language capabilities.

Recent developments (2024 to 2025) have introduced several advances: Mixture-of-Experts architectures such as DeepSeek-VL2 [130] and InternVL2.5 [16] improve efficiency; compact yet powerful models such as MiniCPM-V [136] and Phi-3 Vision [1] enable edge deployment; long-context specialists such as InternLM-XComposer2.5 [139] support 96K

tokens; and specialized OCR-focused VLMs have emerged, including Nougat [10], Kosmos-2.5 [76], GOT-OCR 2.0 [126], Mistral OCR,¹ PaddleOCR-VL [21], DeepSeek-OCR [125], SmolDocling [91] (256M parameters), and DocTR [70].

For document-specific understanding, the LayoutLM family [133, 132, 48] pioneered the integration of textual content, visual features, and 2D positional information. Related architectures include ERNIE-Layout [100] (reading order prediction), VILA [108], DiT [64], LiLT [122], and DLAFormer [123]. These document-specific architectures underpin the layout-aware classification modules used by CopyLookup and EditTrans.

3.2.2 From OCR to Neural Document Conversion

The task of extracting structured content from document images has evolved through several distinct paradigms. Traditional OCR systems like Tesseract [110] recognize characters but lack document semantics, identifying *what* characters appear but not *how* those characters relate to document structure.

Pipeline-based approaches address this limitation by combining OCR with Document Layout Analysis. GROBID [41] extracts structured XML from scientific publications, while recent systems like Marker,² MinerU [121, 94], and Docling [4] implement detection-extraction-conversion pipelines. A common limitation is cascading error, where mistakes in layout detection propagate through later stages and degrade the final output.

End-to-end neural approaches eliminate this pipeline fragility. Donut [56] pioneered treating document understanding as visual question-answering, and Nougat [10] adapted this for academic documents with $\text{\LaTeX}$ -formatted mathematics. Subsequent models include Kosmos-2.5 [76], LOCR [115] (location-guided generation to reduce hallucinations), DocOwl 1.5 [44], μ gat [103], OlmOCR [102], and Dolphin [30]. Architecturally, these models have evolved from Swin Transformer [74] encoders with mBART [73] decoders toward more efficient designs: GOT-OCR 2.0 uses VitDet with a Qwen decoder supporting 8K context, while DeepSeek-OCR [125] introduces context optical compression via a Mixture-of-Experts decoder, achieving 10:1 token reduction. The OmniDocBench benchmark [98] provides standardized evaluation across these approaches.

¹<https://mistral.ai/news/mistral-ocr/>

²<https://github.com/datalab-to/marker>

Despite their quality advantages, these models share a fundamental efficiency limitation: they generate text token by token, even for content that could be directly extracted. This observation motivates the hybrid approaches presented in this chapter.

3.2.3 Speculative Decoding

The computational inefficiency of autoregressive decoding in Large Language Models has prompted a family of acceleration techniques collectively known as speculative decoding. These techniques share a common insight: verification is cheaper than generation. For comprehensive surveys, see [131, 46].

Speculative decoding was introduced independently by Leviathan et al. [63] and Chen et al. [15], employing a draft-then-verify paradigm. A smaller, faster draft model generates candidate tokens, and the larger target model verifies these candidates in parallel. Because Transformer inference can evaluate multiple tokens simultaneously, correctly drafted tokens are accepted “for free,” yielding acceleration without quality degradation. Kim et al. [57] introduced the Big Little Decoder (BiLD) framework, which coordinates small and large models through “fallback” and “rollback” policies and dynamically switches between models based on prediction confidence.

Retrieval-based methods. Draft sources have evolved beyond neural models. Prompt Lookup Decoding³ exploits n-gram overlap between prompts and outputs, and LLMA [135] extends this to reference documents. He et al. [43] introduced REST, which replaces draft models entirely with retrieval from a datastore using suffix arrays to find matching continuations. This technique is well-suited to document processing, where source text can serve as the retrieval corpus. SAM-Decoding [47] uses suffix automata for $O(1)$ average-case retrieval complexity, significantly outperforming n-gram matching approaches, while SuffixDecoding [96] uses suffix trees to cache long token sequences and achieves extreme speedups for agentic AI applications with repetitive patterns.

Learned acceleration methods. Neural approaches to draft generation include Medusa [11] (parallel prediction heads), EAGLE [68, 67] (feature-based uncertainty estimation with dynamic tree construction), and Lookahead Decoding [34] (Jacobi iteration-inspired parallel generation). Zhou et al. [140] showed that knowledge distillation can better align draft and target model distributions, improving acceptance rates by 10 to 45%.

³<https://github.com/apoorvumang/prompt-lookup-decoding>

Multi-token prediction. An alternative paradigm trains models to predict multiple future tokens simultaneously using auxiliary prediction heads. Gloeckle et al. [39] demonstrated that this approach improves both training efficiency and downstream performance, and production systems including DeepSeek-V3 [22] have adopted it. FastMTP [12] extends this by fine-tuning a single shared MTP head for recursive multi-step drafting, achieving $2.03\times$ speedup through self-distillation and vocabulary compression.

System-level optimizations. Staged speculative decoding [112] enables tree-structured speculation across multiple pipeline stages, and PipeSpec [82] introduces hierarchical pipelining that breaks sequential dependencies between draft and verification phases to enable asynchronous execution across multiple models with up to $2.54\times$ speedup. Meta’s production deployment [117] demonstrates that EAGLE-based speculation can scale to serve Llama models, achieving 10% latency improvement over open-source implementations at batch sizes exceeding 100 requests. Additional advances include speculative diffusion decoding [17] and heterogeneous vocabulary speculation [119], with the latter now integrated into Hugging Face Transformers.

The speculative decoding paradigm is particularly well-suited to PDF-to-Markdown conversion because the source PDF already contains most of the text that will appear in the output. The retrieval-based approaches (REST, SAM-Decoding, SuffixDecoding) are especially relevant, as they demonstrate how external data sources can serve as draft providers. The CopyLookup approach (Section 3.3) operationalizes this insight by using PDF-extracted text as a draft source for VLM verification, combining retrieval-based speculation with document conversion.

3.2.4 Text Editing Models

Speculative decoding accelerates generation, but an alternative paradigm questions whether generation is necessary at all. Text editing models propose that many NLP tasks are more naturally framed as modifications to existing text [81].

Tagging-based approaches predict edit operations per token. LaserTagger [80] pioneered this with KEEP/DELETE/INSERT operations, FELIX [79] added reordering capabilities, and PIE [5] and GECToR [97] applied iterative refinement to grammatical error correction. Semi-autoregressive methods such as Edit5 [78], Seq2Edits [114], and Locate&Edit [111] combine tagging with generation for greater flexibility.

The Fill-in-the-Middle (FIM) paradigm [9] teaches models to predict missing content given both left and right context. Originally developed for code completion, FIM has proven valuable for editing tasks in systems like DeepSeek-Coder [42].

The text editing paradigm provides the conceptual foundation for EditTrans (Section 3.4). EditTrans identifies which portions require generation (formulas, tables) and which can be directly copied (plain text). Rather than generating entire documents, EditTrans copies where possible and generates where necessary. The FIM capability enables the VLM to generate missing sections while conditioning on surrounding context to maintain coherence with copied text.

3.3 CopyLookup: Accelerating VLM Decoding via PDF-Sourced Drafts

The CopyLookup approach operationalizes the speculative decoding paradigm for PDF-to-Markdown conversion. It uses text extracted from the source PDF as a draft source for VLM verification.

3.3.1 Design Rationale

CopyLookup is designed to accelerate decoding without changing model behavior. It provides candidate tokens that the VLM verifies rather than generates, while keeping the VLM as the authority for content and formatting decisions. This design ensures that CopyLookup produces *identical* output to standard VLM inference; only the computational path differs.

The approach builds on a key observation: academic PDF pages typically contain 70 to 90% plain text that will appear nearly verbatim in the Markdown output. Only mathematical formulas, table structures, and markup formatting require generation. By enabling the VLM to “copy” text spans through verification rather than generation, CopyLookup can reduce the number of forward passes required.

3.3.2 Methodology

The CopyLookup system comprises two main components: Copyable Text Identification and an enhanced Candidate Generation mechanism [24].

Copyable Text Identification. Not all text extracted from a PDF is suitable for copying: mathematical formulas require $\text{\LaTeX}$ encoding and would produce incorrect Markdown if copied directly, while page numbers, headers, and footers should typically be excluded from the output. To address this, CopyLookup employs a lightweight classifier fine-tuned from ERNIE-Layout [100] using LoRA [45]. This classifier predicts whether each text span should be marked as KEEP (copyable) or DELETE (not copyable), operating at the span level with PyMuPDF⁴ to extract text with bounding box coordinates. The classifier achieves F1 scores exceeding 0.98 with inference times under 30 milliseconds per page.

Candidate Generation with Topping. During VLM decoding, CopyLookup searches the filtered text spans for n -grams matching recent output tokens. When a match is found, subsequent tokens from that span are proposed as candidates for verification. A critical innovation is the “topping” mechanism: when candidates are accepted and the model progresses within a span, that span is moved to the front of the search list. This approach maintains synchronization between the candidate source and the current generation position, addressing the challenge that PDF reading order may not perfectly match the generation order.

Verification and acceptance. The VLM verifies candidate sequences through a single forward pass and accepts tokens that match its predicted distribution. This verification is mathematically equivalent to standard autoregressive generation: accepted tokens are exactly those the model would have generated anyway. The only difference is computational efficiency, as verifying k tokens requires one forward pass rather than k passes.

3.3.3 Experimental Validation

CopyLookup was evaluated on a corpus derived from arXiv papers, with testing covering acceleration across multiple backbone models including Nougat (349M parameters), Kosmos-2.5 (1.37B), Llama-3.2-Vision (10.6B), and Qwen2-VL (12.6B with AWQ quantization [71]). Tests spanned documents from general arXiv submissions as well as domain-specific collections in Economics and Quantum Physics.

⁴<https://pymupdf.readthedocs.io/>

Results demonstrated consistent speedups across all models and domains. Larger models benefited most from the acceleration, with Qwen2-VL achieving up to $1.70\times$ speedup on Quantum Physics documents. Ablation studies confirmed that output quality was perfectly preserved: CopyLookup produces bit-identical results to standard decoding, as guaranteed by the verification mechanism.

3.4 EditTrans: Hybrid Copy-Generate for PDF-to-Markdown

CopyLookup accelerates the VLM’s internal decoding process. EditTrans goes further by decomposing the task to avoid generation for plain text.

3.4.1 Design Rationale

EditTrans is designed to use generation only where it is needed. It routes plain text to a direct copy path and reserves the VLM for complex structures. This leads to a hybrid architecture where routing decisions are determined by content complexity: simple text is copied directly, while formulas, tables, and other complex regions are generated by the VLM.

The approach draws inspiration from text editing models, reconceptualizing PDF conversion as an editing task rather than a generation task. The “source text” is the plain content extracted from the PDF, and the “edits” are the insertions of $\text{\LaTeX}$ -formatted formulas and table markup at appropriate positions.

3.4.2 Methodology

EditTrans operates through a three-stage pipeline: Editable Text Identification, Edit Queue Building, and Edit Actions Execution [26].

Editable Text Identification. Similar to CopyLookup, EditTrans employs an ERNIE-Layout-based classifier to categorize text spans, but uses a three-class scheme: KEEP (copy directly), DELETE (exclude from output), and INSERT_LEFT (generate content before this span). The INSERT_LEFT label indicates positions where the VLM must generate complex content, typically mathematical expressions that precede the next copyable span.

Edit Queue Building. The classified spans are organized into an edit queue that interleaves copy operations and generation triggers. When traversing spans in reading order, KEEP spans contribute their text directly, DELETE spans are skipped, and INSERT_LEFT spans insert a generation trigger followed by the span’s text as a stopping criterion. This queue structure encodes the complete transformation strategy.

Edit Actions Execution. The queue is processed sequentially. For copy operations, tokenized text is appended directly to the output. For generation triggers, the VLM is invoked using the Fill-in-the-Middle paradigm [9]: the model receives the preceding context and is instructed to generate content until matching the beginning of the next copyable span. This FIM approach allows the VLM to produce formulas and tables that naturally integrate with surrounding copied content.

Error robustness. An Edit Actions Skip mechanism provides resilience against classification errors. If the VLM’s generation naturally produces content matching a subsequent span, that span’s copy operation is skipped to prevent duplication. This self-correction capability ensures graceful degradation when the classifier makes mistakes.

3.4.3 Experimental Validation

EditTrans was evaluated on the same arXiv-derived corpus, with backbone models including Nougat, Kosmos-2.5, and OlmOCR. Across all configurations, EditTrans achieved substantial latency reductions of up to 44.5% for Kosmos-2.5 on Quantum Physics documents, corresponding to 43.8% fewer generation steps.

Quality metrics showed that EditTrans not only maintained but often slightly improved output quality. One explanation is that copied text is typically more accurate than regenerated text, which can include transcription errors. EditTrans therefore provides efficiency gains while maintaining output fidelity.

3.5 Comparative Analysis and Discussion

CopyLookup and EditTrans represent complementary approaches to the same underlying problem, offering different trade-offs suitable for different deployment scenarios.

Output fidelity. CopyLookup guarantees bit-identical output to standard VLM inference, making it suitable when exact reproduction is required. EditTrans may produce slightly

different output due to the hybrid copy-generate mechanism, but empirical results suggest this difference is typically beneficial.

Acceleration mechanisms. CopyLookup accelerates the VLM’s internal decoding through speculative verification, while EditTrans reduces the VLM’s workload by delegating simple content to direct copying. CopyLookup provides moderate but guaranteed acceleration; EditTrans provides larger acceleration for text-heavy documents but less benefit for formula-dense content.

Implementation complexity. CopyLookup integrates cleanly with existing VLM inference frameworks using the Hugging Face assisted generation API.⁵ EditTrans requires more substantial modifications to the inference pipeline, including queue management and FIM-based generation.

Implications for accessibility. Both approaches improve the practical feasibility of large-scale PDF remediation. By reducing per-page processing time by 40 to 50%, these methods lower the cost of converting large archives. When combined with the LaTeX Rainbow framework from Chapter 2, faster conversion can support larger datasets for analysis and model development, which in turn can improve conversion quality and efficiency.

Limitations. Both approaches depend on accurate upstream components: CopyLookup requires correct span classification, and EditTrans additionally depends on reading order detection. Documents with highly non-linear layouts (multi-column formats, complex floats, and marginal notes) pose challenges for both systems. Future work could explore integration with more sophisticated layout understanding models such as LOCR [115] and Dolphin [30].

3.6 Conclusions

This chapter presented CopyLookup and EditTrans, two hybrid approaches that bridge traditional text extraction and end-to-end neural generation to enable efficient PDF-to-Markdown conversion. By recognizing that most PDF content can be copied rather than regenerated, these methods achieve acceleration of $1.5\times$ to $1.7\times$ while maintaining or improving output quality.

The key insight underlying both approaches is that VLM capabilities should be applied selectively: generative power is essential for complex structures but wasteful for plain

⁵<https://huggingface.co/blog/assisted-generation>



text. This selective application offers a practical pathway to processing millions of legacy PDF documents that currently remain inaccessible.

Together with the born-accessible document generation capabilities presented in Chapter 2, these efficient transformation methods provide comprehensive coverage of the scholarly document accessibility challenge, preventing future accessibility barriers while remediating existing ones at scale.

4 SOFT: A Semantically Orthogonal Framework for Nuanced Citation Analysis

The preceding chapters established methods for producing and transforming scholarly documents into structured, accessible formats. This chapter examines how such structure can support downstream citation analysis. It introduces the Semantically Orthogonal Framework for Citation Classification (SOFT), a taxonomy that separates citation intent from cited content type. The content herein is based on the author’s work in [27].

Before presenting the SOFT framework, it is essential to understand the historical trajectory of citation analysis, spanning from its origins in information science to the current state-of-the-art in computational linguistics. This context reveals both the achievements and persistent limitations that motivate the design of SOFT.

4.1 The Evolution of Citation Analysis

4.1.1 Origins: From Citation Counts to Indices

The systematic study of citations began with Eugene Garfield’s 1955 article, “Citation Indexes for Science” [37], which proposed indexing scientific literature by tracking citation relationships between papers. This approach lets researchers trace how ideas develop over time. By 1972, Garfield implemented this idea through the Science Citation Index (SCI) and introduced the *Impact Factor* as a metric for journal influence [36]. Despite its known limitations, this metric remains widely used in research evaluation.

These foundational contributions established citation counting as a cornerstone of bibliometrics but also embedded a critical assumption that would constrain the field for decades:

that citations could be meaningfully aggregated without regard to their qualitative differences. Under this assumption, a citation that enthusiastically extends prior work was treated identically to one that critiques it.

4.1.2 Early Classification Frameworks: Recognizing Heterogeneity

Moravcsik and Murugesan first systematically addressed the inadequacy of treating all citations as equivalent in their 1975 publication [88]. Analyzing 30 articles in theoretical physics, they proposed a four-dimensional framework for classifying citations:

- **Conceptual vs. Operational:** Does the citing work use the *ideas* or the *methods/tools* of the cited work?
- **Organic vs. Perfunctory:** Is the citation truly necessary for understanding the citing paper, or merely a ceremonial acknowledgment?
- **Evolutionary vs. Juxtapositional:** Does the citing work build upon or merely compare against the cited work?
- **Confirmative vs. Negational:** Does the citing work support or contradict the cited work?

Their analysis revealed that approximately 40% of citations were “redundant” or “perfunctory,” casting doubt on the validity of raw citation counts as quality indicators. Two years later, Spiegel-Rösing [113] extended this work by proposing 13 citation categories and analyzing their distribution in the history of science, finding significant variation across disciplines.

These early frameworks established a fundamental principle: citation analysis must account for why researchers cite, not just that they cite. However, their manual, small-scale methodologies could not scale to the millions of citations generated annually.

4.1.3 Sociological Critiques: The Limits of Citation Analysis

While classification frameworks aimed to refine citation analysis, a parallel line of research questioned whether citations could ever be reliable indicators of intellectual influence. MacRoberts and MacRoberts [77] documented “The Negational Reference: Or the Art of Dissembling,” revealing the systematic avoidance of negative citations in scientific

literature. Their analysis identified several “dissembling” strategies, including praising work that is criticized, relegating criticism to footnotes, and avoiding direct confrontation with powerful figures in the field. The authors argued that career considerations and peer review dynamics incentivize scientists to “sweeten” or suppress criticism, rendering citation sentiment a biased indicator of actual scientific disagreement.

Greenberg’s 2009 case study provided additional evidence for this sociological perspective [40]. Analyzing a claim about a protein’s role in a disease, Greenberg identified systematic distortions: *citation bias* (selective citing of supportive studies), *citation amplification* (inflating the significance of evidence), *citation invention* (citing papers for claims they never made), and *citation diversion* (citing to work with different findings). Through these mechanisms, a claim with limited empirical support acquired “unfounded authority” in the literature.

These sociological critiques highlight two key implications for citation intent classification. First, negative or critical citations are genuinely rare, not merely difficult to detect, because the scientific reward system discourages them. Recent empirical studies confirm this pattern: an analysis of “stance” in NLP and ML papers found that only 3.9% of NLP papers and 2.3% of ML papers exhibited a negative stance toward prior work, while scite.ai’s analysis of 880 million citation statements found that less than 0.6% of citations are negative. Second, mentioning a work is fundamentally different from substantively engaging with it, and conflating these in a single “Background” or “Related Work” category obscures important distinctions.

4.1.4 The NLP Era: Automated Classification

The transition from manual to automated citation analysis began in the late 1990s when Nanba and Okumura published their work on multi-paper summarization [90]. They introduced the concept of “reference areas,” the sentences surrounding a citation that provide context, and proposed a simple three-way classification: Type B (basis/uses), Type C (comparison/critique), and Type O (other). Their key insight was that Type C citations are particularly valuable for understanding research relationships.

The seminal work for automated citation function classification came from Teufel, Sidharthan, and Tidhar in 2006 [118]. Building on Teufel’s earlier “Argumentative Zoning” framework, they proposed 12 citation categories and developed a supervised machine learning classifier using shallow and linguistically-inspired features. Their analysis revealed the extreme skewness of citation intent distributions, with approximately 60% of

citations falling into a single “Weak” (neutral/background) category while substantive categories like “Comparison” or “Criticism” were rare.

This skewness has persisted across subsequent datasets. The ACL-ARC dataset [55] simplified the schema to 6 categories (Background, Motivation, Uses, Extension, Comparison/Contrast, and Future) and found that 63% of citations in NLP papers serve a Background function. The SciCite dataset [18] further condensed this to 3 categories (Background, Method, and Result Comparison) while also experimenting with “structural scaffolds,” a multitask learning approach that jointly predicts citation intent and section title.

More recently, the scite.ai platform [93] has demonstrated the industrial viability of citation intent classification. Analyzing over 880 million citation statements using a SciBERT-based classifier, the platform categorizes citations as *Supporting* (providing evidence for the cited claim), *Contrasting* (providing evidence against), or *Mentioning* (neutral reference). Their reported distribution of 92.6% Mentioning, 6.5% Supporting, and 0.8% Contrasting underscores the extreme rarity of substantive citations.

Figure 4.1 illustrates this historical trajectory, highlighting the evolution from manual qualitative frameworks to automated classification systems.

4.1.5 Summary: Progress and Persistent Gaps

The evolution of citation analysis has progressed from Garfield’s quantitative indices through qualitative classification frameworks to automated NLP systems. Along this trajectory, three persistent themes emerge.

1. **Not all citations are equal.** From Moravcsik’s “perfunctory” citations to Valenzuela’s finding that only 14.6% of citations are “meaningful” [120], research consistently shows that most citations are ceremonial rather than substantive.
2. **Negative citations are systematically rare and often hidden.** The sociological work of MacRoberts and Greenberg explains why. Scientific norms and career incentives discourage explicit criticism.
3. **Existing schemas conflate distinct dimensions.** As will be shown in the next section, frameworks from Teufel (2006) through SciCite (2019) merge “what” is cited with “why” it is cited. This creates ambiguity that limits annotation consistency and model generalizability.

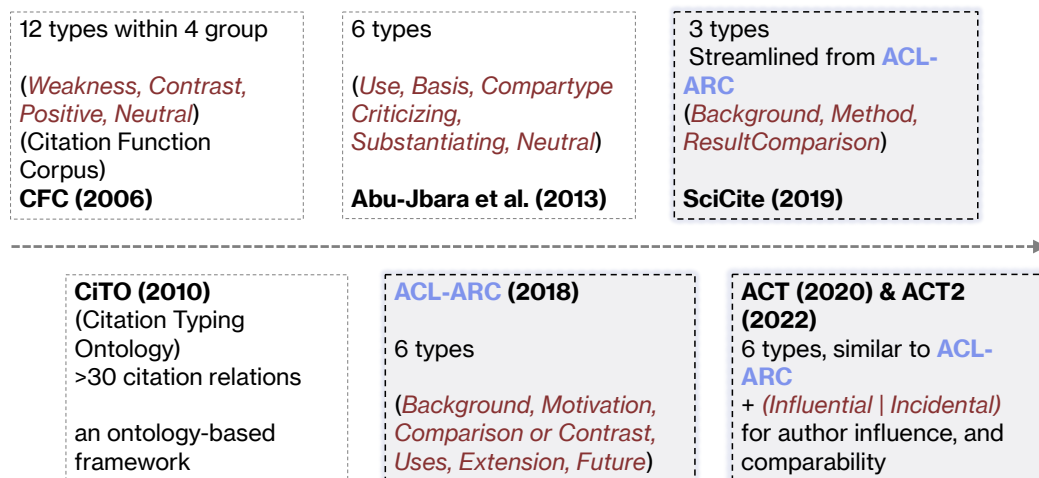


Figure 4.1: The evolution of citation classification frameworks from 1975 to 2025. Early manual frameworks (Moravcsik, Spiegel-Rösing) established the principle that citations serve different functions but could not scale. The NLP era (Teufel 2006 onwards) automated classification but inherited dimensional conflation. SOFT (2025) introduces explicit orthogonality between intent and content dimensions.

The SOFT framework addresses this third gap directly. It explicitly disentangles citation intent from cited content type.

4.2 Limitations of Existing Citation Classification Schemas

This section builds on the historical overview to examine in greater detail the specific limitations of existing schemas that motivate the design of SOFT. Traditional bibliometrics have long relied on coarse signals, with raw citation counts being the most notable example. To move beyond mere quantity, computational linguistics has developed citation intent classification schemas that capture the functional role of a citation. However, nuanced citation analysis is hindered by two factors: poorly structured input from inaccessible PDFs and inherent limitations in the schemas themselves.

A primary limitation is the conflation of citation intent and cited content type. Many existing frameworks merge the action of citing (the *why*, e.g., “to use”) with the object of

the citation (the *what*, e.g., “a dataset”) into a single, ambiguous label. This conceptual entanglement reduces annotation consistency, as annotators must make decisions with ambiguous category boundaries. The conflation can also reduce model generalizability, since the meaning of a label like “Uses” can vary substantially across scientific domains. This lack of semantic precision limits analysis of how knowledge is propagated and reused.

4.2.1 A Linguistic Analysis of Schema Design

The conflation of intent and content in existing schemas is not merely a conceptual issue but manifests as a fundamental linguistic inconsistency in the structure of category labels. A systematic analysis of label syntax across major frameworks reveals a pattern: many labels are structured as verb-noun phrases that fuse the action (what the citing author does) with the object (what aspect of the cited work is referenced) into a single classificatory unit.

Consider the foundational 13-category schema of Spiegel-Rösing [113]. Nearly every category is a verb-object compound, with examples including “Identifying methodology,” “Providing background reading,” “Criticizing previous work,” and “Substantiating claims.” Each label simultaneously encodes an *intent* (identifying, providing, criticizing, substantiating) and a *content type* (methodology, background, work, claims). This linguistic fusion creates an implicit assumption that each intent-content combination requires a distinct label, leading to taxonomic proliferation.

The same pattern persists in later NLP-era frameworks, often obscured by nominalization. The ACL-ARC schema [55] includes labels whose parts of speech are inconsistent: “Uses” is a verb, “Background” is a noun describing a static role, and “Extension” is a nominalized verb (from “extends”). This grammatical heterogeneity forces annotators to switch between two questions: “What is the author doing?” (for verb-like labels) and “What role does the cited work play?” (for noun-like labels). This switching introduces cognitive friction and reduces inter-annotator agreement.

The SciCite schema [18] provides a particularly illustrative case. Its three categories are BACKGROUND, METHOD, and RESULT COMPARISON. These categories conflate semantic dimensions in multiple ways.

- METHOD is a pure content type (what is cited), not an intent.
- BACKGROUND describes positional role in discourse, not a semantic action.

-
- **RESULT COMPARISON** is a noun-phrase that embeds both a content type (**RESULT**) and a nominalized action (**COMPARISON**).

These three labels operate on fundamentally different semantic planes. Yet they are presented as mutually exclusive alternatives in a single classification task.

The CiTO ontology [109] merits deeper examination as both a pioneering achievement and a cautionary tale in schema design. CiTO introduced several forward-thinking concepts: the FRBR hierarchy (distinguishing between a scholarly *Work*, its *Expression*, and its *Manifestation*), explicit separation of *factual* relationships (e.g., `cito:usesDataFrom`) from *rhetorical* relationships (e.g., `cito:extends`, `cito:critiques`), and organization of rhetorical relationships along a polarity axis (Positive, Negative, Neutral). These design choices reveal an implicit recognition that citation semantics involve multiple orthogonal dimensions.

However, CiTO’s attempt at exhaustive coverage led to significant practical limitations. The ontology encodes verb-object fusion through properties like `cito:usesMethodIn`, `cito:usesDataFrom`, and `cito:citesAsSourceDocument`, which conflate intent (“uses”) with content type (“method,” “data,” “source”) at the property level. The fused encoding results in a proliferation of approximately 40 citation properties, 41 subclasses of `cito:Work`, and 45 subclasses of `cito:Expression`, making the taxonomy unwieldy for practical annotation. CiTO was designed under the assumption that authors or editors would manually annotate citations in RDF format, an approach superseded by automated classification using neural language models. CiTO’s pure-verb properties (`cito:extends`, `cito:critiques`, `cito:confirms`) correspond directly to what SOFT formalizes as the Intent dimension, while its content-specific properties implicitly embed what SOFT treats as the orthogonal Content dimension. SOFT can be understood as extracting and formalizing this latent structure.

Among existing approaches, only scite.ai [93] achieves grammatical consistency. Its three categories, *Supporting*, *Contrasting*, and *Mentioning*, are all present participles (verb-ing forms) that uniformly capture the citing author’s action. However, this consistency comes at the cost of *completeness*: scite.ai provides no mechanism for distinguishing what is being supported or contrasted, whether a method, a finding, or a dataset.

Table 4.1 summarizes the linguistic structure of labels across representative frameworks, revealing the pervasiveness of dimensional conflation.

Figure 4.2 visualizes this dimensional conflation problem. Existing frameworks attempt to cover the semantic space through label proliferation, creating an ever-growing list of

Table 4.1: Linguistic analysis of citation classification schemas. V = verb, N = noun, Adj = adjective. Fused labels combine intent (verb) and content (noun) into a single category.

Framework	Example Labels	Structure	Key Problem
Spiegel-Rösing (1977)	“Identifying methodology,” “Criticizing previous work”	V+N fused	Nearly all 13 labels are verb-object compounds
Teufel et al. (2006)	“Uses” (V), “Background” (N), “CoCoGM” (V+N)	Mixed	Inconsistent parts of speech; <i>CoCoGM</i> fuses intent and content
ACL-ARC (2018)	“Uses” (V), “Background” (N), “Extension” (N from V)	Mixed	Grammatical heterogeneity forces perspective shifts during annotation
SciCite (2019)	“Background” (N), “Method” (N), “Result Comparison” (N+N)	All nouns	“Method” is pure content type; labels operate on different semantic planes
CiTO (2010)	usesMethodIn (V+N), extends (V)	Mixed	Mixture of fused and pure labels leads to ~40 properties
scite.ai (2021)	“Supporting” (V-ing), “Contrasting” (V-ing), “Mentioning” (V-ing)	All V-ing	Grammatically consistent but lacks content dimension
SOFT (proposed)	Intent: Use, Modify (V) Content: Discovery, Resource (N)	Two orthogonal dims	Separates verb (intent) from noun (content); avoids fusion

categories. In contrast, SOFT’s orthogonal design captures the same semantic distinctions through the Cartesian product of two compact dimensions.

4.2.2 The Subject-Verb-Object Paradigm

The linguistic analysis above reveals the root cause of dimensional conflation: treating citation classification as a single-axis task. A more principled approach recognizes that any citation act can be decomposed into a *subject-verb-object* (SVO) structure, as illustrated in Figure 4.3.

In this formulation:

Dimension Conflation (Traditional Labels: A_B, A_C, D_B...)	Orthogonal Separation (SOFT) (Dimension 1: {A, D} × Dimension 2: {B, C})
---	--

Figure 4.2: Dimensional conflation in existing schemas versus orthogonal separation in SOFT.

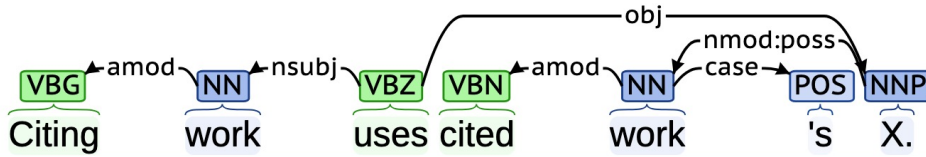


Figure 4.3: The Subject-Verb-Object paradigm for citation analysis. A citation act is decomposed into three components: the citing paper (subject, fixed), the citation intent (verb, e.g., *uses*, *modifies*, *evaluates against*), and the cited content type (object, e.g., *produced resource*, *discovery*). Example: “We evaluated our model on the benchmark dataset from [Smith 2020]” → Intent: EVALUATE AGAINST, Content: PRODUCED RESOURCE.

- The **subject** is always the citing paper, establishing a consistent perspective.
- The **verb** corresponds to the *Citation Intent*, the action performed by the citing author (e.g., *uses*, *modifies*, *evaluates against*).
- The **object** corresponds to the *Cited Content Type*, the nature of the cited contribution (e.g., a method, a discovery, a dataset).

This decomposition reflects the natural structure of language as formalized in semantic role theory and case grammar [54]. Just as a sentence’s meaning arises from the combination of a predicate and its arguments, a citation’s semantics emerge from the orthogonal combination of intent and content. By enforcing this separation, SOFT transforms citation classification from a single unwieldy taxonomy into two independent, manageable dimensions.

4.3 The SOFT Framework: Design and Rationale

This dissertation introduces the Semantically Orthogonal Framework for Citation Classification (SOFT) [27] to overcome the limitations described above. The core design principle is explicit disentanglement: citation semantics are separated into two independent, or orthogonal, dimensions, namely a **Cited Content Type** and a **Citation Intent**. This two-dimensional approach is inspired by semantic role theory, which separates the predicate (the action) from its argument (the object).

The *Cited Content Type* dimension categorizes the nature of the referenced knowledge unit (e.g., a Method, a Dataset, or a Conceptual Contribution), while the *Citation Intent* dimension characterizes the communicative act performed by the citing author (e.g., Using, Extending, or Critiquing the cited work). By separating these concerns, SOFT resolves the ambiguity of previous schemas; for example, citing a dataset as background is clearly distinguished from actively using it in an experiment. This orthogonal design leads to a more robust, interpretable, and extensible taxonomy that improves annotation reliability and supports more fine-grained, cross-disciplinary analysis of scholarly discourse.

Figure 4.4 presents the complete SOFT classification matrix, showing all possible combinations of the Intent and Content dimensions with representative examples.

4.4 Methodology: Applying SOFT to Structured Document Inputs

The performance of any semantic classification framework is intrinsically linked to the quality of its input. This dependency is clearly visible in the text extraction pipelines employed by major citation classification datasets. Table 4.2 summarizes these pipelines.

As illustrated, each dataset relies on rule-based or neural extraction tools. The limitations of these tools directly propagate into the training data. For instance, Jurgens et al. [55] explicitly noted that “less than 10% of citation contexts were corrupted by OCR errors or parsing artifacts” in their ACL-ARC corpus. This figure nonetheless introduces systematic noise into model training. Similarly, Kunnath et al. [89] reported 68 matching failures between GROBID-extracted citations and ground truth labels. Missing paragraphs and unlinked bibliography entries were among the primary causes.

The SOFT framework is designed to leverage well-structured documents that can be reliably produced by the tools presented in the preceding chapters. The outputs from LaTeX Rainbow (Chapter 2) or EditTrans/CopyLookup (Chapter 3) are pivotal in enhancing

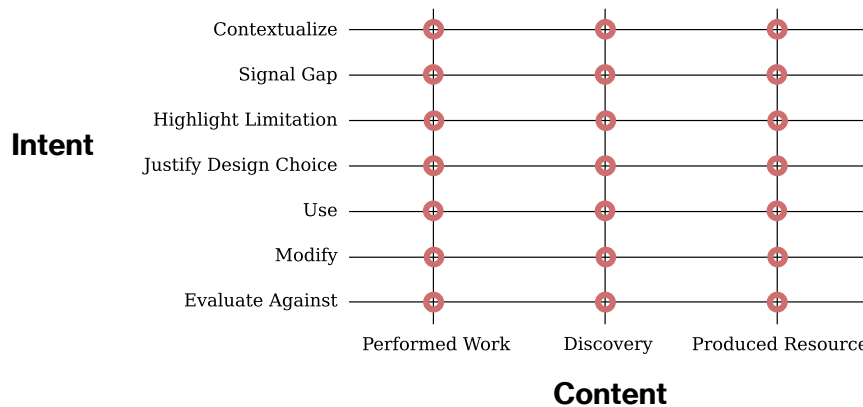


Figure 4.4: The SOFT classification matrix: 7 Citation Intent categories $\times$ 3 Cited Content Type categories. Intent categories (columns): CONTEXTUALIZE, SIGNAL GAP, HIGHLIGHT LIMITATION, JUSTIFY DESIGN CHOICE, USE, MODIFY, EVALUATE AGAINST. Content categories (rows): PERFORMED WORK (methods, approaches), DISCOVERY (findings, claims), PRODUCED RESOURCE (datasets, tools, models). Each cell represents a semantically distinct citation type that can be expressed through orthogonal composition.

SOFT’s performance, as clean, structured Markdown or XML provides unambiguous citation contexts free from the OCR errors and parsing artifacts that plague analyses based on raw PDF text.

The methodology for applying SOFT involves a standard NLP classification pipeline, including feature engineering to extract the citation context and relevant metadata from the structured document, as well as a model architecture (typically a fine-tuned language model) for the classification task. Because SOFT has two independent dimensions, classification can be performed in two ways: multi-task learning or two separate classification tasks. High-quality structured inputs simplify this process by reducing the impact of extraction noise and structural ambiguity.

Table 4.2: Text extraction tools used by major citation classification datasets. The quality and consistency of extracted citation contexts directly impacts classification performance.

Dataset	Source	Extraction Tool	Known Issues
ACL-ARC	ACL Anthology	ParsCit [20]	OCR errors, malformed contexts
SciCite	Semantic Scholar	ScienceParse ¹	Limited section info
ACT2 [89]	CORE [60]	GROBID + ScienceParse	Missing paragraphs, unlinked refs
CC30k [95]	Semantic Scholar	S2 Graph API [59]	Citation mark ambiguity
S2ORC [75]	Open Access	GROBID + LaTeXML ²	PDF/LaTeX inconsistencies

4.5 Evaluation

The SOFT framework was evaluated in comparison with existing schemas. The primary dataset for evaluation was a complete re-annotation of the widely used ACL-ARC citation corpus [55], with the original texts first corrected for OCR and extraction errors, underscoring the importance of high-quality input.

A comprehensive set of metrics was used for the performance analysis. To measure the clarity and usability of the framework itself, Inter-Annotator Agreement (IAA) was calculated using pairwise Cohen’s κ between LLM predictions and human consensus labels. The SOFT framework achieved significantly higher Human-LLM agreement compared to baseline schemas: for Citation Intent, $\kappa = 0.692$ (Qwen); for Cited Content Type, $\kappa = 0.590$ (Gemma). These compare to $\kappa = 0.430$ for ACL-ARC and $\kappa = 0.462$ for SciCite (best cases).

For the classification task, standard metrics including Precision, Recall, and macro-averaged F1-score were used. Fine-tuned models achieved macro F1 of 0.78 for Cited Content Type and 0.65 for Citation Intent under SOFT, compared to 0.51 (ACL-ARC) and 0.62 (SciCite) for intent classification. SOFT demonstrated superior cross-domain generalizability: when transferring from ACL-ARC to ACT2 (spanning 19 scientific disciplines), SOFT-trained models showed only 14-18% F1 degradation, compared to 51-78% for ACL-ARC and 31-63% for SciCite. The evaluation also confirmed that the performance of all models improved with cleaner data, with SOFT-based models particularly benefiting.

4.6 Discussion: Enhancing Bibliometrics and Library Science Applications

The SOFT framework supports a more fine-grained and transparent assessment of scholarly influence. By disentangling intent from content, SOFT allows analysis of how different types of contributions are used across disciplines, supporting distinctions between, for example, a foundational theoretical paper that is widely “discussed” and a software tool that is widely “used.”

This has significant potential applications in digital libraries, including more sophisticated semantic search, smarter recommendation systems, and automated generation of literature reviews. However, the framework has limitations: the current taxonomy is single-label, whereas some citations may serve multiple functions. Additionally, while cross-domain generalizability was evaluated on the ACT2 test set spanning 19 scientific disciplines (SOFT showed only 14-18% F1 degradation compared to 51-78% for ACL-ARC), full training and validation across non-NLP disciplines remains for future work due to the specialized domain knowledge required for expert annotation.

Future work could explore multi-label classification and extend the framework to capture more complex, cross-sentence citation patterns. Nevertheless, the achieved outcome of this work package is an enhanced taxonomy for bibliometric analysis that offers greater clarity, consistency, and analytical power, moving the field from simple citation counting to nuanced contextual understanding.

5 Discussion

The three preceding chapters tackle related but distinct subproblems: creating accessible PDFs from LaTeX, transforming legacy PDFs efficiently, and analyzing documents that have been made accessible. This chapter examines how these contributions connect and where their limitations lie.

5.1 An Integrated Document Lifecycle Framework

Each contribution addresses a different stage of the document lifecycle. Viewed together, they form an integrated framework that spans from document creation to downstream analysis.

5.1.1 From Creation to Analysis

The three contributions form a pipeline that progressively addresses accessibility at each stage of a document's existence.

1. **Prevention (LaTeX Rainbow):** This framework annotates semantic structure in LaTeX sources, enabling PDFs that preserve structural information needed for accessibility and reliable downstream processing.
2. **Remediation (EditTrans & CopyLookup):** For existing PDFs, these hybrid transformation techniques provide a pathway to structured, machine-readable formats. Reported efficiency gains, including up to 44.5% latency reduction, reduce the computational cost of large-scale conversion.

-
-
3. **Application (SOFT Framework):** The preceding stages produce structured inputs that support downstream semantic analysis. SOFT illustrates how cleaner citation contexts and more explicit structure can improve citation classification and related bibliometric analyses.

This pipeline is sequential, and its outputs can be reused across stages. For example, structured outputs from EditTrans and CopyLookup can be used as training or evaluation data for conversion models. Similarly, downstream analyses such as SOFT can motivate more explicit treatment of citation contexts in upstream document processing.

5.2 Positioning Within the Broader Ecosystem

This research does not exist in isolation. Understanding its relationship to concurrent developments and established practices is essential for assessing its contribution and trajectory.

5.2.1 Relationship to the LaTeX Tagged PDF Project

The LaTeX Tagged PDF Project is led by the LaTeX Project Team and represents the most significant concurrent effort toward LaTeX accessibility. Chapter 2 discusses this multi-year initiative, which aims to integrate tagging support directly into the LaTeX kernel. The relationship between LaTeX Rainbow and this project is one of complementarity rather than competition.

The Tagged PDF Project pursues a long-term vision of seamless, kernel-level accessibility. This approach is likely to provide broader coverage and tighter integration as it matures, but its phased development means that comprehensive support is still evolving, particularly for complex packages, tables, and cross-document structures. LaTeX Rainbow provides a deployable approach that can serve authors and institutions while kernel-level support continues to mature. Documents created with LaTeX Rainbow can migrate to native tagging when available, as both approaches target similar PDF/UA standards.

5.2.2 Implications for Digital Libraries and Publishers

The practical deployment of this research depends on its alignment with stakeholder incentives.

For digital libraries and archives: LaTeX Rainbow addresses new submissions. EditTrans and CopyLookup address back-catalog remediation. The combination may offer a reference approach for achieving accessibility compliance.

For publishers: Emerging accessibility mandates create regulatory pressure. The European Accessibility Act becomes effective in June 2025. The tools developed here explore a technical direction that may integrate with existing LaTeX-based workflows.¹

For funding agencies and institutions: The FAIR data principles increasingly govern data management requirements. FAIR stands for Findable, Accessible, Interoperable, and Reusable. Documents processed through this framework have the potential to better satisfy these principles.

5.3 Limitations

5.3.1 Scope Limitations

Language and community: The developed tools are primarily validated on English-language documents within the LaTeX-centric ecosystem of STEM publishing. Although the underlying principles are language-agnostic, practical applicability to other languages remains to be demonstrated, particularly for languages with complex scripts. Applicability to other authoring systems such as Microsoft Word and Google Docs also remains unproven.

Document types: The focus is on scholarly articles, with other document types such as books, theses, technical reports, and presentation slides receiving less attention. Extension to these formats may require additional engineering effort.

Mathematical notation: Mathematical content is a strength of this research, but highly unusual or custom notation may not be handled correctly, including non-standard symbols and complex diagrams embedded in equations.

¹Note that the EAA primarily addresses product and service accessibility broadly. PDF/UA is a specific technical standard for document accessibility. The relationship between these regulatory and technical frameworks requires careful consideration.

5.3.2 Technical Limitations

Dependence on upstream models: The performance of EditTrans and CopyLookup is fundamentally bounded by the accuracy of underlying VLMs and layout analysis models. Errors in layout detection and reading order determination propagate through the pipeline, potentially corrupting final outputs.

Complex layouts: Documents with highly non-linear layouts remain challenging, including multi-column formats with spanning figures, marginalia, and complex float placements. The heuristics used for reading order determination may fail on unusual designs.

Table handling: Tables with merged cells, nested structures, and spanning headers present ongoing challenges that affect both transformation and accessibility tagging. Current implementations may produce semantically incorrect table markup for complex cases.

5.3.3 Evaluation Limitations

Accessibility user studies: Validation against automated accessibility checkers such as PAC and Matterhorn Protocol remains ongoing work. Comprehensive user studies with screen reader users and other assistive technology users have not yet been conducted. Such validation is essential for confirming real-world accessibility improvements.

Long-term maintenance: The sustainability of open-source research tools is an ongoing concern [128]. The implementations released as part of this research require continued maintenance to remain compatible with evolving LaTeX distributions, VLM architectures, and accessibility standards.

5.4 Future Research Directions

The contributions of this thesis suggest several directions for future research. A central insight connects them: document accessibility benefits both human users who rely on assistive technologies and computational systems that process documents.

5.4.1 Toward Accessible Document Intelligence

Current Document VQA and understanding models have improved rapidly, but they remain limited by an inability to directly access deterministic document structure, including DOM trees, tag hierarchies, and logical reading order. Instead, these models often reconstruct structure through visual perception or probabilistic character recognition.

This thesis refers to this gap as a structural accessibility deficit. Current models typically process documents through one of three pathways.

1. **OCR-based pipelines** (LayoutLMs [133, 132, 48], DocFormer [3], StrucText [69]): These models extract text via OCR engines and then encode with Transformers. This cascade introduces error propagation, as OCR mistakes such as misreading “3” as “8” or “\$” as “S” propagate through the pipeline and degrade downstream retrieval and QA accuracy [138]. Studies show that even the best OCR systems fall 7 to 9% short of human-level text extraction [138], creating an “invisible performance ceiling” for LLM-based QA. When OCR error rates exceed 20%, as commonly occurs with historical documents and poor-quality scans, downstream information extraction can become unreliable.
2. **OCR-free pixel processing** (Donut [56], Nougat [10], Pix2Struct [62]): These models process raw images end-to-end, bypassing explicit OCR but requiring the visual encoder to perform text recognition implicitly. Even strong VLMs such as GPT and Gemini achieve 15 to 25% character error rates on this task [138], worse than traditional OCR (approximately 7% CER). These models also face resolution constraints [137]: processing dense A4 documents at 1024×1024 resolution can reduce readability of small content such as footnotes and table entries [53]. When visual evidence is ambiguous, outputs can include hallucinated content; Nougat, for instance, can enter repetition loops and fabricate table rows to maintain syntactic fluency [10].
3. **Hybrid VLM approaches** (GPT-5, Gemini, LLaVA [72]): These models combine frozen visual encoders such as CLIP and ViT with LLMs, inheriting limitations of both components. Such models can produce structural errors, reflecting weak inductive biases for document topology and failures in spatial reasoning and reading order [105]. A 2025 benchmark² found that Gemini 1.5 Pro achieved 95% edit similarity on single-column legal contracts but dropped to 60% on multi-column ArXiv papers, while Claude 3.5 Haiku dropped to 8%. Visual encoders pretrained

²<https://www.applied-ai.com/briefings/pdf-parsing-benchmark/>

primarily on natural images are not optimized for document-specific structure and layout cues.

In each case, the model primarily relies on visual input rather than directly accessing semantic structure. For native digital documents such as PDF, Word, and HTML files, structure can exist deterministically: PDF tag trees can explicitly define “this is a heading” or “this is a table cell.” When pipelines rasterize these structure-bearing files into images, they discard available signals and require models to reconstruct structure, increasing both computational cost and error.

PDF/UA standards benefit human users by providing accessible structure, and the same structural signals can also benefit document processing systems. The contributions of this thesis support this direction: LaTeX Rainbow creates born-accessible PDFs, while EditTrans and CopyLookup perform structured transformations. A longer-term goal is for Document AI systems to consume tagged PDF structure trees, MathML for mathematical notation, and semantic markup as inputs, rather than treating documents primarily as images.

5.4.2 Specific Research Directions

Direction 1: LLM-Assisted PDF/UA-2 Dataset Creation. The arXiv corpus contains over 2 million LaTeX source files, yet most compiled PDFs remain inaccessible. Future work will leverage the strong understanding of LaTeX source code exhibited by LLMs to complete the “last mile” of the LaTeX Project’s tagging initiative. The goal is creating a large-scale, high-quality PDF/UA-2 compliant dataset from existing arXiv papers that can serve as both a training resource for document AI models and a benchmark for accessibility compliance.

Direction 2: Accessibility for Non-LaTeX Documents. LaTeX Rainbow addresses born-accessible creation for LaTeX-authored documents, but the majority of scholarly output is authored in Microsoft Word, Google Docs, or other systems. Future work will extend the accessibility enhancement pipeline to documents not compiled from LaTeX, enabling PDF/UA-2 compliance through post-hoc remediation techniques for a broader range of scholarly outputs.

Direction 3: Rethinking Document AI Interaction Paradigms. Current VLMs and Document AI models often operate without direct access to document semantics and must infer structure from visual representations. Future work will explore interaction paradigms in which AI systems consume documents through accessible APIs, including



tagged PDF structure trees, MathML for mathematical notation, and semantic markup. Such an approach shifts document processing toward treating documents as structured data rather than primarily as images.

6 Conclusion

This dissertation set out to address the lack of structure in scholarly PDFs. Over five publications, it developed methods for creating accessible documents, transforming legacy collections, and demonstrating the downstream value of structured input. This chapter summarizes what was achieved and what remains to be done.

6.1 Summary of Contributions

Three contributions form the core of this work:

1. **LaTeX Rainbow** (Chapter 2): A framework for annotating semantic structure in LaTeX sources that enables “born-accessible” PDF generation through a color-coding alignment mechanism [28, 25].
2. **CopyLookup and EditTrans** (Chapter 3): Hybrid approaches for efficient PDF-to-Markdown transformation that achieve up to $1.7\times$ speedup and 44.5% latency reduction while maintaining output quality [24, 26].
3. **SOFT Framework** (Chapter 4): A semantically orthogonal taxonomy for citation classification that separates citation intent from cited content type [27].

6.2 The Research Journey

The five publications in this thesis trace a research trajectory from initial problem identification to an integrated view of accessible document intelligence.

The research began with an observation that many scholarly PDFs remain inaccessible to screen readers despite decades of accessibility standards. The initial work on LaTeX

Rainbow [28] was motivated by the fact that LaTeX sources contain explicit semantic markup, yet standard pdfTeX workflows typically do not preserve this structure in the resulting PDFs. The color-coding approach provided an indirect but deployable pathway for recovering structural information from compiled output.

The TUG 2024 paper [25] extended this work from individual documents to arXiv corpus scale. Processing hundreds of thousands of LaTeX submissions revealed both strengths and limitations, while feedback from the LaTeX community highlighted the tension between research prototypes and production-ready tools.

Subsequent work emphasized that creation-time accessibility alone cannot address the large archive of existing PDFs. The NLDB 2025 [24] and ICDAR 2025 [26] papers therefore pivot from prevention to remediation. The hybrid methods are motivated by a resource-aware principle: copy content that can be extracted deterministically and reserve generation for complex elements.

The SOFT framework at TPD L 2025 [27] demonstrates downstream value by using structured inputs for citation analysis. The orthogonal decomposition of citation intent and content type also reflects a broader design principle: analyses benefit from separating distinct semantic dimensions rather than encoding them into a single label space.

6.3 Methodological Reflections

Every research project involves choices of problems, methods, and trade-offs. Reflecting on these choices helps situate contributions honestly and guides future work.

pdfTeX over LuaTeX. The decision to work within pdfTeX’s constraints was deliberate because major publishers and platforms (Springer, IEEE, ACM, Overleaf) require pdfTeX compatibility. A solution demanding engine migration would limit practical adoption. The trade-off was engineering complexity: the color-coding mechanism is an indirect workaround, whereas LuaTeX provides this functionality natively.

Hybrid over end-to-end. The hybrid transformation approaches are motivated by the observation that not all sub-tasks require neural generation. When deterministic extraction is sufficient, generation adds latency without improving fidelity. This choice trades architectural simplicity for component efficiency.

Orthogonal over flat classification. The SOFT framework’s dimensional decomposition increased annotation complexity (two labels per citation), but enabled more principled analysis. For nuanced bibliometric applications, the added granularity proves worthwhile.

What would be done differently. With hindsight, user studies with actual assistive technology users should have been prioritized earlier, since technical compliance does not guarantee genuine usability improvements. Earlier investment in multilingual evaluation would also have strengthened claims about transferability.

6.4 Closing Remarks

This dissertation contributes methods that support both proactive creation and large-scale remediation. The “born-accessible” approach treats accessibility as a property of document creation workflows rather than a post-hoc retrofit. The hybrid transformation approaches reduce computational cost, supporting conversion of legacy collections at larger scale.

Much remains to be done. The approaches are validated primarily on English-language, LaTeX-centric documents; extension to other languages, scripts, and authoring systems requires further work. User studies with assistive technology users are essential to validate that technical accessibility translates to genuine usability.

Both human readers and computational systems increasingly depend on structured access to scholarly knowledge, and incentives for accessible document formats are likely to strengthen. The technical foundations developed here, including born-accessible creation, efficient transformation, and semantic analysis, contribute toward a more accessible and machine-actionable scholarly record.

Bibliography

- [1] Marah Abdin et al. *Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone*. 2024. DOI: 10.48550/ARXIV.2404.14219. URL: <https://arxiv.org/abs/2404.14219>.
- [2] Jean-Baptiste Alayrac et al. “Flamingo: a visual language model for few-shot learning”. In: NIPS ’22 (2022).
- [3] Srikar Appalaraju et al. “DocFormer: End-to-End Transformer for Document Understanding”. In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*. Oct. 2021, pp. 993–1003.
- [4] Christoph Auer et al. *Docling Technical Report*. Tech. rep. 2024. arXiv: 2408.09869 [cs.CL]. URL: <https://arxiv.org/abs/2408.09869>.
- [5] Abhijeet Awasthi et al. “Parallel Iterative Edit Models for Local Sequence Transduction”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui et al. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 4260–4270. DOI: 10.18653/v1/D19-1435. URL: <https://aclanthology.org/D19-1435/>.
- [6] Jinze Bai et al. *Qwen-VL: A Versatile Vision-Language Model for Understanding, Localization, Text Reading, and Beyond*. 2023. arXiv: 2308.12966 [cs.CV]. URL: <https://arxiv.org/abs/2308.12966>.
- [7] Shuai Bai et al. *Qwen2.5-VL Technical Report*. 2025. DOI: 10.48550/ARXIV.2502.13923. URL: <https://arxiv.org/abs/2502.13923>.
- [8] Hannah Bast and Claudius Korzen. “A Benchmark and Evaluation for Text Extraction from PDF”. In: *2017 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*. 2017, pp. 1–10. DOI: 10.1109/JCDL.2017.7991564.

-
- [9] Mohammad Bavarian et al. *Efficient Training of Language Models to Fill in the Middle*. 2022. DOI: 10.48550/ARXIV.2207.14255. URL: <https://arxiv.org/abs/2207.14255>.
- [10] Lukas Blecher et al. “Nougat: Neural Optical Understanding for Academic Documents”. In: *International Conference on Learning Representations (ICLR)*. 2023.
- [11] Tianle Cai et al. “Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads”. In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov et al. Vol. 235. Proceedings of Machine Learning Research. PMLR, 21–27 Jul 2024, pp. 5209–5235. URL: <https://proceedings.mlr.press/v235/cai24b.html>.
- [12] Yuxuan Cai et al. *FastMTP: Accelerating LLM Inference with Enhanced Multi-Token Prediction*. 2025. URL: <https://openreview.net/forum?id=J7xDwZSyI4>.
- [13] Anna Capietto et al. *The axessibility package*. CTAN package documentation. 2020. URL: <https://ctan.org/pkg/axessibility>.
- [14] J. A. Chaldecott. “Bibliography - A History of Scientific & Technical Periodicals: The Origins and Development of the Scientific and Technical Press, 2nd edition”. In: *The British Journal for the History of Science* 11.1 (Mar. 1978), pp. 67–67. ISSN: 1474-001X. DOI: 10.1017/S0007087400015971. URL: <http://dx.doi.org/10.1017/S0007087400015971>.
- [15] Charlie Chen et al. “Accelerating Large Language Model Decoding with Speculative Sampling”. In: 2023. arXiv: 2302.01318 [cs.CL]. URL: <https://arxiv.org/abs/2302.01318>.
- [16] Zhe Chen et al. *Expanding Performance Boundaries of Open-Source Multimodal Models with Model, Data, and Test-Time Scaling*. 2024. DOI: 10.48550/ARXIV.2412.05271. URL: <https://arxiv.org/abs/2412.05271>.
- [17] Jacob K Christopher et al. *Speculative Diffusion Decoding: Accelerating Language Generation through Diffusion*. Ed. by Luis Chiruzzo, Alan Ritter, and Lu Wang. Albuquerque, New Mexico, Apr. 2025. DOI: 10.18653/v1/2025.naacl-long.601. URL: <https://aclanthology.org/2025.naacl-long.601/>.
- [18] Arman Cohan et al. “Structural Scaffolds for Citation Intent Classification in Scientific Publications”. In: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. 2019, pp. 3586–3596.

-
- [19] Michael Cooper et al. *Web Content Accessibility Guidelines (WCAG) 2.2*. W3C Recommendation. <https://www.w3.org/TR/2024/REC-WCAG22-20241212/>. Dec. 2024.
- [20] Isaac Councill, C. Lee Giles, and Min-Yen Kan. “ParsCit: an Open-source CRF Reference String Parsing Package”. In: *Proceedings of the Sixth International Conference on Language Resources and Evaluation (LREC’08)*. Ed. by Nicoletta Calzolari et al. Marrakech, Morocco: European Language Resources Association (ELRA), May 2008. URL: <https://aclanthology.org/L08-1291/>.
- [21] Cheng Cui et al. *PaddleOCR-VL: Boosting Multilingual Document Parsing via a 0.9B Ultra-Compact Vision-Language Model*. 2025. DOI: 10.48550/ARXIV.2510.14528. URL: <https://arxiv.org/abs/2510.14528>.
- [22] DeepSeek-AI et al. *DeepSeek-V3 Technical Report*. 2024. DOI: 10.48550/ARXIV.2412.19437. URL: <https://arxiv.org/abs/2412.19437>.
- [23] Alexey Dosovitskiy et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale”. In: *International Conference on Learning Representations*. 2021. URL: <https://openreview.net/forum?id=YicbFdNTTy>.
- [24] Changxu Duan. “Accelerating End-to-End PDF to Markdown Conversion Through Assisted Generation”. In: *Natural Language Processing and Information Systems*. Springer Nature Switzerland, July 2025, pp. 34–48. ISBN: 9783031971419. DOI: 10.1007/978-3-031-97141-9_3. URL: http://dx.doi.org/10.1007/978-3-031-97141-9_3.
- [25] Changxu Duan. “Bridging scientific publication accessibility: LaTeX–markup–PDF–alignment”. In: *TUGboat* 45.2 (2024), pp. 179–184. ISSN: 0896-3207. DOI: 10.47397/tb/45-2/tb140duan-bridging. URL: <http://dx.doi.org/10.47397/tb/45-2/tb140duan-bridging>.
- [26] Changxu Duan. “Layout-Aware Text Editing for Efficient Transformation of Academic PDFs to Markdown”. In: *Document Analysis and Recognition – ICDAR 2025*. Springer Nature Switzerland, Sept. 2025, pp. 221–241. ISBN: 9783032046147. DOI: 10.1007/978-3-032-04614-7_13. URL: http://dx.doi.org/10.1007/978-3-032-04614-7_13.
- [27] Changxu Duan and Zhiyin Tan. “Semantically Orthogonal Framework for Citation Classification: Disentangling Intent and Content”. In: *Linking Theory and Practice of Digital Libraries*. Springer Nature Switzerland, Sept. 2025, pp. 183–206. ISBN: 9783032054098. DOI: 10.1007/978-3-032-05409-8_12. URL: http://dx.doi.org/10.1007/978-3-032-05409-8_12.

-
-
- [28] Changxu Duan, Zhiyin Tan, and Sabine Bartsch. “LaTeX Rainbow: Universal LaTeX to PDF Document Semantic & Layout Annotation Framework”. In: *Proceedings of the Second Workshop on Information Extraction from Scientific Publications*. Ed. by Tirthankar Ghosal et al. Bali, Indonesia: Association for Computational Linguistics, Nov. 2023, pp. 56–67. DOI: 10.18653/v1/2023.wiesp-1.8. URL: <https://aclanthology.org/2023.wiesp-1.8/>.
- [29] Abhimanyu Dubey et al. *The llama 3 herd of models*. 2024.
- [30] Hao Feng et al. “Dolphin: Document Image Parsing via Heterogeneous Anchor Prompting”. In: *Proceedings of the 65th Annual Meeting of the Association for Computational Linguistics (ACL)*. 2025.
- [31] Ulrike Fischer and Frank Mittelbach. “Automated tagging of LaTeX documents—what is possible today, in 2023?” In: *TUGboat* 44.2 (2023), pp. 262–266. ISSN: 0896-3207. DOI: 10.47397/tb/44-2/tb137fischer-tagging23. URL: <http://dx.doi.org/10.47397/tb/44-2/tb137fischer-tagging23>.
- [32] Ulrike Fischer and Frank Mittelbach. “News from the LaTeX Tagged PDF project: 2025”. In: *TUGboat* 46.2 (2025), pp. 244–247. ISSN: 0896-3207. DOI: 10.47397/tb/46-2/tb143fischer-tagging25. URL: <http://dx.doi.org/10.47397/tb/46-2/tb143fischer-tagging25>.
- [33] Fiorenzo Franceschini, Domenico Maisano, and Luca Mastrogiacomio. “A novel approach for estimating the omitted-citation rate of bibliometric databases with an application to the field of bibliometrics”. In: *Journal of the American Society for Information Science and Technology* 64.10 (Aug. 2013), pp. 2149–2156. ISSN: 1532-2890. DOI: 10.1002/asi.22898. URL: <http://dx.doi.org/10.1002/asi.22898>.
- [34] Yichao Fu et al. “Break the Sequential Dependency of LLM Inference Using Lookahead Decoding”. In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov et al. Vol. 235. Proceedings of Machine Learning Research. PMLR, 21–27 Jul 2024, pp. 14060–14079. URL: <https://proceedings.mlr.press/v235/fu24a.html>.
- [35] Miguel A. García-Pérez. “Accuracy and completeness of publication and citation records in the Web of Science, PsycINFO, and Google Scholar: A case study for the computation of h indices in Psychology”. In: *Journal of the American Society for Information Science and Technology* 61.10 (Sept. 2010), pp. 2070–2085. ISSN: 1532-2890. DOI: 10.1002/asi.21372. URL: <http://dx.doi.org/10.1002/asi.21372>.

-
- [36] Eugene Garfield. “Citation Analysis as a Tool in Journal Evaluation: Journals can be ranked by frequency and impact of citations for science policy studies.” In: *Science* 178.4060 (Nov. 1972), pp. 471–479. ISSN: 1095-9203. DOI: 10.1126/science.178.4060.471. URL: <http://dx.doi.org/10.1126/science.178.4060.471>.
- [37] Eugene Garfield. “Citation Indexes for Science: A New Dimension in Documentation through Association of Ideas”. In: *Science* 122.3159 (July 1955), pp. 108–111. ISSN: 1095-9203. DOI: 10.1126/science.122.3159.108. URL: <http://dx.doi.org/10.1126/science.122.3159.108>.
- [38] E. Gaskell. “A History of Scientific and Technical Periodicals: the Origins and Development of the Scientific and Technological Press”. In: *Medical History* 7.3 (July 1963), pp. 292–292. ISSN: 2048-8343. DOI: 10.1017/S0025727300028660. URL: <http://dx.doi.org/10.1017/S0025727300028660>.
- [39] Fabian Gloeckle et al. *Better & Faster Large Language Models via Multi-token Prediction*. Ed. by Ruslan Salakhutdinov et al. 21–27 Jul 2024. URL: <https://proceedings.mlr.press/v235/gloeckle24a.html>.
- [40] S. A Greenberg. “How citation distortions create unfounded authority: analysis of a citation network”. In: *BMJ* 339.jul20 3 (July 2009), b2680–b2680. ISSN: 1468-5833. DOI: 10.1136/bmj.b2680. URL: <http://dx.doi.org/10.1136/bmj.b2680>.
- [41] GROBID. <https://github.com/kermitt2/grobid>. 2008–2025. swb: 1 : dir:dab86b296e3c3216e2241968f0d63b68e8209d3c.
- [42] Daya Guo et al. *DeepSeek-Coder: When the Large Language Model Meets Programming – The Rise of Code Intelligence*. 2024. DOI: 10.48550/ARXIV.2401.14196. URL: <https://arxiv.org/abs/2401.14196>.
- [43] Zhenyu He et al. “REST: Retrieval-Based Speculative Decoding”. In: *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. Ed. by Kevin Duh, Helena Gomez, and Steven Bethard. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 1582–1595. DOI: 10.18653/v1/2024.naacl-long.88. URL: <https://aclanthology.org/2024.naacl-long.88/>.

-
- [44] Anwen Hu et al. *mPLUG-DocOwl 1.5: Unified Structure Learning for OCR-free Document Understanding*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA, Nov. 2024. DOI: 10.18653/v1/2024.findings-emnlp.175. URL: <https://aclanthology.org/2024.findings-emnlp.175/>.
- [45] Edward J Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *International Conference on Learning Representations*. 2022. URL: <https://openreview.net/forum?id=nZeVKeeFYf9>.
- [46] Yunhai Hu et al. *Speculative Decoding and Beyond: An In-Depth Survey of Techniques*. 2025. arXiv: 2502.19732 [cs.CL]. URL: <https://arxiv.org/abs/2502.19732>.
- [47] Yuxuan Hu et al. *SAM Decoding: Speculative Decoding via Suffix Automaton*. Ed. by Wanxiang Che et al. Vienna, Austria, July 2025. DOI: 10.18653/v1/2025.acl-long.595. URL: <https://aclanthology.org/2025.acl-long.595/>.
- [48] Yupan Huang et al. “LayoutLMv3: Pre-training for Document AI with Unified Text and Image Masking”. In: *Proceedings of the 30th ACM International Conference on Multimedia*. MM ’22. Lisboa, Portugal: Association for Computing Machinery, 2022, pp. 4083–4091. ISBN: 9781450392037. DOI: 10.1145/3503161.3548112. URL: <https://doi.org/10.1145/3503161.3548112>.
- [49] International Organization for Standardization. *Document management — Portable document format — Part 1: PDF 1.7*. Standard ISO 32000-1:2008. ISO, 2008. URL: <https://www.iso.org/standard/51502.html>.
- [50] International Organization for Standardization. *ISO 14289-1:2014 – Document management applications – Electronic document file format enhancement for accessibility – Part 1: Use of ISO 32000-1 (PDF/UA-1)*. <https://www.iso.org/standard/64599.html>. ISO Standard, Geneva, Switzerland. 2014.
- [51] International Organization for Standardization. *ISO 14289-2:2024 – Document management applications – Electronic document file format enhancement for accessibility – Part 2: Use of ISO 32000-2 (PDF/UA-2)*. <https://www.iso.org/standard/82278.html>. ISO Standard, Geneva, Switzerland. 2024.
- [52] Chao Jia et al. “Scaling Up Visual and Vision-Language Representation Learning With Noisy Text Supervision”. In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 4904–4916. URL: <https://proceedings.mlr.press/v139/jia21b.html>.

-
-
- [53] Woojun Jung et al. *Visual Funnel: Resolving Contextual Blindness in Multimodal Large Language Models*. 2025. arXiv: 2512.10362 [cs.CV]. URL: <https://arxiv.org/abs/2512.10362>.
- [54] Daniel Jurafsky and James H. Martin. *Speech and Language Processing*. 3rd. Chapter 20: Semantic Role Labeling. Available online: <https://web.stanford.edu/jurafsky/slp3/>. 2024.
- [55] David Jurgens et al. “Measuring the Evolution of a Scientific Field through Citation Frames”. In: *Transactions of the Association for Computational Linguistics* 6 (2018). Ed. by Lillian Lee et al., pp. 391–406. DOI: 10.1162/tac1_a_00028. URL: <https://aclanthology.org/Q18-1028/>.
- [56] Geewook Kim et al. “OCR-Free Document Understanding Transformer”. In: *Computer Vision – ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXVIII*. Tel Aviv, Israel: Springer-Verlag, 2022, pp. 498–517. ISBN: 978-3-031-19814-4. DOI: 10.1007/978-3-031-19815-1_29. URL: https://doi.org/10.1007/978-3-031-19815-1_29.
- [57] Sehoon Kim et al. “Speculative Decoding with Big Little Decoder”. In: *Thirty-seventh Conference on Neural Information Processing Systems*. 2023. URL: <https://openreview.net/forum?id=EfMyf9MC3t>.
- [58] Wonjae Kim, Bokyung Son, and Ildoo Kim. “ViLT: Vision-and-Language Transformer Without Convolution or Region Supervision”. In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 5583–5594. URL: <https://proceedings.mlr.press/v139/kim21k.html>.
- [59] Rodney Kinney et al. “The Semantic Scholar Open Data Platform”. In: (2025). arXiv: 2301.10140 [cs.DL]. URL: <https://arxiv.org/abs/2301.10140>.
- [60] Petr Knuth and Zdenek Zdrahal. “CORE: Three Access Levels to Underpin Open Access”. In: *D-Lib Magazine* 18.11/12 (Nov. 2012). ISSN: 1082-9873. DOI: 10.1045/november2012-knuth. URL: <http://dx.doi.org/10.1045/november2012-knuth>.
- [61] Anukriti Kumar and Lucy Lu Wang. “Uncovering the New Accessibility Crisis in Scholarly PDFs: Publishing Model and Platform Changes Contribute to Declining Scholarly Document Accessibility in the Last Decade”. In: *The 26th International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS ’24. ACM,

-
- Oct. 2024, pp. 1–16. DOI: 10.1145/3663548.3675634. URL: <http://dx.doi.org/10.1145/3663548.3675634>.
- [62] Kenton Lee et al. “Pix2Struct: screenshot parsing as pretraining for visual language understanding”. In: *Proceedings of the 40th International Conference on Machine Learning*. ICML’23. Honolulu, Hawaii, USA: JMLR.org, 2023.
- [63] Yaniv Leviathan, Matan Kalman, and Yossi Matias. “Fast Inference from Transformers via Speculative Decoding”. In: *Proceedings of the 40th International Conference on Machine Learning*. Ed. by Andreas Krause et al. Vol. 202. Proceedings of Machine Learning Research. PMLR, 23–29 Jul 2023, pp. 19274–19286. URL: <https://proceedings.mlr.press/v202/leviathan23a.html>.
- [64] Junlong Li et al. “DiT: Self-supervised Pre-training for Document Image Transformer”. In: *Proceedings of the 30th ACM International Conference on Multimedia*. MM ’22. Lisboa, Portugal: Association for Computing Machinery, 2022, pp. 3530–3539. ISBN: 9781450392037. DOI: 10.1145/3503161.3547911. URL: <https://doi.org/10.1145/3503161.3547911>.
- [65] Junnan Li et al. “BLIP: Bootstrapping Language-Image Pre-training for Unified Vision-Language Understanding and Generation”. In: *Proceedings of the 39th International Conference on Machine Learning*. Ed. by Kamalika Chaudhuri et al. Vol. 162. Proceedings of Machine Learning Research. PMLR, 2022. URL: <https://proceedings.mlr.press/v162/li22n.html>.
- [66] Minghao Li et al. “DocBank: A Benchmark Dataset for Document Layout Analysis”. In: (Dec. 2020). Ed. by Donia Scott, Nuria Bel, and Chengqing Zong, pp. 949–960. DOI: 10.18653/v1/2020.coling-main.82. URL: <https://aclanthology.org/2020.coling-main.82/>.
- [67] Yuhui Li et al. “EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees”. In: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 7421–7432. DOI: 10.18653/v1/2024.emnlp-main.422. URL: <https://aclanthology.org/2024.emnlp-main.422/>.
- [68] Yuhui Li et al. “EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty”. In: *Proceedings of the 41st International Conference on Machine Learning*. Ed. by Ruslan Salakhutdinov et al. Vol. 235. Proceedings of Machine Learning Research. PMLR, 21–27 Jul 2024, pp. 28935–28948. URL: <https://proceedings.mlr.press/v235/li24bt.html>.

-
- [69] Yulin Li et al. “StrucTexT: Structured Text Understanding with Multi-Modal Transformers”. In: *Proceedings of the 29th ACM International Conference on Multimedia*. MM ’21. Virtual Event, China: Association for Computing Machinery, 2021, pp. 1912–1920. ISBN: 9781450386517. DOI: 10.1145/3474085.3475345. URL: <https://doi.org/10.1145/3474085.3475345>.
- [70] Haofu Liao et al. *DocTr: Document Transformer for Structured Information Extraction in Documents*. 2023. arXiv: 2307.07929 [cs.CV]. URL: <https://arxiv.org/abs/2307.07929>.
- [71] Ji Lin et al. “AWQ: Activation-aware Weight Quantization for On-Device LLM Compression and Acceleration”. In: vol. 28. 4. New York, NY, USA: Association for Computing Machinery, Jan. 2025, pp. 12–17. DOI: 10.1145/3714983.3714987. URL: <https://doi.org/10.1145/3714983.3714987>.
- [72] Haotian Liu et al. *Visual instruction tuning*. New Orleans, LA, USA, 2023.
- [73] Yinhan Liu et al. “Multilingual Denoising Pre-training for Neural Machine Translation”. In: *Transactions of the Association for Computational Linguistics* 8 (2020). Ed. by Mark Johnson, Brian Roark, and Ani Nenkova, pp. 726–742. DOI: 10.1162/tac1_a_00343. URL: <https://aclanthology.org/2020.tac1-1.47/>.
- [74] Ze Liu et al. “Swin Transformer: Hierarchical Vision Transformer using Shifted Windows”. In: *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*. IEEE, Oct. 2021, pp. 9992–10002. DOI: 10.1109/iccv48922.2021.00986. URL: <http://dx.doi.org/10.1109/ICCV48922.2021.00986>.
- [75] Kyle Lo et al. “S2ORC: The Semantic Scholar Open Research Corpus”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Ed. by Dan Jurafsky et al. Online: Association for Computational Linguistics, July 2020, pp. 4969–4983. DOI: 10.18653/v1/2020.acl-main.447. URL: <https://aclanthology.org/2020.acl-main.447/>.
- [76] Tengchao Lv et al. “Kosmos-2.5: A multimodal literate model”. In: *arXiv preprint arXiv:2309.11419* (2023). arXiv: 2309.11419 [cs.AI].
- [77] Michael H. MacRoberts and Barbara R. MacRoberts. “The Negational Reference: or the Art of Dissembling”. In: *Social Studies of Science* 14.1 (Feb. 1984), pp. 91–94. ISSN: 1460-3659. DOI: 10.1177/030631284014001006. URL: <http://dx.doi.org/10.1177/030631284014001006>.

-
- [78] Jonathan Mallinson et al. “EditT5: Semi-Autoregressive Text Editing with T5 Warm-Start”. In: *Findings of the Association for Computational Linguistics: EMNLP 2022*. Ed. by Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 2126–2138. DOI: 10.18653/v1/2022.findings-emnlp.156. URL: <https://aclanthology.org/2022.findings-emnlp.156/>.
- [79] Jonathan Mallinson et al. “FELIX: Flexible Text Editing Through Tagging and Insertion”. In: *Findings of the Association for Computational Linguistics: EMNLP 2020*. Ed. by Trevor Cohn, Yulan He, and Yang Liu. Online: Association for Computational Linguistics, Nov. 2020, pp. 1244–1255. DOI: 10.18653/v1/2020.findings-emnlp.111. URL: <https://aclanthology.org/2020.findings-emnlp.111/>.
- [80] Eric Malmi et al. “Encode, Tag, Realize: High-Precision Text Editing”. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Ed. by Kentaro Inui et al. Hong Kong, China: Association for Computational Linguistics, Nov. 2019, pp. 5054–5065. DOI: 10.18653/v1/D19-1510. URL: <https://aclanthology.org/D19-1510/>.
- [81] Eric Malmi et al. “Text Generation with Text-Editing Models”. In: *Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Tutorial Abstracts*. Ed. by Miguel Ballesteros, Yulia Tsvetkov, and Cecilia O. Alm. Seattle, United States: Association for Computational Linguistics, July 2022, pp. 1–7. DOI: 10.18653/v1/2022.naacl-tutorials.1. URL: <https://aclanthology.org/2022.naacl-tutorials.1/>.
- [82] Bradley McDanel et al. *PipeSpec: Breaking Stage Dependencies in Hierarchical LLM Decoding*. Ed. by Wanxiang Che et al. Vienna, Austria, July 2025. DOI: 10.18653/v1/2025.findings-acl.669. URL: <https://aclanthology.org/2025.findings-acl.669/>.
- [83] Norman Meuschke et al. “A Benchmark of PDF Information Extraction Tools Using a Multi-task and Multi-domain Evaluation Framework for Academic Documents”. In: *Information for a Better World: Normality, Virtuality, Physicality, Inclusivity*. Vol. 13972. LNCS. Cham: Springer Nature Switzerland, 2023, pp. 383–405. ISBN: 978-3-031-28031-3 978-3-031-28032-0. DOI: 10.1007/978-3-031-28032-0_31.

-
- [84] Frank Mittelbach and Ulrike Fischer. “The LaTeX Tagged PDF project—A status and progress report”. In: *TUGboat* 43.3 (2022), pp. 268–272. ISSN: 0896-3207. DOI: 10.47397/tb/43-3/tb135mitt-tagged. URL: <http://dx.doi.org/10.47397/tb/43-3/tb135mitt-tagged>.
- [85] Ross Moore. “Implementing PDF standards for mathematical publishing”. In: *TUGboat* 39.2 (2018), pp. 131–135. URL: <https://tug.org/TUGboat/tb39-2/tb122moore-pdf.pdf>.
- [86] Ross Moore. “Ongoing efforts to generate “tagged PDF” using pdfTeX”. In: *TUGboat* 30.2 (2009), pp. 170–175. URL: <https://tug.org/TUGboat/tb30-2/tb95moore.pdf>.
- [87] Ross Moore. “Tagged mathematics in PDFs for accessibility and other purposes”. In: *Work in Progress at CICM 2013*. 2013. URL: <http://ceur-ws.org/Vol-1010/paper-01.pdf>.
- [88] Michael J. Moravcsik and Poovanalingam Murugesan. “Some Results on the Function and Quality of Citations”. In: *Social Studies of Science* 5.1 (Feb. 1975), pp. 86–92. ISSN: 1460-3659. DOI: 10.1177/030631277500500106. URL: <http://dx.doi.org/10.1177/030631277500500106>.
- [89] Suchetha Nambanoor Kunnath et al. “ACT2: A multi-disciplinary semi-structured dataset for importance and purpose classification of citations”. In: *Proceedings of the Thirteenth Language Resources and Evaluation Conference*. Ed. by Nicoletta Calzolari et al. Marseille, France: European Language Resources Association, June 2022, pp. 3398–3406. URL: <https://aclanthology.org/2022.lrec-1.363/>.
- [90] Hidetsugu Nanba and Manabu Okumura. “Towards multi-paper summarization reference information”. In: *Proceedings of the 16th International Joint Conference on Artificial Intelligence - Volume 2*. IJCAI’99. Stockholm, Sweden: Morgan Kaufmann Publishers Inc., 1999, pp. 926–931.
- [91] Ahmed Nassar et al. *SmolDocling: An ultra-compact vision-language model for end-to-end multi-modal document conversion*. Oct. 2025.
- [92] National Information Standards Organization. *JATS: Journal Article Tag Suite, version 1.3*. Standard ANSI/NISO Z39.96-2021. NISO, July 2021. DOI: 10.3789/ansi.niso.z39.96-2021. URL: <https://doi.org/10.3789/ansi.niso.z39.96-2021>.

-
- [93] Josh M. Nicholson et al. “scite: A smart citation index that displays the context of citations and classifies their intent using deep learning”. In: *Quantitative Science Studies* 2.3 (2021), pp. 882–898. ISSN: 2641-3337. DOI: 10.1162/qss_a_00146. URL: http://dx.doi.org/10.1162/qss_a_00146.
- [94] Junbo Niu et al. *MinerU2.5: A Decoupled Vision-Language Model for Efficient High-Resolution Document Parsing*. 2025. arXiv: 2509.22186 [cs.CV]. URL: <https://arxiv.org/abs/2509.22186>.
- [95] Rochana R. Obadage, Sarah M. Rajtmajer, and Jian Wu. *CC30k: A Citation Contexts Dataset for Reproducibility-Oriented Sentiment Analysis*. 2025. arXiv: 2511.07790 [cs.DL]. URL: <https://arxiv.org/abs/2511.07790>.
- [96] Gabriele Oliaro et al. “SuffixDecoding: Extreme Speculative Decoding for Emerging AI Applications”. In: *The Thirty-ninth Annual Conference on Neural Information Processing Systems*. 2025. URL: <https://openreview.net/forum?id=uwL0vbeEVn>.
- [97] Kostiantyn Omelianchuk et al. “GECToR – Grammatical Error Correction: Tag, Not Rewrite”. In: *Proceedings of the Fifteenth Workshop on Innovative Use of NLP for Building Educational Applications*. Ed. by Jill Burstein et al. Seattle, WA, USA → Online: Association for Computational Linguistics, July 2020, pp. 163–170. DOI: 10.18653/v1/2020.bea-1.16. URL: <https://aclanthology.org/2020.bea-1.16/>.
- [98] Linke Ouyang et al. *OmniDocBench: Benchmarking Diverse PDF Document Parsing with Comprehensive Annotations*. June 2025. DOI: 10.1109/cvpr52734.2025.02313. URL: <http://dx.doi.org/10.1109/CVPR52734.2025.02313>.
- [99] PDF Association. *Matterhorn Protocol 1.1*. Tech. rep. PDF/UA conformance testing checklist. PDF Association, 2019. URL: <https://pdfa.org/resource/matterhorn-protocol/>.
- [100] Qiming Peng et al. “ERNIE-Layout: Layout Knowledge Enhanced Pre-training for Visually-rich Document Understanding”. In: *Findings of the Association for Computational Linguistics: EMNLP 2022*. Ed. by Yoav Goldberg, Zornitsa Kozareva, and Yue Zhang. Abu Dhabi, United Arab Emirates: Association for Computational Linguistics, Dec. 2022, pp. 3744–3756. DOI: 10.18653/v1/2022.findings-emnlp.274. URL: <https://aclanthology.org/2022.findings-emnlp.274/>.

-
- [101] Oriane Pierrès, Felix Schmitt-Koopmann, and Alireza Darvishy. “PDF Accessibility in International Academic Publishers”. In: *Computers Helping People with Special Needs*. Springer Nature Switzerland, 2024, pp. 38–46. ISBN: 9783031628467. DOI: 10.1007/978-3-031-62846-7_5. URL: http://dx.doi.org/10.1007/978-3-031-62846-7_5.
- [102] Jake Poznanski et al. *olmOCR: Unlocking Trillions of Tokens in PDFs with Vision Language Models*. 2025. arXiv: 2502.18443 [cs.CL].
- [103] Fabio Quattrini et al. *μgat: Improving Single-Page Document Parsing by Providing Multi-page Context*. Milan, Italy, 2025. DOI: 10.1007/978-3-031-91572-7_13. URL: https://doi.org/10.1007/978-3-031-91572-7_13.
- [104] Alec Radford et al. “Learning Transferable Visual Models From Natural Language Supervision”. In: *Proceedings of the 38th International Conference on Machine Learning*. Ed. by Marina Meila and Tong Zhang. Vol. 139. Proceedings of Machine Learning Research. PMLR, 18–24 Jul 2021, pp. 8748–8763. URL: <https://proceedings.mlr.press/v139/radford21a.html>.
- [105] Pooyan Rahmanzadehgervi et al. “Vision Language Models are blind”. In: *Computer Vision – ACCV 2024*. Springer Nature Singapore, Dec. 2024, pp. 293–309. ISBN: 9789819609178. DOI: 10.1007/978-981-96-0917-8_17. URL: http://dx.doi.org/10.1007/978-981-96-0917-8_17.
- [106] Chris Rowley, Ulrike Fischer, and Frank Mittelbach. “Accessibility in the LaTeX kernel—experiments in Tagged PDF”. In: *TUGboat 40.2* (2019), pp. 157–158. URL: <https://www.tug.org/TUGboat/tb40-2/tb125rowley-tagpdf.pdf>.
- [107] Ather Sharif et al. “Understanding Screen-Reader Users’ Experiences with Online Data Visualizations”. In: *Proceedings of the 23rd International ACM SIGACCESS Conference on Computers and Accessibility*. ASSETS ’21. ACM, Oct. 2021, pp. 1–16. DOI: 10.1145/3441852.3471202. URL: <http://dx.doi.org/10.1145/3441852.3471202>.
- [108] Zejiang Shen et al. “VILA: Improving Structured Content Extraction from Scientific PDFs Using Visual Layout Groups”. In: *Transactions of the Association for Computational Linguistics 10* (2022). Ed. by Brian Roark and Ani Nenkova, pp. 376–392. DOI: 10.1162/tac1_a_00466. URL: <https://aclanthology.org/2022.tac1-1.22/>.
- [109] David Shotton. “CiTO, the Citation Typing Ontology”. In: *Journal of Biomedical Semantics 1*. Suppl 1 (2010), S6. ISSN: 2041-1480. DOI: 10.1186/2041-1480-1-s1-s6. URL: <http://dx.doi.org/10.1186/2041-1480-1-s1-s6>.

-
- [110] R. Smith. *An Overview of the Tesseract OCR Engine*. USA, 2007.
- [111] Hye Ryung Son and Jay-Yoon Lee. *Locate&Edit: Energy-based Text Editing for Efficient, Flexible, and Faithful Controlled Text Generation*. 2024. arXiv: 2407.00740 [cs.CL]. URL: <https://arxiv.org/abs/2407.00740>.
- [112] Benjamin Spector and Chris Re. “Accelerating LLM Inference with Staged Speculative Decoding”. In: 2023. arXiv: 2308.04623 [cs.AI]. URL: <https://arxiv.org/abs/2308.04623>.
- [113] Ina Spiegel-Rosing. “Science Studies: Bibliometric and Content Analysis”. In: *Social Studies of Science* 7.1 (Feb. 1977), pp. 97–113. ISSN: 1460-3659. DOI: 10.1177/030631277700700111. URL: <http://dx.doi.org/10.1177/030631277700700111>.
- [114] Felix Stahlberg and Shankar Kumar. “Seq2Edits: Sequence Transduction Using Span-level Edit Operations”. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Ed. by Bonnie Webber et al. Online: Association for Computational Linguistics, Nov. 2020, pp. 5147–5159. DOI: 10.18653/v1/2020.emnlp-main.418. URL: <https://aclanthology.org/2020.emnlp-main.418/>.
- [115] Yu Sun et al. *LOCR: Location-Guided Transformer for Optical Character Recognition*. Ed. by Yaser Al-Onaizan, Mohit Bansal, and Yun-Nung Chen. Miami, Florida, USA, Nov. 2024. DOI: 10.18653/v1/2024.findings-emnlp.314. URL: <https://aclanthology.org/2024.findings-emnlp.314/>.
- [116] Martin Szomszor and Euan Adie. “Overton: A bibliometric database of policy document citations”. In: *Quantitative Science Studies* 3.3 (2022), pp. 624–650. ISSN: 2641-3337. DOI: 10.1162/qss_a_00204. URL: http://dx.doi.org/10.1162/qss_a_00204.
- [117] Bangsheng Tang et al. *Efficient Speculative Decoding for Llama at Scale: Challenges and Solutions*. 2025. arXiv: 2508.08192 [cs.CL]. URL: <https://arxiv.org/abs/2508.08192>.
- [118] Simone Teufel, Advait Siddharthan, and Dan Tidhar. “Automatic classification of citation function”. In: *Proceedings of the 2006 Conference on Empirical Methods in Natural Language Processing*. Ed. by Dan Jurafsky and Eric Gaussier. Sydney, Australia: Association for Computational Linguistics, July 2006, pp. 103–110. URL: <https://aclanthology.org/W06-1613/>.

-
- [119] Nadav Timor et al. *Accelerating LLM Inference with Lossless Speculative Decoding Algorithms for Heterogeneous Vocabularies*. Ed. by Aarti Singh et al. 13–19 Jul 2025.
- [120] Marco Antonio Valenzuela-Escarcega, Vu A. Ha, and Oren Etzioni. “Identifying Meaningful Citations”. In: *AAAI Workshop: Scholarly Big Data*. 2015. URL: <https://cdn.aaai.org/ocs/ws/ws0121/10185-46008-1-PB.pdf>.
- [121] Bin Wang et al. *MinerU: An Open-Source Solution for Precise Document Content Extraction*. 2024. arXiv: 2409.18839 [cs.CV]. URL: <https://arxiv.org/abs/2409.18839>.
- [122] Jiapeng Wang, Lianwen Jin, and Kai Ding. “LiLT: A Simple yet Effective Language-Independent Layout Transformer for Structured Document Understanding”. In: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Ed. by Smaranda Muresan, Preslav Nakov, and Aline Villavicencio. Dublin, Ireland: Association for Computational Linguistics, May 2022, pp. 7747–7757. DOI: 10.18653/v1/2022.acl-long.534. URL: <https://aclanthology.org/2022.acl-long.534/>.
- [123] Jiawei Wang, Kai Hu, and Qiang Huo. “DLAFormer: An End-to-End Transformer For Document Layout Analysis”. In: *Document Analysis and Recognition - ICDAR 2024*. Springer Nature Switzerland, 2024, pp. 40–57. ISBN: 9783031705465. DOI: 10.1007/978-3-031-70546-5_3. URL: http://dx.doi.org/10.1007/978-3-031-70546-5_3.
- [124] Peng Wang et al. *Qwen2-VL: Enhancing Vision-Language Model’s Perception of the World at Any Resolution*. 2024. arXiv: 2409.12191 [cs.CV]. URL: <https://arxiv.org/abs/2409.12191>.
- [125] Haoran Wei, Yaofeng Sun, and Yukun Li. *DeepSeek-OCR: Contexts Optical Compression*. 2025. arXiv: 2510.18234 [cs.CV]. URL: <https://arxiv.org/abs/2510.18234>.
- [126] Haoran Wei et al. *General OCR Theory: Towards OCR-2.0 via a Unified End-to-end Model*. 2024. DOI: 10.48550/ARXIV.2409.01704. URL: <https://arxiv.org/abs/2409.01704>.
- [127] Mark D. Wilkinson, Michel Dumontier, IJsbrand Jan Aalbersberg, et al. “The FAIR Guiding Principles for scientific data management and stewardship”. In: *Scientific Data* 3 (2016), p. 160018. DOI: 10.1038/sdata.2016.18. URL: <https://doi.org/10.1038/sdata.2016.18>.

-
- [128] Greg Wilson et al. “Best Practices for Scientific Computing”. In: *PLOS Biology* 12.1 (2014), e1001745. doi: 10.1371/journal.pbio.1001745. URL: <https://doi.org/10.1371/journal.pbio.1001745>.
- [129] William Douglas Woody, David B. Daniel, and Crystal A. Baker. “E-books or textbooks: Students prefer textbooks”. In: *Computers & Education* 55.3 (Nov. 2010), pp. 945–948. ISSN: 0360-1315. doi: 10.1016/j.compedu.2010.04.005. URL: <http://dx.doi.org/10.1016/j.compedu.2010.04.005>.
- [130] Zhiyu Wu et al. *DeepSeek-VL2: Mixture-of-Experts Vision-Language Models for Advanced Multimodal Understanding*. 2024. arXiv: 2412.10302 [cs.CV]. URL: <https://arxiv.org/abs/2412.10302>.
- [131] Heming Xia et al. *Unlocking Efficiency in Large Language Model Inference: A Comprehensive Survey of Speculative Decoding*. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand, Aug. 2024. doi: 10.18653/v1/2024.findings-acl.456. URL: <https://aclanthology.org/2024.findings-acl.456/>.
- [132] Yang Xu et al. “LayoutLMv2: Multi-modal Pre-training for Visually-rich Document Understanding”. In: *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*. Ed. by Chengqing Zong et al. Online: Association for Computational Linguistics, Aug. 2021, pp. 2579–2591. doi: 10.18653/v1/2021.acl-long.201. URL: <https://aclanthology.org/2021.acl-long.201/>.
- [133] Yiheng Xu et al. “LayoutLM: Pre-training of Text and Layout for Document Image Understanding”. In: *KDD ’20: The 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, Aug. 20, 2020, pp. 1192–1200. doi: 10.1145/3394486.3403172. URL: <http://dx.doi.org/10.1145/3394486.3403172>.
- [134] An Yang et al. *Qwen3 Technical Report*. 2025. URL: <https://arxiv.org/abs/2505.09388>.
- [135] Nan Yang et al. *Inference with Reference: Lossless Acceleration of Large Language Models*. 2023. arXiv: 2304.04487 [cs.CL]. URL: <https://arxiv.org/abs/2304.04487>.
- [136] Yuan Yao et al. *MiniCPM-V: A GPT-4V Level MLLM on Your Phone*. 2024. doi: 10.48550/ARXIV.2408.01800. URL: <https://arxiv.org/abs/2408.01800>.

-
- [137] Jiabo Ye et al. “UReader: Universal OCR-free Visually-situated Language Understanding with Multimodal Large Language Model”. In: *Findings of the Association for Computational Linguistics: EMNLP 2023*. Ed. by Houda Bouamor, Juan Pino, and Kalika Bali. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 2841–2858. DOI: 10.18653/v1/2023.findings-emnlp.187. URL: <https://aclanthology.org/2023.findings-emnlp.187/>.
- [138] Junyuan Zhang et al. “OCR Hinders RAG: Evaluating the Cascading Impact of OCR on Retrieval-Augmented Generation”. In: (Oct. 2025), pp. 17443–17453. URL: https://openaccess.thecvf.com/content/ICCV2025/papers/Zhang_OCR_Hinders_RAG_Evaluating_the_Cascading_Impact_of_OCR_on_ICCV_2025_paper.pdf.
- [139] Pan Zhang et al. *InternLM-XComposer-2.5: A Versatile Large Vision Language Model Supporting Long-Contextual Input and Output*. 2024. DOI: 10.48550/ARXIV.2407.03320. URL: <https://arxiv.org/abs/2407.03320>.
- [140] Yongchao Zhou et al. “DistillSpec: Improving Speculative Decoding via Knowledge Distillation”. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: <https://openreview.net/forum?id=rsY6J3ZaTF>.



Publications

blah blah blah...

L^AT_EX Rainbow: Universal L^AT_EX to PDF Document Semantic & Layout Annotation Framework

Changxu Duan¹, Zhiyin Tan², Sabine Bartsch³

Technische Universität Darmstadt, Germany^{1,3}

¹changxu.duan@tu-darmstadt.de ²zhiyin.tanh@gmail.com

³sabine.bartsch@tu-darmstadt.de

Abstract

Machine Learning models in the field of Information Extraction for Scientific Publications require high-quality labeled data. The large amount of easily accessible L^AT_EX source code is a treasure trove of high-quality labeled data. However, existing datasets comprised of document collections and PDF extraction tools have limitations: (1) The hierarchical structure of papers is lost because labeling is done in terms of pages rather than documents; (2) The reading order is not extracted, which potentially muddles the extracted contextual structure; (3) Papers included in the datasets are not likely to be up-to-date. To address these challenges, we propose L^AT_EX Rainbow, a framework that bridges L^AT_EX to PDF that can automatically annotate and extract semantic and layout information from L^AT_EX source code. This framework extends existing annotation methods by taking into account the properties of different existing approaches. It can produce token-level semantic structure annotations, preserve the paper's reading order, and extract the table of contents, i.e., the article's section structure. L^AT_EX Rainbow enables anyone to extend their datasets with the latest documents. The project is open-sourced on GitHub¹ for community contributions and use.

1 Introduction

Scientific publications are often delivered in a form that is unstructured from the perspective of the underlying data, notably layout-focused formats such as PDFs. These formats, while visually appealing and optimized for human comprehension, present significant challenges when it comes to automatic Information Extraction (IE). For example, it is difficult for PDF extraction software to distinguish which part of a PDF page constitute the actual contents of the paper as opposed to other elements such as headers, metadata, author and affiliation

etc. and multimodal contents such as images, tables, equations etc. and their captions (Meuschke et al., 2023; Bast and Korzen, 2017). Additionally, these documents often contain elements that are not directly related to the core content, such as watermarks (Chia et al., 2018), publisher details and header information that serves navigation in collections. These elements, often introduced by the publishing process, further complicate the extraction process as they are not semantically linked to the main content. They appear within the layout of the page, but are hard to distinguish from the paper contents.

In order to solve these problems, there has been a surge in the development of document understanding machine learning models over the past few years (Cui, 2021; Subramani et al., 2021; Han et al., 2023). These models are designed to delve deep into documents, extracting semantic information by harnessing both their visual and textual attributes. However, machine learning, being a data-driven approach, requires extensive labeled data. Considering that existing PDF extraction tools cannot guarantee the accuracy of the extraction (Meuschke et al., 2023). In this context, L^AT_EX code has emerged as a valuable resource. Many of the weakly supervised annotated document IE datasets have their genesis in L^AT_EX code (Li et al., 2020b; Schmitt-Koopmann et al., 2022; Anitei et al., 2023).

L^AT_EX is a typesetting system commonly used for scientific publication, and L^AT_EX code can be easily compiled into PDF format. The explicit markup in L^AT_EX code describes the structure and formatting of the document. Given that scientific publications inherently maintain a hierarchical and semantic structure, such as sections, subsections, figures, tables, equations, etc. These elements are all clearly defined within L^AT_EX commands. This clarity facilitates automatic annotation systems in identifying and categorizing document elements, as they are al-

¹<https://github.com/InsightsNet/texannotate>

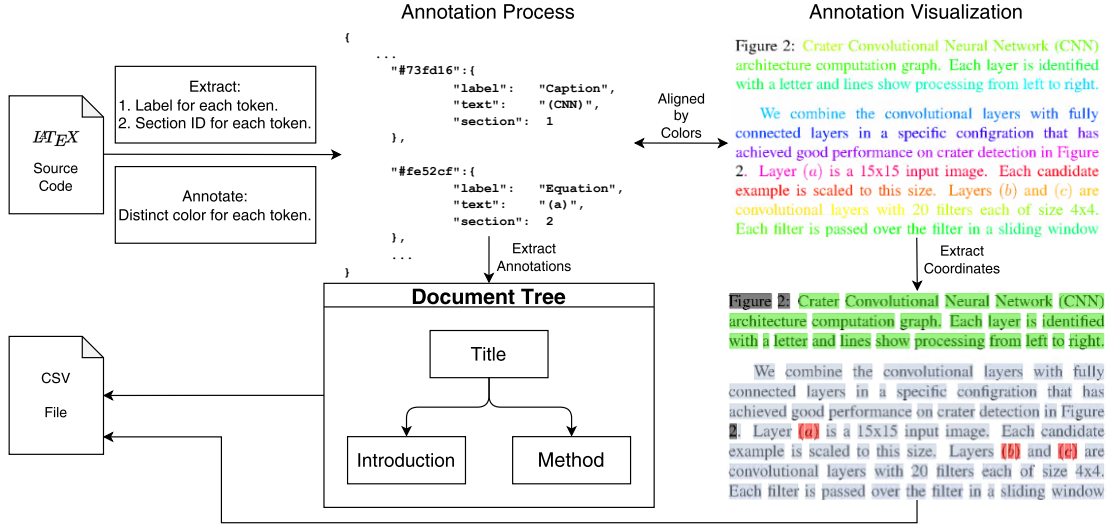


Figure 1: Process of document annotation and extraction. In this figure example, a paper (Cohen et al., 2016) from arXiv is annotated by color and have been extracted with the semantic layout label for each token.

ready explicitly delineated and classified by the author’s markup (Ogawa, 1994). The author’s intent can be inferred more effectively from the structural and semantic cues within $\text{\LaTeX}$ code, leading to a more accurate and context-aware interpretation of the document. With the increasing release of scientific publications in $\text{\LaTeX}$ source code, particularly on arXiv², there has been a surge in the number of PDF Information Extraction datasets derived from $\text{\LaTeX}$. These datasets predominantly utilize $\text{\LaTeX}$ coloring features, namely colored fonts or drawing of colored boxes within the PDF. However, there are notable shortcomings in the current PDF Information Extraction datasets. One of the key issues is that current annotations are often made on a per page basis, and no popular datasets explicitly annotate whether an element spanning two pages belongs to the same entity, e.g. a paragraph or itemized list. As a result, an element spanning across multiple pages might be interpreted as two distinct entities instead of one continuous element. This discrepancy also affects the hierarchical structure of scientific publications. Consider that document digitization standards, including Journal Article Tag Suite (JATS)³ and Text Encoding Initiative (TEI)⁴ provide definitions of section trees, which are beneficial to IE (Kikuchi et al., 2014; Hu et al., 2022; Landolsi et al., 2023). It would be a shame to lose

the hierarchy.

Additionally, recent models and datasets derived from $\text{\LaTeX}$ often omit reading order details (Li et al., 2020b; Blecher et al., 2023). $\text{\LaTeX}$ is a complex ecosystem with a vast collection of packages filled with numerous command definitions via Comprehensive TeX Archive Network (CTAN). Various templates each have their own unique writing conventions. This leads to the possibility that some elements may be mislabeled. Given that different publications adopt varied page layouts and $\text{\LaTeX}$ autonomously determines the positioning of tables and figures based on its internal rules (Mittelbach et al., 2004), there’s a significant risk that automated information extraction tools might misinterpret the intended reading order and context.

Moreover, authors of such resources do not always publish the code used in compiling the dataset. This means that current datasets are unlikely to incorporate the most recent papers or newer version of old papers. This hampers reproducibility of the process of dataset building as well as impeding scalability. Meanwhile, it is difficult for users to modify the annotation style to match their demands.

In this paper, we build upon several approaches for automatic PDF annotation of datasets and introduce a generalized framework that yields document-oriented, fine-grained, reading-ordered annotations that exclude extraneous content based on $\text{\LaTeX}$ source code. Figure 1 is a simplified representation of the labeling process. Our framework improves the accuracy and robustness of IE for scientific publications that has $\text{\LaTeX}$ code. Further-

²<https://info.arxiv.org/help/submit/index.html>

³<https://jats.nlm.nih.gov/articleauthoring/tag-library/1.3/element/sec.html>

⁴<https://tei-c.org/release/doc/tei-p5-doc/en/html/ref-tree.html>

more, we believe the new annotated data comes from our framework could drive more accurate IE machine learning models for PDF only papers, i.e. scanned paper. Our contributions are:

1. Our framework refine the categories, per document based reading order and hierarchy through a well-designed coloring strategy.
2. Enhancing code parsing capabilities by invoking the parsing databases of modern $\text{\LaTeX}$ integrated writing environment.
3. Providing CSV tables output per PDF document, easily modifiable to meet user needs.

This framework is free software and available under the Apache 2.0 license.

2 Related Work

2.1 PDF Information Extraction Softwares

Currently there are many software solutions or services that provide PDF content extraction functions. Adobe Extract⁵ and Apache Tika⁶ provide API service to extract texts from PDF, but they do not provide fine-grained labeling. Camelot⁷ and Tabula⁸ focus on table extraction. RefExtract⁹ specializes in extracting references. CERMINE (Tkaczyk et al., 2015), GROBID¹⁰ PdfAct¹¹ and Science Parse¹² support the identification of more categories. PyMuPDF¹³ allows access to information about the more underlying details of the PDF file. However, a benchmark demonstrates their imperfect performance (Meuschke et al., 2023).

2.2 Document Datasets

Many of the datasets’ annotation were taken from $\text{\LaTeX}$. TableBank (Li et al., 2020a) specialize in table extraction. DocBank (Li et al., 2020b) extended from TableBank, provides token-level fine-grained categories labeling. FormulaNet (Schmitt-Koopmann et al., 2022) and IBEM (Anitei et al., 2023) focus on mathematical formulas, especially in-line formulas, which can easily be confused with

plain-texts. SciBank (Grijalva et al., 2022) produces block-level annotations.

There are also many datasets from non- $\text{\LaTeX}$ sources. PubLayNet (Zhong et al., 2019) and DocLayNet (Pfitzmann et al., 2022) obtained particularly large amounts of labeling using automated and manual methods, respectively. XFUND (Xu et al., 2022) manually labeled multilingual tabular data. ReadingBank (Wang et al., 2021) is extracted from Microsoft Word documents, which standardize the reading order of blocks within a page. M⁶Doc (Cheng et al., 2023) extracted large-scale data using a half machine learning, half manual approach.

2.3 Document Understanding Models

With the gradual enrichment of document data IE resources, machine learning model development is driven by increasingly larger datasets using different approaches. LayoutLMs (Xu et al., 2020, 2021a,b; Huang et al., 2022) and its variants (Shen et al., 2022), make it possible to analyze document layout from the 2D coordinates of texts plus visual features. Donut (Kim et al., 2022), on the other hand, changes the structure of model to a separate visual encoder and language model decoder without obtaining texts directly from the document. Nougat (Blecher et al., 2023) follows Donut in implementing PDF to markup language conversion. However, Nougat’s approach is page-based, and cross-page paragraphs may be incorrectly sliced by figures or tables on a page. With the explosion of Large Language Models (LLM), a number of on-line document understanding systems have sprung up, such as Explainpaper¹⁴ and OpenRead¹⁵, but these platforms are commercial and closed-source. In this paper we present a framework that is freely available and can support the community to enhance the open-source approach.

3 Methodology

We present a five-step approach to annotate the $\text{\LaTeX}$ code to PDF annotation. (1) Initially, the PDF file is processed the existing font colors and shapes in the file are captured. (2) Subsequently, the corresponding $\text{\LaTeX}$ source code of the PDF is parsed. Each token within the file is assigned a distinct color. Furthermore, figures within the document are enclosed within borderless rectangles and are highlighted with unique background colors.

⁵<https://www.adobe.io/apis/documentcloud/dcsdk/pdf-extract.html>

⁶<https://tika.apache.org/>

⁷<https://github.com/camelot-dev/camelot>

⁸<https://github.com/chezou/tabula-py>

⁹<https://github.com/inspirehep/refextract>

¹⁰<https://github.com/kermitt2/grobid>

¹¹<https://github.com/ad-freiburg/pdfact>

¹²<https://github.com/allenai/science-parse>

¹³<https://github.com/pymupdf/PyMuPDF>

¹⁴<https://www.explainpaper.com/>

¹⁵<https://www.openread.academy/>

These specific colors act as pointers to the respective segments of the $\text{\LaTeX}$ source code, ensuring traceability. (3) Upon completing the $\text{\LaTeX}$ parsing, the source code is compiled into a color-coded PDF. (4) The framework then aligns text and figures from the PDF document with their respective segments in the $\text{\LaTeX}$ source code by color. This alignment facilitates the extraction of semantic annotations and coordinates of each token and the establishment of a hierarchical structure throughout the document. (5) Finally, we packaged all the annotation as CSV files.

In order to complete these steps, the framework needs three functions: PDF Element and Color Extraction, Color Generation and Annotation, and PDF Compilation. The subsequent sections provide the implementation of each function in detail.

3.1 Element and Color Extraction from PDF

In this function we use an off-the-shelf Python package `pdfplumber`¹⁶ to read the details inside the PDF file. This tool is proficient in pinpointing the position, font, and color of every character on a page. Additionally, it can determine the position and color attributes of all rectangles on the page, which encompasses both border and fill colors. By default, `pdfplumber` utilizes DeviceRGB color space, extracting colors as tuples of three floating-point numbers. For example, the color black is represented as (0.0, 0.0, 0.0) while red is (1.0, 0.0, 0.0). However, modern computer languages, sometimes struggle with accurately storing and accessing floating-point numbers. This inherent inaccuracy implies that color matching based on these numbers might be prone to errors, stemming from cumulative inaccuracies.

In our framework, colors for fonts are represented as tuples of 8-bit values, namely red is represented as (255, 0, 0) or `#ff0000` in hexadecimal. When `pdfplumber` extracts colors from PDF document, each tuple element value is incremented in steps of 0.00392, for instance, 8-bit (0, 1, 2) translates to floating-point (0.0, 0.00392, 0.00784). Given that $\frac{1}{255} = 0.00392156862$, we are already dealing with a discrepancy. To mitigate this threat, we employed `matplotlib`'s `to_hex()`¹⁷ method to ensure precise floating-point to 8-bit RGB value matches. We also provide details in the selection of tools for extracting color in Appendix A.

¹⁶<https://github.com/jsvine/pdfplumber>

¹⁷https://matplotlib.org/stable/api/_as_gen/matplotlib.colors.to_hex.html

3.2 Colors Generation and Annotation

In this function, we process the $\text{\LaTeX}$ source code to determine color assignments for each element. Ideally, each element should have a unique color. A straightforward approach would be to incrementally assign hexadecimal numbers from `#000000` to `#ffffff`. However in practice we have found that such an increment leads to a very insignificant color change. For instance, the token gradually changes from black `#000000` to a blue shade almost indistinguishable from black `#000001`, then `#000002`, until it reaches full blue `#0000ff`. This makes neighboring tokens almost the same color, and distinguishing between them can be challenging for both computer displays and humans eyes.

To enhance visibility and facilitate manual error-checking, we adopted a hue-based color generation strategy. More specifically, we use the Hue, Saturation, Value (HSV) color space to cyclically extract colors and rearrange them into appropriate groups. Each HSV color is then converted to an RGB tuple. Finally, around 9 million colors were grouped and sorted to be used in the next step of color marking.

For every recognized token, it gets substituted with:

```
\color[RGB]{0,0,1}<TOKEN>
```

Where `<token>` is the token to be colored. Each identified figure is replaced with:

```
\colorbox[rgb]{0,0,1}{<FIGURE>}
```

Where `<FIGURE>` is the command of including the figure file, or the block of drawing a figure. Distinct colors are allocated to each segment.

We also insert rules that ensure the required packages are imported and any rectangle placed beneath an figure does not disrupt the document's original layout.

```
\usepackage{xcolor}
\usepackage{tcolorbox}
\setlength{\fboxsep}{0pt}
\setlength{\fboxrule}{0pt}
```

Next we need to parse $\text{\LaTeX}$ source code and apply the above annotation rules to them.

3.2.1 $\text{\LaTeX}$ Parsing and Annotation

Broadly, elements in $\text{\LaTeX}$ source code comprises four classes: body text, macro, environment, and comments. We have the following parsing strategies for each of these elements.

- **Body text** segment undergoes tokenization using spaCy¹⁸ tokenizer in order to split punctuation correctly. We track of the number of space characters after each token to maintain the integrity the original PDF page layout. Each token is color-marked and recorded.
- **Macro** begins with a backslash, and its arguments are encompassed within braces. We focus on labeling certain macros such as `\title{}` and `\author{}`, attributing their parameter literals with relevant semantic structure labels. Arguments within the curly braces will be parsed as body text if it will appear in the compiled PDF. Notably, `\includegraphics{}` will be treated as a whole and marked with a colorful borderless rectangle, as it is an inserted figure without fonts. `\input{}` and `\include{}` will point to another source code file, and we recursively parse the contents of the file.
- **Environment** consists of entities encapsulated between start and end commands. Elements within a environment are recursively parsed as macro or body text. For example, `\begin{table}...\end{table}` is a table environment and the elements within this table element such as `\caption{}` will be parsed as macro. Specifically, drawing environment `\begin{tikzpicture}...\end{tikzpicture}` will be treated as a whole block and marked with a colorful borderless rectangle. Note that only elements within the document environment are colored.
- **Comment** element starts with a percent sign `%` and continues until the end of this line in the $\text{\LaTeX}$ source code. We ignore the annotation of comment as it does not affect any part of the compiled PDF.

We employ the Python package `pylatexenc`¹⁹ to traverse and parse the $\text{\LaTeX}$ source code, character by character. `pylatexenc` contains a collection of commands created by the contributors, which defines: the name of commands; whether the command has a variant or not e.g. `\section{}` and `\section*{}`; and the number of command arguments, including optional arguments in square brackets and required arguments in curly braces.

¹⁸<https://spacy.io/>

¹⁹<https://github.com/phfaist/pylatexenc>

In practice, we found these predefined rules insufficient, prompting us to manually augment the definition file.

However, as the number of parsed source codes increases, the trend of newly encountered undefined commands does not stop. We introduced the parsing database from $\text{\LaTeX}$ Workshop²⁰ and `TeXstudio`²¹ in order to extend our database of parsing rules once and for all. $\text{\LaTeX}$ Workshop is an extension for a popular code editor Visual Studio Code²², aiming to provide all-in-one features and utilities for $\text{\LaTeX}$ typesetting. `TeXstudio` is an integrated writing environment for creating $\text{\LaTeX}$ documents. They are featured by a particularly complete database of automatically generated commands from CTAN. We implemented the method to download all the definitions from the repository in JSON format. They are stored on a package-by-package basis, i.e., each package that can be referenced by $\text{\LaTeX}$ with `\usepackage{}` has a corresponding JSON file that contains all the commands for the package, including macros and environments. Our parser first traverses through $\text{\LaTeX}$ source code, collects all package loads, and then reads the relevant JSON entries. These commands are subsequently transformed and integrated into the parsing rules for `pylatexenc`.

Next, we allocate each color the semantic layout label to which it belongs. The set of labels aligns with those used in GROBID and DocBank:

- **Abstract** is assigned to body texts within abstract environment, or argument of macro `\abstract{}` and its variants.
- **Author** is assigned to argument of macro `\author[]{}{}`, `\address{}` and their variants.
- **Caption** is assigned to macro `\caption{}` within table or figure environment.
- **Equation** is assigned to all the element that marked with mathematical mode by `pylatexenc`.
- **Figure** is assigned to drawings or imported figures.
- **List** is assigned to body texts within `itemize` or `enumerate` environment.

²⁰<https://github.com/James-Yu/LaTeX-Workshop>

²¹<https://www.texstudio.org/>

²²<https://code.visualstudio.com/>

- **Paragraph** is assigned to body texts within document environment.
- **Reference** is assigned to body texts within bibliography environment, or macro `\bibliography{}`.
- **Section** is assigned to macros that indicate a new section in command.
- **Table** is assigned to environments that include tabular in command.
- **Title** is assigned to argument of macro `\title[]{}{}` and its variants.

With the annotations in place, the $\text{\LaTeX}$ source code is ready for compilation. Following this, `pdfplumber` could determine the color and position of each letter, utilizing the letter's color to match the corresponding annotation.

Simultaneously, the article's hierarchical structure is encapsulated within a tree data structure. $\text{\LaTeX}$ delineates a hierarchy spanning seven levels through its macro command²³. We record for each colored element the node of the tree it belongs to. To ensure a coherent hierarchy, we additionally define the paper title as the root level of document. That is, all tokens in the title of a scientific paper belong to Title node, while all tokens in the Introduction section belong to the Introduction node, and Introduction node is a child of Title node in the tree of this document. In addition we discuss the argument of hierarchical structure in Appendix B.

3.3 PDF Compilation

To initiate PDF Compilation process, two specific lines of code are added to the beginning of the source file: `\pdfoutput=1` instructs the compiler to produce a PDF instead of PostScript, an alternative publication format; `\interactionmode=1` signals the compiler to persist with the output generation, even if it encounters an error on a page.

Publications that accept submissions in $\text{\LaTeX}$ format, including arXiv, often recommend using `pdftex` as their preferred rendering engine. This engine is integrated into the contemporary $\text{\LaTeX}$ distribution, TeXLive. Given the complexities in setting up this distribution, we opted for the Docker image²⁴ of TeXLive 2023 to establish our compilation environment.

²³https://www.overleaf.com/learn/latex/Sections_and_chapters

²⁴<https://hub.docker.com/r/texlive/texlive>

Automated $\text{\LaTeX}$ compilation presents a challenge, especially in pinpointing the master source file. This is because $\text{\LaTeX}$ allows for multiple .tex source files to be consolidated and compiled into one overarching master PDF. To navigate this challenge, we integrated arXiv's AutoTeX²⁵ automatic compilation system. AutoTeX, a Perl-based toolkit, excels at discerning the primary source file within a project. Our PDF compilation mechanism derives some of its functionalities from an open-source AutoTeX wrapper²⁶.

However, during our practice, we observed AutoTeX's compilation regulations as overly stringent. There were instances when it halted the compilation due to minor errors, even when the same content had successfully passed arXiv's publication standards. Specifically, AutoTeX will stop compiling and try other compilers immediately after the compiler returns an error code, even if the ignored error signal placed in front of the compiler to successfully output the compiled PDF file. The PDF file produced in this step is wrong because of the BibTeX mechanism. BibTeX is reference management software for formatting lists of references, which needs to be run twice by the compiler in order to correctly output references, and in-text citations. To address this, we modified AutoTeX's code²⁷, enabling it to bypass certain errors and persist with PDF generation. We configured AutoTeX to control the compiler to run at least twice, even if an error code is returned initially.

In conclusion, we combine TeXLive 2023, AutoTeX, and a Python-based API service into a single container. This container, accessible via HTTP, accepts source code and efficiently returns the compilation outcomes.

4 Capabilities Overview

During the export phase, outputs are organized into two DataFrames:

- The first DataFrame represents the Table of Contents nodes, which was created during the annotation process. Each line denotes a tree node, with every node possessing a unique ID and an ID indicating its parent. This structured approach ensures that the succeeding page

²⁵<https://metacpan.org/pod/TeX::AutoTeX>

²⁶<https://github.com/andrewhead/texcompile>

²⁷The modified AutoTeX can be found at <https://github.com/Fireblossom/TeX-AutoTeX-Mod>

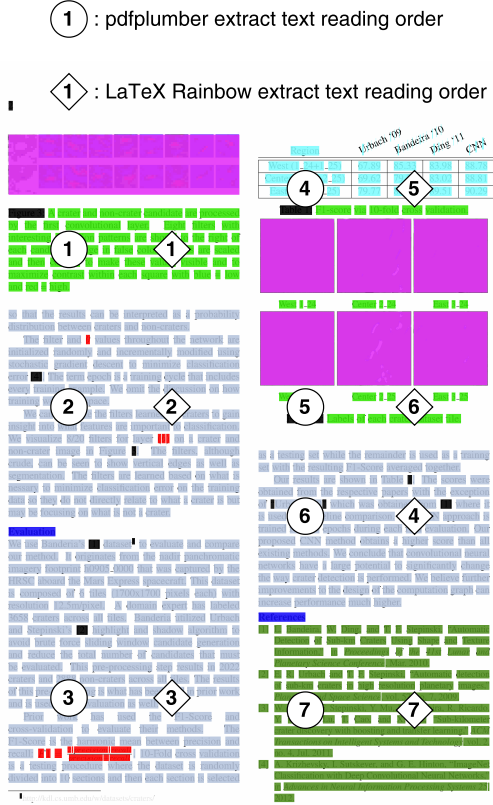


Figure 2: Extracting reading order from an elementally complex page. This example is taken from the second page of (Cohen et al., 2016).

elements to be exported can be systematically assigned to their respective nodes.

- Every row in the second DataFrame denotes either a figure or token extracted from the PDF. These tokens are allocated a number indicating their reading order and a section ID, if they are part of the author’s main content, starting from 0. A value of -1 in reading order indicates elements not penned by the author, and auto-generated by the $\text{\LaTeX}$ template. This facilitates a straightforward exclusion of such elements during further analysis. The label column encompasses semantic structure labels including: Abstract, Author, Caption, Equation, Figure, Footer, List, Paragraph, Reference, Section, Table, and Title.

Both DataFrames can be exported as CSV files.

4.1 Use Case: Text Extraction with Reading Orders

Reading order refers to the sequence in which content is meant to be read. In documents, it usually

follows a top-to-bottom, left-to-right pattern, but there can be exceptions, especially in multilingual or complex layout documents.

Figure 2 illustrates such a scenario. The labels in the figure represent the order of the text extracted by the different methods. In this case, starting from the sixth block, the paragraphs extracted using pdfplumber are interrupted by tables and pictures.

4.2 Use Case: Section-Weighted Scientific Paper Summarization using LLM

Scientific papers are structured documents with different sections, each serving a distinct purpose (Wcg, 2008). While traditional summarization techniques consider the entire document holistically (Ibrahim Altmami and El Bachir Menai, 2022), section-weighted summarization assigns different weights to different sections, recognizing that some parts of a paper may be more informative or critical than others for a quick understanding (Cohan et al., 2018). The advantage of our framework is the ability to accurately obtain sections as compared to any current PDF extraction tool. We provide a simple example in our GitHub repository.

5 Validation of Annotations

In addition to the unique features introduced in Section 4, we also have to verify its consistency with the annotation methods for existing datasets. We use the DocBank dataset (Li et al., 2020b) to assess the reliability of annotations generated by $\text{\LaTeX}$ Rainbow framework. Docbank dataset contains annotations for 1.5 million content elements across 500K scientific publication pages. It comprises papers from arXiv published between 2014 and 2018, spanning fields like physics, mathematics, and computer science. Due to its extensive size, range of subjects, numerous annotated elements, and labeling method, Docbank is considered a benchmark dataset for $\text{\LaTeX}$ sources.

Since DocBank is a very large dataset, we extracted a subset for time and feasibility reasons. We extracted $\text{\LaTeX}$ source code of 100 papers in DocBank from arXiv. They are then annotated and compiled by our framework, 61 papers are successfully annotated and compiled. 39 papers raise errors. We summarize the reasons and numbers for failures:

1. Parsing errors in the source code, such as unmatched bracket pairs or expressions that are digestible by pdfTeX but not by pyLaTeXenc,

Labels	Precision	Recall	F_1 -Score
Abstract	0.9779	0.8197	0.8918
Author	0.5027	0.5515	0.5260
Caption	0.4676	0.3851	0.4224
Equation	0.1957	0.9016	0.3216
Footer	0.2029	0.2612	0.2284
List	0.4462	0.1762	0.2526
Paragraph	0.9379	0.6080	0.7377
Reference	0.8355	0.9718	0.8985
Section	0.5119	0.3777	0.4347
Table	0.8806	0.7939	0.8350
Title	0.3429	0.5320	0.4170

Table 1: Precision, Recall, and F_1 -Score of annotations from L^AT_EX Rainbow, compare to DocBank.

cause the parser to misread and yield empty results. (23)

2. Compilation failures. It was reported²⁸ that some source code could not be compiled successfully due to change of compilation environment. (15)
3. File encoding problem. The source code contains characters that Python cannot handle. (1)

To gauge consistency across annotated datasets, Cohen’s Kappa Coefficient $\kappa = \frac{p_o - p_e}{1 - p_e}$ is applied to test consistency across annotated datasets. where p_o is the empirical probability of agreement on the label assigned to any sample, and p_e is the expected agreement when both annotators assign labels randomly. p_e is estimated using a per-annotator empirical prior over the class labels (Artstein and Poesio, 2008). Cohen suggested the Kappa result be interpreted as follows: values ≤ 0 as indicating no agreement and 0.01–0.20 as none to slight, 0.21–0.40 as fair, 0.41–0.60 as moderate, 0.61–0.80 as substantial, and 0.81–1.00 as almost perfect agreement (McHugh, 2012). We get $\kappa = 0.32$. This value demonstrates the fair consistency of our approach with the existing baseline.

We further assessed Precision, Recall, and F_1 -Score using DocBank’s annotations as the gold standard. As detailed in Table 1, our framework’s annotations align closely with DocBank in the Abstract, Paragraph, Reference, and Table categories, and differs in other categories. Notably, there is a considerable degree of inconsistency in the semantic labeling of some categories. The reasons

for these inconsistencies are mainly differences in annotation strategies and difficulties in aligning our annotations with DocBank. We delve deeper into the inconsistencies in Appendix C.

6 Known Issues and Future Work

In the future, our primary goal is to update the parser so that it can tolerate syntax errors that the pdfTeX compiler can tolerate.

We also plan to expand the L^AT_EX Rainbow framework with parallelization capabilities. Because the pdfTeX engine does not support multi-threaded parallelism, this makes them slow to compile, especially for long files. The idea is to enable parallelism at the scale of multiple files. Given the containerized nature of our PDF compilation service, this transition should be seamless.

For extensive projects, consider the example of *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, which comprises 912 papers. The cumulative number of tokens in such projects might surpass the maximum number of colors that can be allocated (9 million). In these cases, the L^AT_EX Rainbow framework is unable to perform the annotation. Our next steps include refining the color system to enhance its usability and distinctiveness.

Coloring the L^AT_EX element doesn’t always work. For example, `\url{}` command forces its arguments to be blue, instead of the color we assigned. In addition to the coloring method, there is also the SyncTeX (Laurens, 2008) plugin that allows compiled PDF elements to be linked back to the L^AT_EX source code. It is directly involved in the compilation process and records the correspondence through the internal auxiliary files. Our plan is to go deeper into its mechanisms to establish a more robust PDF to L^AT_EX source code correspondence.

Different publisher templates interpret L^AT_EX terms uniquely, making it challenging for our database to account for every variation. For example, `\lstinputlisting[[]]{}{}` defines the content to be displayed to the PDF with a substring value in the optional argument. This is significantly different from the definition of most commands. As mentioned in Section 5, there are many papers that cannot yet be parsed correctly. Therefore, we greatly welcome and depend on the open-source community to contribute the detailed parsing rules for each template.

²⁸<https://info.arxiv.org/help/faq/texlive.html>

7 Conclusion

In this paper we introduce a framework which can be used to establish a correspondence between L^AT_EX code and PDF elements, exporting detailed semantic annotations. Our framework meticulously extracts semantic markup, maintaining the layout fidelity of the associated PDF files. The structured information extracted by our framework helps in better document indexing, searching and analysis. It improves document accessibility and helps develop and refine document understanding tools.

Our framework is more than just yet another toolkit to the growing list of document datasets. By ensuring versatility and adaptability as well as scalability, we aim for it to become a universal tool that can facilitate enhanced document analysis across multiple disciplines and applications. We sincerely hope that open-source community can derive innovative uses and benefits from our solution.

8 Acknowledgments

The framework reported in this paper was conducted within the research project InsightsNet (<https://insightsnet.org/>) which is funded by the Federal Ministry of Education and Research (BMBF) under grant no. 01UG2130A.

References

- Dan Anitei, Joan Andreu Sánchez, José Miguel Benedí, and Ernesto Noya. 2023. [The ibem dataset: A large printed scientific image dataset for indexing and searching mathematical expressions](#). *Pattern Recognition Letters*, 172:29–36.
- Ron Artstein and Massimo Poesio. 2008. Inter-coder agreement for computational linguistics. *Computational linguistics*, 34(4):555–596.
- Hannah Bast and Claudius Korzen. 2017. A benchmark and evaluation for text extraction from pdf. In *Proceedings of the 17th ACM/IEEE Joint Conference on Digital Libraries, JCDL '17*, page 99–108. IEEE Press.
- Lukas Blecher, Guillem Cucurull, Thomas Scialom, and Robert Stojnic. 2023. [Nougat: Neural optical understanding for academic documents](#).
- Hiuyi Cheng, Peirong Zhang, Sihang Wu, Jiaxin Zhang, Qiyuan Zhu, Zecheng Xie, Jing Li, Kai Ding, and Lianwen Jin. 2023. M6doc: A large-scale multi-format, multi-type, multi-layout, multi-language, multi-annotation category dataset for modern document layout analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 15138–15147.
- Wai Chong Chia, Phoey Lee Teh, and Colin Mathew Hew D Gill. 2018. [Text extraction and categorization from watermark scientific document in bulk](#). In *2018 3rd International Conference on Computational Intelligence and Applications (ICCIA)*, pages 47–51.
- Arman Cohan, Franck Dernoncourt, Doo Soon Kim, Trung Bui, Seokhwan Kim, Walter Chang, and Nazli Goharian. 2018. [A discourse-aware attention model for abstractive summarization of long documents](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 2 (Short Papers)*, pages 615–621, New Orleans, Louisiana. Association for Computational Linguistics.
- Joseph Paul Cohen, Henry Z Lo, Tingting Lu, and Wei Ding. 2016. [Crater detection via convolutional neural networks](#). *47th Lunar and Planetary Science Conference*.
- Lei Cui. 2021. *Document AI: Benchmarks, Models and Applications (Presentation@ICDAR 2021)*. DIL workshop in ICDAR 2021.
- Felipe Grijalva, Carla Parra, Marco Gallardo, Erick Santos, Byron Acuña, Juan Carlos Rodríguez, and Julio Larco. 2022. [Scibank: A large dataset of annotated scientific paper regions for document layout analysis](#).
- Xue Han, Yi-Tong Wang, Jun-Lan Feng, Chao Deng, Zhan-Heng Chen, Yu-An Huang, Hui Su, Lun Hu, and Peng-Wei Hu. 2023. [A survey of transformer-based multimodal pre-trained modals](#). *Neurocomputing*, 515:89–106.
- Faegheh Hasibi and Svein Erik Bratsberg. 2014. [Non-hierarchical structures: How to model and index overlaps?](#)
- P. Hu, Z. Zhang, J. Zhang, J. Du, and J. Wu. 2022. [Multimodal tree decoder for table of contents extraction in document images](#). In *2022 26th International Conference on Pattern Recognition (ICPR)*, pages 1756–1762, Los Alamitos, CA, USA. IEEE Computer Society.
- Yupan Huang, Tengchao Lv, Lei Cui, Yutong Lu, and Furu Wei. 2022. [Layoutlmv3: Pre-training for document ai with unified text and image masking](#). In *Proceedings of the 30th ACM International Conference on Multimedia, MM '22*, page 4083–4091, New York, NY, USA. Association for Computing Machinery.
- Nouf Ibrahim Altmami and Mohamed El Bachir Menai. 2022. [Automatic summarization of scientific articles: A survey](#). *Journal of King Saud University - Computer and Information Sciences*, 34(4):1011–1028.
- Yuta Kikuchi, Tsutomu Hirao, Hiroya Takamura, Manabu Okumura, and Masaaki Nagata. 2014. [Single document summarization based on nested tree structure](#). In *Proceedings of the 52nd Annual Meeting of*

- the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 315–320, Baltimore, Maryland. Association for Computational Linguistics.
- Geewook Kim, Teakgyu Hong, Moonbin Yim, JeongYeon Nam, Jinyoung Park, Jinyeong Yim, Wonseok Hwang, Sangdoo Yun, Dongyoon Han, and Seunghyun Park. 2022. Ocr-free document understanding transformer. In *European Conference on Computer Vision (ECCV)*.
- Mohamed Yassine Landolsi, Lobna Hlaoua, and Lotfi Ben Romdhane. 2023. [Hybrid method to automatically extract medical document tree structure](#). *Engineering Applications of Artificial Intelligence*, 120:105922.
- Jérôme Laurens. 2008. [Direct and reverse synchronization with SyncTeX](#). 29(3):365–371.
- Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, Ming Zhou, and Zhoujun Li. 2020a. [TableBank: Table benchmark for image-based table detection and recognition](#). In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 1918–1925, Marseille, France. European Language Resources Association.
- Minghao Li, Yiheng Xu, Lei Cui, Shaohan Huang, Furu Wei, Zhoujun Li, and Ming Zhou. 2020b. [DocBank: A benchmark dataset for document layout analysis](#). In *Proceedings of the 28th International Conference on Computational Linguistics*, pages 949–960, Barcelona, Spain (Online). International Committee on Computational Linguistics.
- Yves Marcoux, Michael Sperberg-McQueen, and Claus Huitfeldt. 2013. [Modeling overlapping structures](#). *Balises Series on Markup Technologies*.
- Mary L McHugh. 2012. Interrater reliability: the kappa statistic. *Biochemia medica*, 22(3):276–282.
- Norman Meuschke, Apurva Jagdale, Timo Spinde, Jelena Mitrović, and Bela Gipp. 2023. [A Benchmark of PDF Information Extraction Tools Using a Multi-task and Multi-domain Evaluation Framework for Academic Documents](#). In *Information for a Better World: Normality, Virtuality, Physicality, Inclusivity*, volume 13972 of *LNCIS*, pages 383–405. Springer Nature Switzerland, Cham.
- Frank Mittelbach, Michel Goossens, Johannes Braams, David Carlisle, and Chris Rowley. 2004. *The LaTeX Companion*, 2 edition. Addison-Wesley. ISBN-10: 0201362996 ISBN-13: 978-0201362992.
- Arthur Ogawa. 1994. [Object-oriented programming, descriptive markup, and tex](#). *TUGboat*, 15(3):325–330.
- Birgit Pfizmann, Christoph Auer, Michele Dolfi, Ahmed S. Nassar, and Peter Staar. 2022. [Doclaynet: A large human-annotated dataset for document-layout segmentation](#). page 3743–3751.
- Felix M. Schmitt-Koopmann, Elaine M. Huang, Hans-Peter Hutter, Thilo Stadelmann, and Alireza Darvishy. 2022. [Formulanet: A benchmark dataset for mathematical formula detection](#). *IEEE Access*, 10:91588–91596.
- Zejiang Shen, Kyle Lo, Lucy Lu Wang, Bailey Kuehl, Daniel S. Weld, and Doug Downey. 2022. [VILA: Improving structured content extraction from scientific PDFs using visual layout groups](#). *Transactions of the Association for Computational Linguistics*, 10:376–392.
- Nishant Subramani, Alexandre Matton, Malcolm Greaves, and Adrian Lam. 2021. [A survey of deep learning approaches for ocr and document understanding](#).
- Dominika Tkaczyk, Paweł Szostek, Mateusz Fedoryszak, Piotr Jan Dendek, and Łukasz Bolikowski. 2015. Cermine: automatic extraction of structured metadata from scientific literature. *International Journal on Document Analysis and Recognition (IJ-DAR)*, 18:317–335.
- Zilong Wang, Yiheng Xu, Lei Cui, Jingbo Shang, and Furu Wei. 2021. [LayoutReader: Pre-training of text and layout for reading order detection](#). In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 4735–4744, Online and Punta Cana, Dominican Republic. Association for Computational Linguistics.
- Peh Wcg. 2008. [Basic structure and types of scientific papers](#). *Singapore Med J*, 49(7).
- Yang Xu, Yiheng Xu, Tengchao Lv, Lei Cui, Furu Wei, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, Wanxiang Che, Min Zhang, and Lidong Zhou. 2021a. [LayoutLMv2: Multi-modal pre-training for visually-rich document understanding](#). In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pages 2579–2591, Online. Association for Computational Linguistics.
- Yiheng Xu, Minghao Li, Lei Cui, Shaohan Huang, Furu Wei, and Ming Zhou. 2020. [Layoutlm: Pre-training of text and layout for document image understanding](#). In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery Data Mining, KDD '20*, page 1192–1200, New York, NY, USA. Association for Computing Machinery.
- Yiheng Xu, Tengchao Lv, Lei Cui, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, and Furu Wei. 2021b. [Layoutxlm: Multimodal pre-training for multilingual visually-rich document understanding](#).
- Yiheng Xu, Tengchao Lv, Lei Cui, Guoxin Wang, Yijuan Lu, Dinei Florencio, Cha Zhang, and Furu Wei. 2022. [XFUND: A benchmark dataset for multilingual visually rich form understanding](#). In *Findings of the Association for Computational Linguistics: ACL 2022*, pages 3214–3224, Dublin, Ireland. Association for Computational Linguistics.

Xu Zhong, Jianbin Tang, and Antonio Jimeno Yepes. 2019. [Publaynet: largest dataset ever for document layout analysis](#). In *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pages 1015–1022. IEEE.

A Selection of Color Extraction Tools

In Addition to pdfplumber. We also experimented with the PyMuPDF²⁹ package for color extraction, enticed by its capacity to extract colors as integer values. However, it uses the sRGB color space, which introduced mismatches between our annotations and extracted values. A notable misalignment was observed with colors #000000 and #000100 in RGB are both being misconstrued as the singular color #000000 in sRGB. As PyMuPDF features in able to extract more refined information, such as block number, line number, whether it is italicized or superscript, and so on. Some valuable information is lost by not using this package. We will continue to study underlying logic of the packages and adjust the implementation described in this paper.

B Note on Non-hierarchical Structure of Document

In the realm of document digitization, there’s ongoing debate regarding the hierarchical structure of articles. Specifically, discussions revolve around overlapping structures within the XML and TEI communities³⁰ (Marcoux et al., 2013; Hasibi and Bratsberg, 2014). Overlaps arise when a document embodies multiple structures that intersect non-hierarchically, making it impossible to represent the document as a tree. Such as a metrical structure of feet and lines in poetry. Since our exported tree isn’t an XML file, we can sidestep the non-hierarchical structure issue by distinctly defining section elements and overlapping elements.

C Inconsistencies to DocBank

There are several categories in Table 1 that differ very much. In this section we summarize the potential causes from two perspectives.

C.1 Labeling Strategies

Among them Equation and Title have particularly high recall values and very low precision. Additionally, Paragraph’s recall is relatively low. This is due

²⁹<https://github.com/pymupdf/PyMuPDF>

³⁰<https://tei-c.org/release/doc/tei-p5-doc/it/html/NH.html>

to the difference in annotation strategies between DocBank and our approach.

More specifically, our rules recognize more commands as Title and Equation than Paragraph in DocBank.

- For Title, DocBank only recognizes the exact `\title{}`, while $\text{\LaTeX}$ Rainbow framework recognizes all commands that contain the word title. Such as `\aistatstitle{}` or `\begin{title}`.
- For Equation, we not only recognize specific commands like `\begin{equation}`. We also annotate the mathematical expressions on the line. We do realize that it can introduce potential inaccuracies, for example in practice we have found in-line formulas used as italics instead of mathematical formulas. We will refine the rules in future updates.
- In DocBank, any text not color-coded, including page numbers, headers, and copyrights, is defined as a Paragraph. In contrast, $\text{\LaTeX}$ Rainbow doesn’t tag these elements with a semantic layout label. This approach in DocBank seems imprecise and could introduce potential biases.

C.2 DocBank and $\text{\LaTeX}$ Rainbow annotations are not aligned

In practical we found that using tokens position doesn’t consistently match the labeling. Namely, for identical pages of the same paper from arXiv, the tokens in the same locations differ between DocBank and $\text{\LaTeX}$ Rainbow. Upon close examination of the annotated pages, we observed that DocBank’s annotation coordinates diverge from the arXiv document’s. We summarize two reasons for this.

- Papers may have been updated since the release of DocBank and we annotate the latest version of the paper. This may result in changes to the content and layout of the paper.
- Changes in compilation environments and compiler versions may also have led to subtle differences in compiled PDF layout.

For tokens that could not be linked, we had to use matching of contexts and tokens, which may have caused misalignment. This in turn affects the evaluation.

The evaluation is available as a Jupyter notebook in the GitHub repository.

Bridging scientific publication accessibility: L^AT_EX–markup–PDF–alignment

Changxu Duan

Abstract

This paper introduces a method to enhance the accessibility of scientific publications across multiple formats. Prior initiatives have predominantly centered on transforming L^AT_EX source code or PDF documents into markup languages like XML and Markdown. However, such methods typically overlook preserving the visual layout inherent in PDF pages. The approach in this paper exploits the color properties in L^AT_EX code to maintain consistency and alignment between the visual presentation of documents in PDF format and the digital presentation of markup languages and L^AT_EX code. This strategy not only fills in the gaps of previous approaches but also promotes the integration and accessibility of scientific document formats.

1 Introduction

Accessibility in PDFs ensures that documents are usable by everyone, including those with disabilities, by making them navigable and readable through assistive technologies. This involves structuring PDFs with metadata tags to define reading order and document elements, embedding text attributes for readability, providing alternative text for visual content, and including navigational aids like bookmarks. These measures, aligned with standards like PDF/UA, ensure that the PDFs are not only accessible but also comply with legal requirements for inclusivity [8].

Accessible PDFs benefit not only individuals with disabilities but also the broader user base. Features that make documents accessible, such as clear navigation and structured headings, improve the overall user experience and enhance the document's usability for everyone. These features make it easier to navigate through the text, find information quickly, and convert the document into other formats as needed. Additionally, the structure required for accessibility, such as tagged PDFs, aids in the correct reflow of text and associated graphics when adjusting the size of a document or its viewing mode. This adaptability is essential as digital content is increasingly accessed on a diverse array of devices, including smartphones and tablets.

L^AT_EX-sourced PDFs often lack accessibility primarily due to the inherent complexity and customization capabilities of L^AT_EX. The L^AT_EX system allows for a vast array of macros and packages, enabling highly complex document structures that do not au-

tomatically support accessible features necessary for assistive technologies, such as structured headings and alternative text for images. Unlike modern document creation tools that include built-in accessibility features, traditional L^AT_EX compilers like pdfL^AT_EX and XeL^AT_EX do not inherently support tagging.

Moreover, even with L^AT_EX packages designed to facilitate tagging, such as `tagpdf` [5, 15, 16], integrating these features requires significant technical expertise and meticulous configuration. The numerous L^AT_EX macros can interact unpredictably, potentially undermining the structural integrity needed for accessible documents. Additionally, the T_EX engines that process L^AT_EX are primarily focused on print quality, not digital accessibility. This focus, combined with a general lack of awareness among L^AT_EX users about accessibility standards, further complicates the production of accessible PDFs. The responsibility for ensuring accessibility often falls on the authors, who must navigate the steep learning curve of both L^AT_EX and accessibility requirements.

Other studies on L^AT_EX accessibility have converted L^AT_EX directly into a markup language, skipping the generation of PDF files [2]. While converting L^AT_EX to markup languages directly might improve accessibility by leveraging the inherent structural and semantic capabilities of HTML or XML, it also means losing out on the robust, cross-platform fidelity and rich feature set that PDFs offer.

In this paper, I explore a method that adds accessibility to existing PDF files without manually altering the existing L^AT_EX code.

2 Related work

2.1 L^AT_EX to markup transformation

Currently, two tools capable of converting L^AT_EX source code to markup languages are L^AT_EX_{XML} [13] and Pandoc [12]. However, both tools directly convert from L^AT_EX to markup languages like HTML or Markdown without compiling to PDF first. This approach bypasses some of the benefits that come from generating and utilizing PDF files, including the precise control over layout and typography that PDFs offer.

2.2 L^AT_EX–PDF alignment methods

The current methods for aligning PDF and L^AT_EX content typically rely on kernel patching or color-based alignment techniques. One prominent kernel-based alignment method is SyncT_EX [9], widely implemented across various L^AT_EX editors. This method provides a dynamic link between the source L^AT_EX code and the generated PDF, facilitating easy navigation between them. Color-based alignment methods

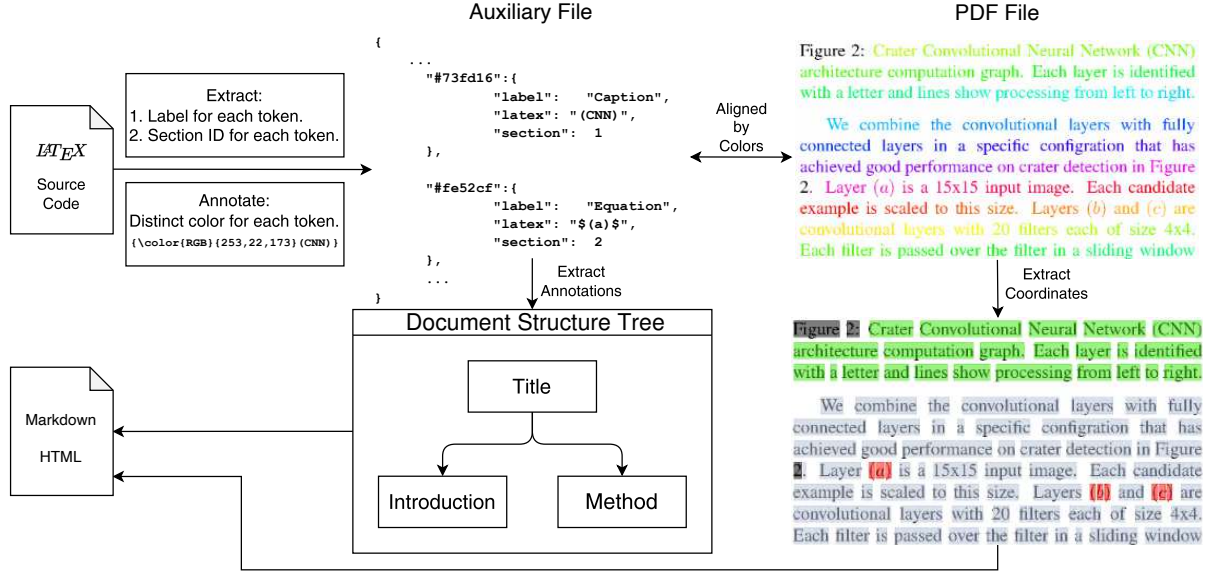


Figure 1: The overall process of accessibility annotation. The alignment of $\text{\LaTeX}$ -markup-PDF is built up with auxiliary files.

include approaches like those demonstrated by the DocBank [11] dataset. In this dataset, different types of page elements are assigned unique colors, which are then used to facilitate the extraction and analysis of document layouts.

My approach builds upon the color-based alignment strategy used by DocBank. By assigning distinct colors to different elements within the $\text{\LaTeX}$ code and then analyzing these in the generated PDF, I can accurately align and categorize content, enhancing the effectiveness of document layout analysis. This extension of the DocBank method allows for more detailed and precise handling of the alignment between the $\text{\LaTeX}$ source and the PDF output.

3 Methodology

The method discussed in this paper is extended from $\text{\LaTeX}$ Rainbow [4], a tool designed for generating Document Layout Analysis datasets. $\text{\LaTeX}$ Rainbow begins by downloading a paper’s source code from arXiv, which it then processes using a Python-based $\text{\LaTeX}$ parser. Each element within the body of the paper’s source code is assigned a unique RGB color using commands from the `xcolor` package. The tool maintains a dictionary to map these colors to specific element labels including title, author, abstract, math, table, body, caption, figure, and reference. After coloring, $\text{\LaTeX}$ Rainbow compiles the annotated $\text{\LaTeX}$ code into a PDF using a containerized compilation environment. This step ensures that the environment is consistent and reproducible, minimizing the

effects of system-specific variations on the compilation process. The final output is a color-rich PDF file, from which I can extract the color of each element. By extracting the color of each element in the PDF, I can align the label of each element in the PDF.

$\text{\LaTeX}$ Rainbow provides processing from $\text{\LaTeX}$ code annotations to PDF annotations. However, at the moment it is not directly convertible to Markup languages, for this reason, I have extended its functionality in three steps to generate accessible annotations. Figure 1 shows my annotation process.

3.1 Preprocessing $\text{\LaTeX}$ code

While arXiv ensures that the $\text{\LaTeX}$ source code it hosts is of high quality and guaranteed to compile, this does not necessarily mean the source is error-free. The $\text{\LaTeX}$ compilation process is robust enough to handle certain syntactical inaccuracies—like unclosed curly brackets—without halting the generation of a visually correct PDF. This tolerance can mask underlying issues in the source code, such as unclosed environments or improperly ordered `\end{document}` commands, which may not disrupt the PDF output but do indicate imperfect coding practices.

These minor discrepancies, though often overlooked by the $\text{\TeX}$ engine, present challenges for the more rigid Python-based `pylatexenc` parser, which requires stricter syntactic adherence. To ensure compatibility and facilitate smoother processing with such tools, it is necessary to first identify and rectify these errors. My approach involves applying a set

of rules to filter out these syntactical errors before they reach the parser, thus enhancing the reliability of tools that are less tolerant of the flexible parsing inherent to $\text{\LaTeX}$ compilers.

When dealing with $\text{\LaTeX}$ sources, custom commands often present significant parsing challenges, especially for `pylatexenc` that requires more straightforward syntactic structures. In academic papers, it is not uncommon for authors to simplify their $\text{\LaTeX}$ coding by defining macros that encapsulate common $\text{\LaTeX}$ environments. For instance,

```
\newcommand{\beq}{\begin{equation}}
\newcommand{\eeq}{\end{equation}}
```

replace standard $\text{\LaTeX}$ environment tags to streamline the writing process. While these macros are fully compatible with the $\text{\TeX}$ engine, they can complicate the parsing process for software that does not inherently interpret these user-defined shortcuts.

To address this, I use `de-macro` [7], which expands these custom commands back into their standard $\text{\LaTeX}$ forms. This tool not only handles the expansion of simple custom commands but also aids in consolidating $\text{\LaTeX}$ content spread across multiple files brought together with the `\input` command.

This preprocessing step ensures that the $\text{\LaTeX}$ code is transformed into a format that `pylatexenc` can accurately interpret, thereby maintaining the integrity of the document structure and content in environments that are less tolerant of such $\text{\LaTeX}$ customization.

3.2 Compiling $\text{\LaTeX}$ and extracting annotations

$\text{\LaTeX}$ Rainbow assigns a specific color to each element within the abstract code tree, compiles these elements into a new $\text{\LaTeX}$ source file, and then creates a color-coded PDF from this file. I have enhanced the method for extracting these annotations by employing two Python-based PDF parsers: `pdfplumber` [17] and `PyMuPDF`.¹

The reason for using two different parsers is to take advantage of their unique strengths. `pdfplumber` is effective at extracting the original colors from the PDF, which is crucial for maintaining accurate placement and alignment in the annotations. In contrast, `PyMuPDF` converts all colors to sRGB, a format that sometimes blends similar colors, which can lead to slight inaccuracies in recognizing distinct colors.

However, `PyMuPDF` excels at extracting detailed font information more precisely than `pdfplumber`. To combine the strengths of both parsers, they are used

sequentially: `pdfplumber` first identifies and extracts colors and their positions, and then `PyMuPDF` uses this positional information to accurately extract font styles, sizes, and attributes such as italic, bold, superscript, and subscript. This sequential use of both parsers ensures that we capture comprehensive details about both the colors and the textual elements of the PDF.

The annotations and output files generated by the $\text{\LaTeX}$ Rainbow framework can be merged to create what are accessibility annotations. They can be seamlessly converted into various markup languages, such as HTML or Markdown, to facilitate wider accessibility and ease of use.

Accessibility annotation aligns with PDF, $\text{\LaTeX}$, and Markup languages through element positions. This alignment ensures that the annotations are accurately reflected across different formats, enhancing the document's accessibility and maintaining consistency across various platforms.

3.3 Standardization of annotations

After I got the Markdown or HTML form of the PDF accessibility markup, I encountered an issue with some math formulas not being correctly displayed by browser-based math rendering libraries like `MathJax` [3]. This was primarily due to the persistence of custom math symbols defined using `\def` commands that were not adequately transformed into their corresponding Markdown forms, despite the earlier cleanup of the $\text{\LaTeX}$ code mentioned in Section 3.1.

To address this challenge, I utilized $\text{\LaTeX}$ MML [13], a powerful tool designed to parse and convert $\text{\LaTeX}$ code into a markup language. It ensures that the $\text{\LaTeX}$ formula code is not only transformed into a format compatible with web standards but also that it unifies the styles of mathematical symbols sourced from various $\text{\LaTeX}$ packages.

I reassembled the annotations extracted from the PDF file into a `.tex` file.

```
\documentclass{article}
\input{preamble} % read from main.tex
\begin{document}
\section{C1B5E0}
 $y = x + 1$  % the first math formula
\section{1B3810}
 $y = x + 2$  % the second math formula
...
\end{document}
```

This file loads the preamble of the paper's source, and then puts all the mathematical formulas of a paper into separate sections, with the section's title

¹ github.com/pymupdf/PyMuPDF

being its hexadecimal RGB color code in the accessibility annotation.

By using $\text{\LaTeX}$ XML to convert the assembled $\text{\LaTeX}$ code into HTML, I parsed the generated HTML document using BeautifulSoup, a robust HTML parser. This allowed me to navigate through the HTML structure efficiently and identify the sections containing mathematical formulas. The final step in my process involved replacing the original mathematical formula code within the accessibility annotations with the standardized code generated by $\text{\LaTeX}$ XML.

After the above three steps, we obtain a PDF file, an auxiliary file to record the label of each element on the PDF file, and its reading order, and an auxiliary file to record the tree structure of the document.

These auxiliary files help me to transform a PDF file, or any page, or any one of the chapters, into a markup language such as HTML or Markdown.

4 Serving as a dataset-making pipeline

According to arXiv, 90% of their submissions include $\text{\LaTeX}$ source code [2], and each submission is accompanied by a PDF. The remaining 10% are available only as PDFs and might lack accessibility tags due to the absence of $\text{\LaTeX}$ sources. Given the necessity for all scientific publications to be accessible, there's a need to derive accessibility tags directly from the PDF files. While some existing API² services offer this capability, they often produce significant mislabeling and noise.

To address these limitations, recent advancements in machine learning, particularly vision language models, have introduced document-specific solutions. An example of such innovation is the optical character recognition (OCR) model named Nougat [1], which utilizes a transformer architecture. Nougat processes screenshots of academic papers and converts them into Markdown. Importantly, it integrates mathematical formulas and tables within the page by converting them into $\text{\LaTeX}$ code in the Markdown output, showcasing a significant step forward in document processing technology. Converted Markdown also provide accessibility to PDF.

One challenge with machine learning models, including Nougat, is their lack of precision. Nougat's accuracy issues may stem from inadequately detailed training data. Its training process aligns entire PDF pages to Markdown, which is less precise than aligning based on specific page element positions. This page-based alignment and Nougat's method of pro-

cessing one word at a time can lead to errors in recognizing the location and reading order of subsequent words as the OCR task progresses.

The effectiveness of machine learning depends heavily on the quality of the training data; the model needs detailed and high-quality data to extract sufficient features that enhance its performance. My method, which generates auxiliary files to record precise element positions and accessibility markers in PDFs, could serve as an invaluable resource for creating enhanced training datasets. This could significantly improve the performance of machine learning models like Nougat by providing them with more accurate and fine-grained data on text positioning and structure. My approach could help bridge the gap between current OCR capabilities and the demands for higher accessibility and accuracy in document processing.

5 Comparison to tagpdf and SyncTeX

tagpdf [5, 15, 16] is a $\text{\LaTeX}$ package designed to facilitate the creation of PDF/UA-compliant accessible PDFs by providing core commands for tagging within $\text{\LaTeX}$. This approach allows updates directly to the $\text{\LaTeX}$ kernel, avoiding the complexities associated with external patches. Highlighted at various $\text{\TeX}$ conferences, tagpdf addresses a gap in tools for experimenting with PDF tagging and accessibility. It introduces several enhancements to the $\text{\LaTeX}$ kernel, including new PDF management features, automatic markup of paragraphs, and refined handling of page elements. tagpdf can be accessed through the “test-phase” key in the latex-lab package, allowing users to implement these features during the developmental stages of their documents.

SyncTeX [9] is a utility integrated into a $\text{\TeX}$ engine. It enhances the workflow between text editors and output viewers by providing synchronization capabilities, allowing seamless navigation between source code and the output PDF. SyncTeX generates an auxiliary file, which applications use to synchronize the text within the editor with the corresponding location in the PDF file.

My method, similar to SyncTeX, generates several auxiliary files that record coordinates corresponding to the positions of elements within a PDF. It also captures accessibility markers, including tags for elements, the reading order, and the expression of formulas. These elements are not embedded directly into the PDF, setting my approach apart from tagpdf. This strategy serves as a practical temporary solution, providing some of the functionality of tagpdf while it is still in the experimental stage.

² developer.adobe.com/document-services/apis/pdf-accessibility-auto-tag/



Figure 4: Labels of each crater dataset tile.

as a testing set while the remainder is used as a training set with the resulting F1-Score averaged together.

Our results are shown in Table 1. The scores were obtained from the respective papers with the exception of “Urbach ’09” which was obtained from [1] where it is used as a baseline comparison. Our CNN approach is

Figure 2: Example of coloring not working. The Figure has the caption label “Figure 4”, the reference number in Table “1”, and the citation “[1]”. They are all kept black because these elements are not colorable in the annotated $\text{\LaTeX}$ code.

6 Future work

6.1 Alignment without coloring

This work bridges PDF and $\text{\LaTeX}$ and Markup using color: the unique colors of each element. However, utilizing color also implies that it occupies a channel within the PDF output, leading to specific challenges.

First, there are instances where elements initially assigned a specific color by the author are overwritten in my process, resulting in the loss of original color information. Second, some elements, such as hyperlinks or caption labels in figures and tables (as depicted in Figure 2), derive their colors from package-level definitions rather than directly from the user’s $\text{\LaTeX}$ source code. For example, the `\url` command standardizes hyperlink colors across the document, which precludes assigning unique colors to individual links. Similarly, captions of figures and tables typically do not allow for unique coloring.

To address these issues, my forthcoming work will explore alternative methods beyond using color as a markup channel. Specifically, I plan to employ other embeddable features within the PDF, such as the tagging capabilities offered by the `tagpdf` package, as discussed in Section 5. By manually writing these tags, I aim to preserve the distinctiveness of document elements without overriding the original color assignments.

6.2 Parsing $\text{\LaTeX}$ code with $\text{\TeX}$ engines

Despite meticulous efforts to parse user intent in their writing, a significant portion of papers from the arXiv remains under-annotated. My assumption was that every well-structured paper would include essential elements such as a title, author information, address, section titles, and body text. However, in practice, approximately 40% of papers lack annotations for at

least one of these components. The most commonly missing annotations are those for authors, titles, and abstracts, often due to the use of customized style files that obfuscate or alter standard formatting.

In Section 3.1, I discuss how user-defined macros have been partially managed using the `de-macro` package. However, numerous style files from journals and conference proceedings introduce additional commands, complicating the parsing process for the Python-based parser `pylatexenc`. The flexibility of $\text{\LaTeX}$, attributed to its Turing-completeness [6], particularly challenges `pylatexenc` due to the prevalent use of the `\def` commands and `\if` conditions in style files, rendering the parser ineffective.

A $\text{\TeX}$ engine, which must parse the source file during compilation, provides tracing options like `\tracingmacros=1` that help humans understand how the $\text{\TeX}$ engine expands custom commands. This tracing is detailed through logs that elucidate the functioning of various packages. There are $\text{\LaTeX}$ packages available to assist users in simplifying the log to make understanding the expansion of macros easier [10, 14]. Building on this, my planned approach involves enhancing the Python parser to interpret these logs. By leveraging the $\text{\TeX}$ engine’s capabilities through log analysis, the parser is then expected to construct and interpret abstract syntax trees more accurately.

6.3 `expl3` in $\text{\LaTeX}$ XML

Section 3.3 discusses how I utilized $\text{\LaTeX}$ XML to standardize mathematical formulas and tables within the paper code. This standardization process can be notably time-consuming. If `expl3` is not included in the preamble, this conversion only takes a few seconds. However, if `expl3` is loaded in a paper’s preamble, converting the $\text{\LaTeX}$ source code into HTML using $\text{\LaTeX}$ XML can take over 20 minutes with $\text{\TeX}$ Live 2024, or 10 minutes with $\text{\TeX}$ Live 2021. The increased processing time can be attributed to the necessity for $\text{\LaTeX}$ XML to load the entire `expl3` package during each conversion, a package that has seen significant expansion in recent years due to active development. The $\text{\LaTeX}$ XML development team has acknowledged this issue and is considering solutions such as caching `expl3.sty` or rewriting $\text{\LaTeX}$ XML in Rust to improve efficiency.³

In future work, I plan to evaluate the potential impact of disabling the `expl3` package loading on the standardization process. I anticipate minimal impact, as the standardization primarily depends on packages

³ github.com/bruce miller/LaTeXML/issues/2268

related to mathematics and table formatting rather than the `expl3` package.

7 Conclusion

In this paper, I introduce a method for enhancing the accessibility of PDF files that are compiled from $\text{\LaTeX}$ sources. This approach leverages coloring techniques to generate accessibility annotations and to align content across $\text{\LaTeX}$, PDF, and various markup languages. Currently, my method applies exclusively to scientific papers available on arXiv that include $\text{\LaTeX}$ source code. However, it also serves a broader purpose by facilitating the creation of datasets. These datasets can be utilized to train machine learning models, which can improve the generation of accessibility annotations for scientific papers where only the PDF versions are available. This development holds the potential for increasing the accessibility of scientific literature.

Acknowledgement

This work was conducted within the research project InsightsNet (insightsnet.org) which is funded by the Federal Ministry of Education and Research (BMBF) under grant no. 01UG2130A.

References

- [1] L. Blecher, G. Cucurull, et al. Nougat: Neural optical understanding for academic documents, 2023. arxiv.org/abs/2308.13418.
- [2] S. Brinn, C. Cameron, et al. A framework for improving the accessibility of research papers on arxiv.org, 2024. arxiv.org/abs/2212.07286.
- [3] D. Cervone. MathJax: a platform for mathematics on the web. *Notices of the AMS*, 59(2):312–316, 2012.
- [4] C. Duan, Z. Tan, S. Bartsch. LaTeX rainbow: Universal LaTeX to PDF document semantic & layout annotation framework. In *Proceedings of the Second Workshop on Information Extraction from Scientific Publications*, T. Ghosal, F. Grezes, et al., eds., pp. 56–67, Bali, Indonesia, Nov. 2023. Association for Computational Linguistics. doi.org/10.18653/v1/2023.wiesp-1.8
- [5] U. Fischer. On the road to Tagged PDF: About StructElem, marked content, PDF/A and squeezed B ars. *TUGboat* 42(2):170–173, 2021. doi.org/10.47397/tb/42-2/tb131fischer-tagpdf
- [6] A.M. Greene. B S X: An interpreter written in T X. *TUGboat* 11(3):381–392, Sept. 1990. tug.org/TUGboat/tb11-3/tb29greene.pdf
- [7] P. G acs. de-macro — Expand private macros in a document, Dec. 2020. ctan.org/pkg/de-macro
- [8] ISO Central Secretary. Document management applications — electronic document file format enhancement for accessibility. Standard ISO 14289-2:2024, International Organization for Standardization, Geneva, CH, 2024. www.iso.org/standard/82278.html
- [9] J. Laurens. Direct and reverse synchronization with SyncT X. *TUGboat* 29(3):365–371, 2008. tug.org/TUGboat/tb29-3/tb93laurens.pdf
- [10] B. Le Floch. unravel: Watching T X digest tokens, Jan. 2024. ctan.org/pkg/unravel
- [11] M. Li, Y. Xu, et al. DocBank: A benchmark dataset for document layout analysis. In *Proceedings of the 28th International Conference on Computational Linguistics*, D. Scott, N. Bel, C. Zong, eds., pp. 949–960, Barcelona, Spain (Online), Dec. 2020. International Committee on Computational Linguistics. doi.org/10.18653/v1/2020.coling-main.82
- [12] J. MacFarlane, A. Krewinkel, J. Rosenthal. Pandoc. github.com/jgm/pandoc
- [13] B. Miller. L T XML: A L T X to XML/HTML/MathML Converter, Feb. 2024. math.nist.gov/~BMiller/LaTeXML/
- [14] F. Mittelbach. The trace package. *TUGboat* 22(1/2):93–99, Mar. 2001. tug.org/TUGboat/tb22-1-2/tb70mitt.pdf
- [15] F. Mittelbach, C. Rowley. L T X Tagged PDF — a blueprint for a large project. *TUGboat* 41(3):292–298, 2020. doi.org/10.47397/tb/41-3/tb129mitt-tagpdf
- [16] C. Rowley, U. Fischer, F. Mittelbach. Accessibility in the L T X kernel — experiments in Tagged PDF. *TUGboat* 40(2):157–158, 2019. tug.org/TUGboat/tb40-2/tb125rowley-tagpdf.pdf
- [17] J. Singer-Vine, The pdfplumber contributors. pdfplumber, July 2024. github.com/jsvine/pdfplumber

  Changxu Duan
 Technische Universit t Darmstadt
 Residenzschloss 1
 64283 Darmstadt
 Germany
[changxu.duan \(at\) tu-darmstadt dot de](mailto:changxu.duan@tu-darmstadt.de)
 ORCID 0000-0003-0547-0901



Accelerating End-to-End PDF to Markdown Conversion Through Assisted Generation

Changxu Duan^(✉) 

Technische Universität Darmstadt, Darmstadt, Germany
duan@linglit.tu-darmstadt.de

Abstract. Converting data from machine-unreadable formats like PDFs into Markdown has the potential to enhance the accessibility of scientific research. Existing end-to-end decoder transformer models can transform screenshots of PDFs into Markdown, offering more flexibility than pipeline-based methods. Yet, decoding text token by token from scratch is inefficient, especially when dense text can be directly copied from the PDF. To address this challenge, this paper modifies Prompt Lookup Decoding (PLD) to extract candidate sequences directly from PDF files, leveraging the high n-gram overlap between PDFs and their Markdown equivalents. A new method, Copy Lookup Decoding (CLD), is introduced here to enhance PLD’s candidate generation mechanism. Experiments demonstrate that CLD can accelerate the conversion process by up to $1.70 \times$ at original quality. The codebase for this paper is open-source on GitHub (<https://github.com/Fireblossom/CopyLookup>).

Keywords: PDF-to-Markdown Conversion · Assisted Generation · Document Layout Analysis

1 Introduction

Scientific research is almost exclusively published in PDF, which is an unstructured text format whose content is not readily machine-readable. PDF formatted scientific documents contain complex elements like mathematical formulas, figures, headers, and tables, as well as densely laid-out text. Vision-Language Models (VLM) can understand documents in the form of document screenshots, but their comprehension is not yet reliable enough [9].

Markdown is an ideal format in terms of machine-readability due to its simple syntax and structured format which preserves context and enhances comprehensibility. Its support for rich text elements such as boldface, italics, and hyperlinks enriches the data, providing a more detailed context for models to generate accurate outputs. Furthermore, the Markdown community has extended the syntax so that it can support tables as well as advanced mathematical and chemical formulas [23]. Therefore, converting PDF files to Markdown is worth the effort.

A widely used method for converting PDFs to Markdown utilizes a pipeline-based approach, as demonstrated by Marker [37]. This process begins with a document layout analysis that segments each PDF page into various blocks and identifies the type of each block. Marker then determines the reading order of these blocks. Based on their types, different modules are called to convert specific elements, such as formulas and tables, into Markdown. The process concludes with a cleanup phase to correct errors. However, pipeline-based approaches can struggle with documents that have complex layouts, as errors made in one step can propagate and accumulate through subsequent steps, impacting overall performance, as shown in Fig. 1.

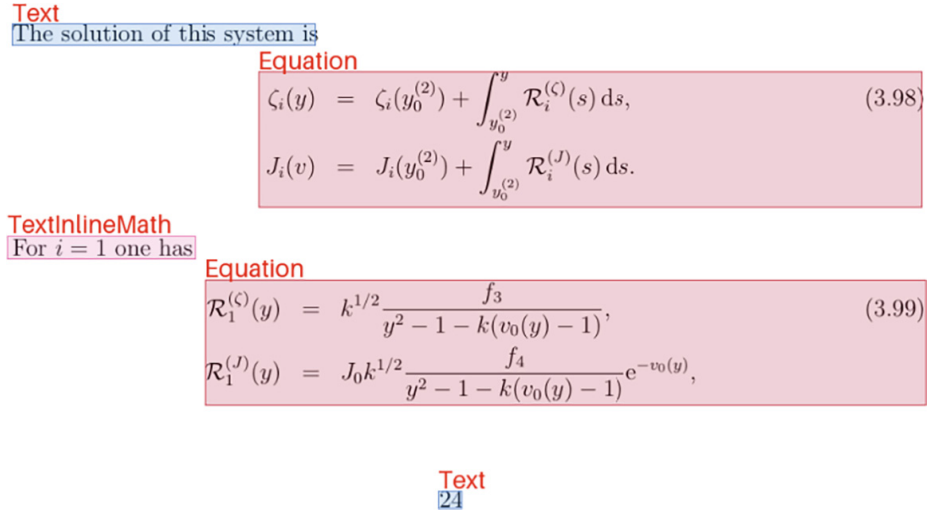


Fig. 1. Example of Marker’s layout analysis on a scientific document with a complex layout. Marker detects math equations and plain text lines correctly. However, the inline math expression and the plain text line near it were incorrectly merged; the page number was also recognized as plain text. Error results from the document layout analysis accumulate and are propagated to the next steps in the pipeline.

An alternative to pipeline-based methods is end-to-end decoder transformer models, such as Nougat [4]. The model is a tiny VLM that reads a screenshot of a PDF page and directly extracts the content into Markdown, automatically preserving the correct reading order and converting mathematical formulas into L^AT_EX code. However, Nougat faces limitations in processing speed as it generates text token-by-token from scratch, which can significantly slow down the document conversion process.

Assisted Generation [14, 16] can accelerate transformer decoder models without degrading generation quality because all candidates are validated by the transformer model. Prompt Lookup Decoding (PLD) [31] proves particularly advantageous when there is a substantial overlap of n-grams between the input (prompt) and the output of LLM. This method is up to 2.4× faster in tasks such as summarization and contextual question answering. Given that PDFs

frequently contain dense text suitable for direct copying, implementing PLD could streamline the conversion process and decrease computational expenses.

The contributions of this paper include:

1. Modifying original PLD (mPLD) to enable the extraction of candidate sequences from PDF files. mPLD is plug-and-play, requires no fine-tuning, and is compatible with any end-to-end PDF-to-Markdown conversion models.
2. Proposing Copy Lookup Decoding (CLD) to enhance candidate generation in mPLD.
3. Releasing the dataset-making scripts and the arXiv IDs of the documents used in the experiments to enhance reproducibility and observe copyright.

2 Related Work

2.1 Pipeline-Based PDF-to-Markdown Conversion

GROBID [1] is a machine learning library for extracting, parsing, and restructuring documents, including PDFs, into structured XML encoded documents, which can be converted to Markdown. However, it is not flexible because it converts formulas and tables into images, thus hampering subsequent accessibility. DocTR [25] and DocBed [46] first identify the document layout and then extract text content. Marker, MinerU [6] and Docling [36] realize PDF-to-Markdown conversion through text layout analysis, reading order detection, formula or table extraction, and finally combine them into Markdown. Some close-sourced online services such as LlamaParse¹ and Mistral OCR² also provide PDF-to-Markdown conversion, using a process similar to Marker as described in their documentation.

2.2 End-to-End PDF-to-Markdown Conversion

End-to-end PDF-to-Markdown conversion models build on top of Vision-Language Models (VLM) and Document Understanding (DU) transformer models. Donut [15] is a DU model consisting of a visual encoder and a language model decoder without obtaining texts directly from the document. Donut’s pre-training objective is to retell the text from a document screenshot. Nougat [4] follows Donut in implementing a screenshot-to-Markdown conversion of academic documents with an arXiv-sourced dataset. μ gat [30] extends Nougat’s input to handle elements that cross the single-page limit. LOCR [35] solves the problem of Nougat’s hallucination and repetition using an additional location prompt. Kosmos-2.5 [22] implements a generalized screenshot-to-Markdown conversion with a larger model size. The approaches described in this paper attempt to speed up Nougat and Kosmos-2.5.

¹ <https://cloud.llamaindex.ai/parse>.

² <https://mistral.ai/news/mistral-ocr>.

General-purpose VLMs are also used to achieve screenshot-to-Markdown conversions. LlamaOCR [24], Zerox OCR [2] and Parallex [34] all feed screenshots of PDFs to open- and closed-source VLMs and direct the models to return the desired output by passing in the appropriate prompts. In this paper we replicate their methodology and conduct experiments on acceleration with two off-shelf open-source models Llama3.2 [3] and Qwen2-VL [38] as backbones.

2.3 Document Layout Analysis

Recent Document Layout Analysis (DLA) models have become increasingly powerful thanks to the availability of large-scale document layout datasets [13, 18, 28, 45]. Computer vision models have been able to extract layouts in screenshots of documents [17, 39, 42]. Language models have also been applied to recognize layouts. LayoutLM [41] and its variant VILA [32] are transformer encoder models that analyze document layouts from the texts and their 2D coordinates. LayoutLMv2 and v3 [12, 40] additionally attach visual features to the transformer encoder.

In this work, our Copyable Text Identification module for CLD is fine-tuned from ERNIE Layout [27].

2.4 Assisted Generation

To mitigate the high inference latency stemming from autoregressive decoding in Large Language Models (LLMs), Assisted Generation [16] has emerged as a novel decoding paradigm for LLM inference. In each decoding step, this method first uses a smaller LLM drafts several future tokens efficiently, and then verifies them in parallel. PLD [31] and LLMA [43] matches the last few tokens against an earlier position in the input prompt and selects the text span as a candidate. LookaheadDecoding [10] introduces multiple special tokens at the end of the input prompt to enable parallel drafting and convert the draft into an n-gram candidate. Lookahead [44] implements a multi-branch drafting strategy. Due to the low time overhead of the drafting phase, these methods can consistently improve the inference speed of the target LLM.

Some recent studies [5, 19, 20] have used the hidden states of the LLM as inputs to their draft models. However, these methods require additional parameters and need to be trained for each backbone model. In this work, we modified PLD to demonstrate the effectiveness of our approach as PLD is the simplest to implement and has already been integrated into the huggingface library.

3 Methodology

Initially, PLD checks for a matching n-gram to the current prediction in the input (prompt) of the transformer model, copying several tokens directly from the prompt as candidates. Subsequently, the LLM would verify all the candidate tokens simultaneously, determining whether each is a plausible next token in

sequence from left to right, until encountering the first implausible candidate. The transformer model would then accept the last sequence that was deemed reasonable. Modified PLD (mPLD) involves a straightforward change: shifting the source of the candidate sequences from the prompt to all the text on the PDF page.

However, this modification introduces two significant issues. First, not all text in a PDF is readily copyable, which can limit the effectiveness of the extraction. Second, by default, mPLD extracts the candidate sequence from the first matching n-gram position on the page, posing the risk that candidates located later in the document may not be correctly matched.

To solve these problems, we designed Copy Lookup Decoding (CLD), which both filters out possible PDF noise and matches candidates to the back of the page through Copyable Text Identification and Candidate Generation.

3.1 Copyable Text Identification

Inspired by DLA-related works, we assert that whether the text is copyable or not is correlated with its layout information. Specifically, we suggest that:

1. Plain text found in paragraphs should be copyable.
2. Page elements such as mathematical formulas, tables, and figures should **not** be copyable.
3. Elements that do not convey relevant content, including headers, footers, and page numbers, should **not** be copyable.

Following VILA [32], we assume that text copyability is homogeneous at the span level. We use PyMuPDF [29] to extract span-level text and bounding boxes from the PDF. PyMuPDF generally preserves the correct reading order, except in cases of floating elements such as footnotes and captions. We fine-tune the ERNIE Layout model for token classification using LoRA [11] and an unofficial implementation [26] that supports an extended input length of 1024 tokens through RoPE [33], addressing the issue of PDF pages often exceeding 512 tokens. The classification head of ERNIE Layout was modified to classify tokens as either KEEP or DELETE, simplifying the target from the broader range of layout categories typically used in DLA models (e.g., up to 10 categories in VILA) to just two.

As we have fine-tuned ERNIE Layout for token classification, and each span may contain more than one token, a voting classifier [7] is applied to decide the prediction of the spans. We demonstrate an example of Copyable Text Identification in Fig. 2.

3.2 Candidates Generation

After identifying which spans of text in the PDF are copyable, we merge adjacent copyable spans while keeping non-adjacent copyable spans separate. All merged spans are stored in a list for subsequent processing.

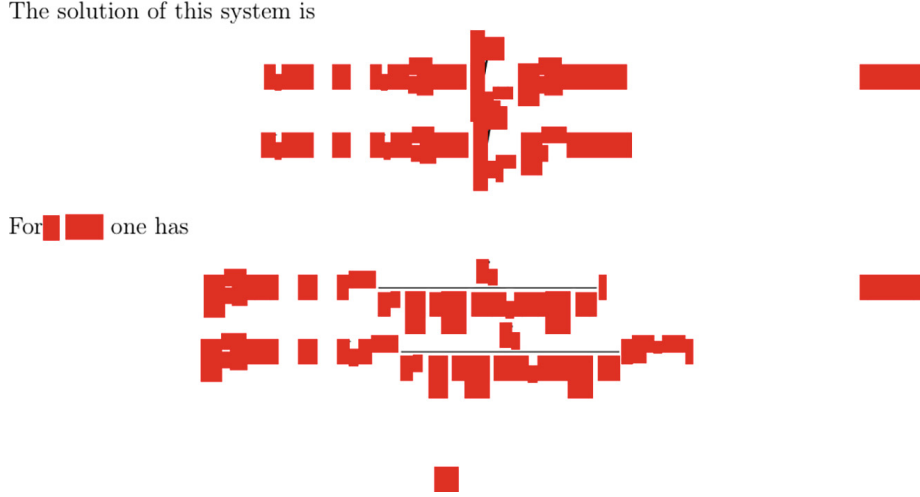


Fig. 2. Example of Copyable Text Identification. In this example, our model detects the same scientific document as in Fig. 1. The **DELETE** parts, including in-line and standalone mathematical formulas, and page numbers are identified and masked out. The **KEEP** parts including copyable texts are returned by the CLD’s candidate generator.

During candidate generation, CLD checks each span in the list, starting from the first, to find a matching n-gram. If no match is found in the first span, the search continues through subsequent spans until a match is found or all the spans are exhausted. When a matching n-gram is detected in any span, the following tokens from that span are taken as candidates.

To align with the PDF-to-Markdown conversion’s adherence to reading order, we implemented a topping mechanism. Once candidates are extracted, the PDF-to-Markdown conversion model evaluates whether these candidates are acceptable tokens in sequence. If a candidate is accepted, it indicates that the model is progressing within the span containing that candidate. At this juncture, we reassemble the span list by moving the currently active span and all subsequent spans to the front of the list while relocating all preceding spans to the end. This approach allows CLD to dynamically generate candidates that are synchronized with the ongoing progress of the PDF-to-Markdown conversion, enhancing both candidate accuracy and efficiency.

4 Experiments

4.1 PDF-to-Markdown Models

We test the conversion times and quality differences for scratch generation, mPLD, and CLD using two publicly available PDF-to-Markdown conversion specialized models, plus two general-purpose and open-sourced VLMs across the three test datasets described in Sect. 4.2. All tests were conducted on a machine with $1 \times A100$ GPU.

- **Task specialized models:** Nougat³ and Kosmos-2.5⁴
- **General-purpose VLMs:** Llama-3.2-11B-Vision-Instruct⁵ and Qwen2-VL-72B-Instruct-AWQ⁶. Qwen2-VL is labeled as a 12.6B parameter in the model card because of the quantization technique [21]. We use Prompt in Prompt 1 for these models.

```

1 Convert the following PDF page to Markdown. Return only
  the Markdown with no explanation text.
2 Leave out any page numbers and redundant headers or
  footers.
3 Do not include any code blocks (e.g. "```markdown" or
  "```") in the response. If unable to parse, return an
  empty string.

```

Prompt 1: Prompt that instructs Llama and Qwen2-VL to complete PDF-to-Markdown conversion task.

4.2 Dataset Building

As there is no existing dataset released as PDFs at this time, we downloaded the L^AT_EX source code bundles for the July and August 2023 papers from arXiv. Then we used L^AT_EX-Rainbow [8], which compiles L^AT_EX to PDF and annotates semantic labels, reading order, and L^AT_EX code

corresponding to mathematical formulas and tables for each element on a page. A total of 14,320 papers were annotated.

Spans are extracted from these pages and are labeled as either **KEEP** or **DELETE** based on the results of the annotation from the previous step. Pages that are challenging to read, such as those containing full-page images, long tables, or bibliographies, are excluded from the dataset. Finally, a dataset was assembled consisting of 180,146 pages, each annotated with span-level text copyable labels and their corresponding bounding boxes. We randomly split the training set to 162,127 pages and the test set to 18,019 pages. The vast majority of the pages in this dataset are in English.

Our method extracts text spans from PDFs, which requires access to the full text of academic papers. As arXiv does not grant permission to repost the full text⁷, we publish the scripts for creating the datasets along with the arXiv numbers of the documents to provide reproducibility.

For the PDF-to-Markdown conversion speed test, we randomly selected 128 pages from the test set. In addition, we also randomly selected 128 pages each from papers categorized in Economics and Quantum Physics respectively to test the performance of mPLD and CLD in different domain documents.

³ <https://huggingface.co/facebook/nougat-base>.

⁴ <https://huggingface.co/microsoft/kosmos-2.5>.

⁵ <https://huggingface.co/meta-llama/Llama-3.2-11B-Vision>.

⁶ <https://huggingface.co/Qwen/Qwen2-VL-72B-Instruct-AWQ>.

⁷ https://info.arxiv.org/help/license/reuse.html#full_text.

5 Results

5.1 Copyable Text Identification

We use F1 scores as the metric, without averaging because our model is a binary classifier. We tested token-level scores and span-level scores separately, and the prediction of spans was implemented by voting, as shown in Sect. 3.1.

The F1 score for token-level is **0.985**, while the F1 score for span-level is **0.988**. The average inference time per page for the classifier is **0.03** s.

5.2 PDF-To-Markdown Conversion Speed

Table 1. The speed of PDF-to-Markdown conversion models and Assisted Generation methods was evaluated across three test datasets, with $t(s)$ representing the average generation time per page in seconds. The extra 0.03s required for CLD is due to the overhead of Copyable Text Identification.

		Test Dataset					
		arXiv		Economics		Quantum Physics	
Backbone Model	Assisted Generation	$t(s)$	Speed Up	$t(s)$	Speed Up	$t(s)$	Speed Up
Nougat-base (349M)	–	6.48	–	5.73	–	6.80	–
	mPLD	5.92	1.09×	5.05	1.13×	6.13	1.11×
	CLD	5.85 (+0.03)	1.10×	4.60 (+0.03)	1.24×	5.56 (+0.03)	1.22×
Kosmos-2.5 (1.37B)	–	12.21	–	11.09	–	12.77	–
	mPLD	9.42	1.30×	8.04	1.38×	9.23	1.38×
	CLD	8.37 (+0.03)	1.45×	7.60 (+0.03)	1.45×	7.85 (+0.03)	1.62×
Llama-3.2-Vision (10.6B)	–	33.83	–	30.53	–	35.03	–
	mPLD	29.20	1.16×	23.28	1.31×	32.08	1.09×
	CLD	26.02 (+0.03)	1.30×	21.83 (+0.03)	1.40×	27.61 (+0.03)	1.27×
Qwen2-VL (12.6B)	–	138.85	–	119.42	–	156.06	–
	mPLD	97.65	1.32×	75.21	1.59×	100.00	1.56×
	CLD	91.53 (+0.03)	1.41×	72.88 (+0.03)	1.64×	91.88 (+0.03)	1.70×

The conversion speed test results are shown in Table 1. In our experiments, both mPLD and CLD speed up PDF-to-Markdown conversion. Larger models benefit more from mPLD and CLD in terms of conversion speed. The inference time of Copyable Text Identification module in CLD is included in the average conversion time per page. In Fig. 3 we show an example of how CLD can accelerate Kosmos-2.5.

one obtains from (3.75)

$$\|y_{\{0\}}^{\wedge\{2\}}\| = 1 - k z_{\{0\}}(v^{\wedge\{2\}}) = 1 - \frac{1}{2} k^{\wedge\{1/2\}} + \frac{1}{4} k \ln k^{\wedge\{-1\}} + O(k) \quad (3.94)$$

and due to (3.79)

$$\|y_{\{1\}}^{\wedge\{2\}}\| = -k z_{\{1\}}(v^{\wedge\{2\}}) = -A(\zeta_{\{0\}}) \frac{k^{\wedge\{1/2\}}}{\{2J_{\{0\}}\} \cdot \text{Big}(\{1 + O(k^{\wedge\{1/2\}})\} \cdot \text{Big}(\{1 + O(k^{\wedge\{-5/2\}})\})).} \quad (3.95)$$

Then, using the Taylor formula with respect to $\|(\mu)\|$, we shift initial conditions (3.92) to the point $\|y = y_{\{0\}}^{\wedge\{2\}}\|$ as follows:

$$\begin{aligned} \|v_{\{0\}}(y_{\{0\}}^{\wedge\{2\}})\| &= v_{\{0\}}^{\wedge\{2\}}, \quad \|v_{\{1\}}(y_{\{0\}}^{\wedge\{2\}})\| = -\frac{dv_{\{0\}}\{dy\}(y_{\{0\}}^{\wedge\{2\}})}{\|y_{\{0\}}^{\wedge\{2\}}\|} \cdot y_{\{1\}}^{\wedge\{2\}}, \\ \|zeta_{\{1\}}(y_{\{0\}}^{\wedge\{2\}})\| &= \zeta_{\{1\}}^{\wedge\{2\}}, \quad \|zeta_{\{2\}}(y_{\{0\}}^{\wedge\{2\}})\| = \zeta_{\{2\}}^{\wedge\{2\}} - \frac{dzeta_{\{1\}}\{dy\}(y_{\{0\}}^{\wedge\{2\}})}{\|y_{\{0\}}^{\wedge\{2\}}\|} \cdot y_{\{1\}}^{\wedge\{2\}}, \\ \|J_{\{0\}}(y_{\{0\}}^{\wedge\{2\}})\| &= J_{\{0\}}^{\wedge\{2\}}, \quad \|J_{\{2\}}(y_{\{0\}}^{\wedge\{2\}})\| = J_{\{2\}}^{\wedge\{2\}} - \frac{dJ_{\{1\}}\{dy\}(y_{\{0\}}^{\wedge\{2\}})}{\|y_{\{0\}}^{\wedge\{2\}}\|} \cdot y_{\{1\}}^{\wedge\{2\}}. \end{aligned} \quad (3.96)$$

Substituting (3.93) into (3.91) and expanding with respect to $\|(\mu)\|$, one obtains equations for the $\|(i)\|$ -th approximations $\|(\zeta_{\{i\}})\|$ and $\|(J_{\{i\}})\|$:

$$\|(\frac{d\zeta_{\{i\}}}{dy})\| = \mathcal{R}_{\{i\}}^{\wedge\{(\zeta)\}}(y), \quad (3.97) \quad \|(\frac{dJ_{\{i\}}}{dy})\| = \mathcal{R}_{\{i\}}^{\wedge\{(J)\}}(y).$$

The solution of this system is

$$\begin{aligned} \|\zeta_{\{i\}}(y)\| &= \zeta_{\{i\}}^{\wedge\{2\}}(y_{\{0\}}^{\wedge\{2\}}) + \int_{y_{\{0\}}^{\wedge\{2\}}}^y \mathcal{R}_{\{i\}}^{\wedge\{(\zeta)\}}(s) ds, \\ \|J_{\{i\}}(y)\| &= J_{\{i\}}^{\wedge\{2\}}(y_{\{0\}}^{\wedge\{2\}}) + \int_{y_{\{0\}}^{\wedge\{2\}}}^y \mathcal{R}_{\{i\}}^{\wedge\{(J)\}}(s) ds. \end{aligned} \quad (3.98)$$

For $\|(i=1)\|$ one has

$$\begin{aligned} \|\mathcal{R}_{\{1\}}^{\wedge\{(\zeta)\}}(y)\| &= k^{\wedge\{1/2\}} \frac{f_{\{3\}}\{y^{\wedge\{2\}} - 1 - k(v_{\{0\}}(y) - 1)\}}{\mathcal{R}_{\{1\}}^{\wedge\{(J)\}}(y)}, \\ \|\mathcal{R}_{\{1\}}^{\wedge\{(J)\}}(y)\| &= J_{\{0\}} k^{\wedge\{1/2\}} \frac{f_{\{4\}}\{y^{\wedge\{2\}} - 1 - k(v_{\{0\}}(y) - 1)\}}{\mathcal{R}_{\{1\}}^{\wedge\{(\zeta)\}}(y)}. \end{aligned} \quad (3.99)$$

Fig. 3. Example of PDF-to-Markdown conversion done by Kosmos-2.5 with the assistance of CLD. In this example, the highlighted parts are the tokens proposed by CLD and accepted by Kosmos-2.5’s validation steps.

Table 2. Ablation study results showing the impact of Copyable Text Identification (CTI) and Candidate Generation (CG) on inference time (t in seconds). Both modules contribute to reducing inference time, with their combination achieving the highest speed-up. CLD degrades to the original mPLD method when CTI and CG are removed.

Modules	Backbone Model							
	Nougat-base		Kosmos-2.5		Llama-3.2-Vision		Qwen2-VL	
	$t(s)$	Speed Up	$t(s)$	Speed Up	$t(s)$	Speed Up	$t(s)$	Speed Up
base (mPLD)	6.13	-	9.23	-	32.08	-	100.00	-
base + CTI	5.80 (+0.03)	1.05×	8.66 (+0.03)	1.06×	27.83 (+0.03)	1.15×	92.83 (+0.03)	1.07×
base + CG	5.78	1.06×	8.68	1.06×	29.30	1.09×	96.99	1.03×
base + CTI + CG	5.60 (+0.03)	1.10×	7.85 (+0.03)	1.17×	27.61 (+0.03)	1.16×	91.88 (+0.03)	1.08×

Table 3. Assisted Generation methods losslessly preserve the original output.

Assisted Generation Quality Loss		
Model	mPLD	CLD
Nougat-base	0%	0%
Kosmos-2.5	0%	0%
Llama3.2-Vision	0%	0%
Qwen2-VL	0%	0%

5.3 Ablation Study

Table 2 demonstrates the impact of different module combinations on inference time and relative speed-up for Nougat-base and Kosmos-2.5 backbone models. Additionally, ablation studies on Llama and Qwen2-VL show the same trend. These findings emphasize the practical relevance of our methods.

5.4 PDF-to-Markdown Conversion Quality

In addition to the conversion speed, we also discuss the conversion quality of the models. In this work, comparing the accuracy of individual models is highly unfeasible, mainly due to the following reasons:

- Assisted Generation is a lossless method and it doesn’t change the original output. Therefore, we may quote its accuracy in the model’s original papers.
- Neither Nougat nor Kosmos-2.5 have publicly released their test datasets, which may be due to copyright reasons described in Sect. 4.2. It is difficult for us to reproduce their results fairly.
- The output style of the PDF-to-Markdown conversion task is flexible. Specifically, the reading order of floating elements in a page, including tables and figures, is not fixed; Nougat tends to add floating elements at the end of a page, while Kosmos-2.5 tends to add floating elements at the beginning of a page.
- The formatting of tables and formulas is also flexible. Specifically, Nougat uses the $\text{\LaTeX}$ format for tables, while Kosmos-2.5 uses the Markdown format. Formulas can start and end with either a dollar sign or a backslash with parentheses.

For these reasons, we can only provide a very unfair comparison of conversion quality. Unquestionably, Assisted Generation does not degrade the quality of model generation, as shown in Table 3. We adopted the evaluation metric from the Nougat [4]. Our evaluation of the four models on our test datasets, as shown in Table 4. However, lower scores do not necessarily indicate poorer readability on a manual inspection basis, making this evaluation an imperfect quality assessment.

Table 4. PDF-to-Markdown Conversion Quality of Different Models. This is not a fair comparison, we are simply presenting the results.

Models	Edit Dist↓	BLEU↑	F1 ↑
Nougat-base	0.38	0.54	0.73
Kosmos-2.5	0.43	0.51	0.68
Llama-3.2-Vision	0.53	0.42	0.61
Qwen2-VL	0.46	0.47	0.64
Marker (Pipeline-based)	0.45	0.44	0.62

A case-by-case analysis revealed that Llama and Qwen2-VL struggled to follow instructions precisely. Llama frequently generates incorrect $\text{\LaTeX}$ code in formulas, often rendering them invalid in Markdown. Additionally, it tends to reconstruct PDF content as $\text{\LaTeX}$ rather than converting it to Markdown. Both Qwen2-VL and Llama misplace footnotes, placing them at the end of the document instead of their cited positions, increasing their error rates. We plan to refine our evaluation metrics in future work.

6 Conclusion

In this paper, we introduced mPLD and CLD, which are lightweight Assisted Generation methods for end-to-end PDF-to-Markdown conversion using decoder transformer models. mPLD does not add any parameters that require training, and it can be seamlessly added to an existing transformer decoder model. CLD is based on the off-the-shelf ERNIE Layout model. Minimal fine-tuning was performed, and the weights file size is kept at 27 MB.

The ever-increasing number of PDF publications appearing year after year shows no signs of slowing down. While pipeline-based approaches can help to get the flood of scientific information and new knowledge under control, the development of such technologies is very complex in practice and hinders the creation of infrastructures and systems to track research and assist the scientific community with applications such as dedicated scientific search engines and recommender systems.

Through our efforts, we hope to make end-to-end PDF-to-Markdown conversions with simpler structures that better drive accessibility and structured extraction of scientific information.

7 Limitations

In the work, we observed that CLD improves over mPLD mainly by extracting candidates more accurately through a filtering mechanism and a topping mechanism that better recognizes the current PDF-to-Markdown conversion progress.

However, CLD does not yet explicitly detect the position of the conversion progress of the decoder transformer on the page. The contemporaneous work, LOCR, which predicts the position of the next token on the PDF page using a miniature Prompt Module attached to Nougat, might enhance candidate generation. We plan to incorporate LOCR with CLD in the next step.

Kosmos-2.5, Nougat, Llama, and Qwen2-VL discard figures from pages, as they can only generate text. ERNIE Layout is capable of identifying them. Our goal is to explore additional ways to properly integrate figures into Markdown output. For example, Mistral OCR encodes images with base64 strings.

Acknowledgments. This work was conducted within the research project Insight-sNet (insightsnet.org), which is funded by the Federal Ministry of Education and

Research (BMBF) under grant no. 01UG2130A. We gratefully acknowledge support from Dr. Wolfgang Stille and the hessian.AI Service Center (funded by the Federal Ministry of Education and Research, BMBF, grant no. 01IS22091) and the hessian.AI Innovation Lab (funded by the Hessian Ministry for Digital Strategy and Innovation, grant no. S-DIW04/0013/003). We want to thank Dr. Sabine Bartsch for supervising this research. We also thank the anonymous reviewers for their insightful comments and suggestions, which greatly improved the manuscript.

References

1. Grobid (2008–2024). <https://github.com/kermitt2/grobid>
2. getomni ai: zerox (2024). <https://github.com/getomni-ai/zerox>. gitHub repository
3. AI, M.: The llama 3 herd of models (2024). <https://arxiv.org/abs/2407.21783>
4. Blecher, L., Cucurull, G., Scialom, T., Stojnic, R.: Nougat: neural optical understanding for academic documents (2024). <https://openreview.net/forum?id=fUtxNAKpdV>
5. Cai, T., et al.: Medusa: simple LLM inference acceleration framework with multiple decoding heads. In: Salakhutdinov, R., et al. (eds.) Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 235, pp. 5209–5235. PMLR (2024). <https://proceedings.mlr.press/v235/cai24b.html>
6. Contributors, M.: Mineru: A one-stop, open-source, high-quality data extraction tool (2024). <https://github.com/pendatalab/MinerU>
7. Diem, M., Kleber, F., Sablatnig, R.: Text classification and document layout analysis of paper fragments. In: 2011 International Conference on Document Analysis and Recognition, pp. 854–858 (2011). <https://doi.org/10.1109/ICDAR.2011.175>
8. Duan, C., Tan, Z., Bartsch, S.: LaTeX rainbow: universal LaTeX to PDF document semantic & layout annotation framework. In: Ghosal, T., Grezes, F., Allen, T., Lockhart, K., Accomazzi, A., Blanco-Cuaresma, S. (eds.) Proceedings of the Second Workshop on Information Extraction from Scientific Publications, pp. 56–67. Association for Computational Linguistics, Bali (2023). <https://doi.org/10.18653/v1/2023.wiesp-1.8>. <https://aclanthology.org/2023.wiesp-1.8/>
9. Fu, D., et al.: Isobench: benchmarking multimodal foundation models on isomorphic representations. In: First Conference on Language Modeling (2024). <https://openreview.net/forum?id=KZd1EErRJ1>
10. Fu, Y., Bailis, P., Stoica, I., Zhang, H.: Break the sequential dependency of LLM inference using lookahead decoding. In: Salakhutdinov, R., Kolter, Z., Heller, K., Weller, A., Oliver, N., Scarlett, J., Berkenkamp, F. (eds.) Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 235, pp. 14060–14079. PMLR (2024). <https://proceedings.mlr.press/v235/fu24a.html>
11. Hu, E.J., et al.: LoRA: low-rank adaptation of large language models. In: International Conference on Learning Representations (2022). <https://openreview.net/forum?id=nZeVKeeFYf9>
12. Huang, Y., Lv, T., Cui, L., Lu, Y., Wei, F.: Layoutlmv3: pre-training for document AI with unified text and image masking. In: Proceedings of the 30th ACM International Conference on Multimedia, MM ’22, pp. 4083–4091. Association for Computing Machinery, New York (2022). <https://doi.org/10.1145/3503161.3548112>

13. Jaume, G., Kemal Ekenel, H., Thiran, J.P.: Funsd: a dataset for form understanding in noisy scanned documents. In: 2019 International Conference on Document Analysis and Recognition Workshops (ICDARW), vol. 2, pp. 1–6 (2019). <https://doi.org/10.1109/ICDARW.2019.10029>
14. Gante, J.: Assisted generation: a new direction toward low-latency text generation (2023). <https://doi.org/10.57967/hf/0638>. <https://huggingface.co/blog/assisted-generation>
15. Kim, G., et al.: Ocr-free document understanding transformer. In: European Conference on Computer Vision (ECCV) (2022)
16. Leviathan, Y., Kalman, M., Matias, Y.: Fast inference from transformers via speculative decoding. In: Proceedings of the 40th International Conference on Machine Learning. ICML’23. JMLR.org (2023)
17. Li, K., et al.: Cross-domain document object detection: benchmark suite and method. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12912–12921 (2020). <https://doi.org/10.1109/CVPR42600.2020.01293>
18. Li, M., et al.: DocBank: a benchmark dataset for document layout analysis. In: Scott, D., Bel, N., Zong, C. (eds.) Proceedings of the 28th International Conference on Computational Linguistics, pp. 949–960. International Committee on Computational Linguistics, Barcelona (2020). <https://doi.org/10.18653/v1/2020.coling-main.82>
19. Li, Y., Wei, F., Zhang, C., Zhang, H.: EAGLE-2: faster inference of language models with dynamic draft trees. In: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (eds.) Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 7421–7432. Association for Computational Linguistics, Miami (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.422>. <https://aclanthology.org/2024.emnlp-main.422>
20. Li, Y., Wei, F., Zhang, C., Zhang, H.: EAGLE: speculative sampling requires rethinking feature uncertainty. In: Salakhutdinov, R., et al. (eds.) Proceedings of the 41st International Conference on Machine Learning. Proceedings of Machine Learning Research, vol. 235, pp. 28935–28948. PMLR (2024). <https://proceedings.mlr.press/v235/li24bt.html>
21. Lin, J., et al.: Awq: activation-aware weight quantization for llm compression and acceleration. In: MLSys (2024)
22. Lv, T., et al.: Kosmos-2.5: a multimodal literate model. arXiv preprint [arXiv:2309.11419](https://arxiv.org/abs/2309.11419) (2023)
23. Mathpix: mathpix-markdown-it (2024). <https://github.com/Mathpix/mathpix-markdown-it>. gitHub repository
24. Mghari, H.E.: llama-ocr (2024). <https://github.com/Nutlope/llama-ocr>. gitHub repository
25. Mindee: doctr: Document text recognition (2021). <https://github.com/mindee/doctr>
26. NormXU: Ernie-layout-pytorch (2023). <https://github.com/NormXU/ERNIE-Layout-Pytorch>. gitHub repository
27. Peng, Q., et al.: ERNIE-layout: layout knowledge enhanced pre-training for visually-rich document understanding. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2022, pp. 3744–3756. Association for Computational Linguistics, Abu Dhabi (2022). <https://doi.org/10.18653/v1/2022.findings-emnlp.274>. <https://aclanthology.org/2022.findings-emnlp.274/>

28. Pfitzmann, B., Auer, C., Dolfi, M., Nassar, A.S., Staar, P.: Doclaynet: a large human-annotated dataset for document-layout segmentation. In: Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '22, pp. 3743–3751. Association for Computing Machinery, New York (2022). <https://doi.org/10.1145/3534678.3539043>
29. pymupdf: Pymupdf (2024). <https://github.com/pymupdf/PyMuPDF>. gitHub repository
30. Quattrini, F., Zaccagnino, C., Cascianelli, S., Righi, L., Cucchiara, R.: mugat: improving single-page document parsing by providing multi-page context (2024)
31. Saxena, A.: Prompt lookup decoding (2023). <https://github.com/apoorvumang/prompt-lookup-decoding/>
32. Shen, Z., Lo, K., Wang, L.L., Kuehl, B., Weld, D.S., Downey, D.: VILA: improving structured content extraction from scientific PDFs using visual layout groups. *Trans. Assoc. Comput. Linguist.* **10**, 376–392 (2022). https://doi.org/10.1162/tacl_a-00466. <https://aclanthology.org/2022.tacl-1.22/>
33. Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., Liu, Y.: Roformer: enhanced transformer with rotary position embedding (2023). <https://arxiv.org/abs/2104.09864>
34. Summed-AI: parallex (2024). <https://github.com/Summed-AI/parallex>. gitHub repository
35. Sun, Y., Zhou, D., Lin, C., He, C., Ouyang, W., Zhong, H.S.: LOCR: location-guided transformer for optical character recognition. In: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 5480–5497. Association for Computational Linguistics, Miami (2024). <https://doi.org/10.18653/v1/2024.findings-emnlp.314>. <https://aclanthology.org/2024.findings-emnlp.314>
36. Team, D.S.: Docling technical report. Technical report (2024). <https://doi.org/10.48550/arXiv.2408.09869>
37. VikParuchuri: marker (2025). <https://github.com/VikParuchuri/marker>. gitHub repository
38. Wang, P., et al.: Qwen2-vl: enhancing vision-language model’s perception of the world at any resolution (2024). <https://arxiv.org/abs/2409.12191>
39. Wu, X., Hu, Z., Du, X., Yang, J., He, L.: Document layout analysis via dynamic residual feature fusion. In: 2021 IEEE International Conference on Multimedia and Expo (ICME), pp. 1–6 (2021). <https://doi.org/10.1109/ICME51207.2021.9428465>
40. Xu, Y., et al.: LayoutLMv2: multi-modal pre-training for visually-rich document understanding. In: Zong, C., Xia, F., Li, W., Navigli, R. (eds.) *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, vol. 1: Long Papers, pp. 2579–2591. Association for Computational Linguistics (2021). <https://doi.org/10.18653/v1/2021.acl-long.201>. <https://aclanthology.org/2021.acl-long.201/>
41. Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., Zhou, M.: Layoutlm: pre-training of text and layout for document image understanding. In: Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20, pp. 1192–1200. Association for Computing Machinery, New York (2020). <https://doi.org/10.1145/3394486.3403172>
42. Yang, H., Hsu, W.H.: Vision-based layout detection from scientific literature using recurrent convolutional neural networks. In: 2020 25th International Conference on Pattern Recognition (ICPR), pp. 6455–6462 (2021). <https://doi.org/10.1109/ICPR48806.2021.9412557>

43. Yang, N., et al.: Inference with reference: lossless acceleration of large language models (2023). <https://arxiv.org/abs/2304.04487>
44. Zhao, Y., Xie, Z., Liang, C., Zhuang, C., Gu, J.: Lookahead: an inference acceleration framework for large language model with lossless generation accuracy. In: Proceedings of the 30th ACM SIGKDD Conference on Knowledge Discovery and Data Mining, KDD '24, pp. 6344–6355. Association for Computing Machinery, New York (2024). <https://doi.org/10.1145/3637528.3671614>
45. Zhong, X., Tang, J., Jimeno Yepes, A.: Publaynet: largest dataset ever for document layout analysis. In: 2019 International Conference on Document Analysis and Recognition (ICDAR), pp. 1015–1022 (2019). <https://doi.org/10.1109/ICDAR.2019.00166>
46. Zhu, W., Sokhandan, N., Yang, G., Martin, S., Sathyanarayana, S.: Docbed: a multi-stage ocr solution for documents with complex layouts. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 36, no. 11, pp. 12643–12649 (2022). <https://doi.org/10.1609/aaai.v36i11.21539>. <https://ojs.aaai.org/index.php/AAAI/article/view/21539>



Layout-Aware Text Editing for Efficient Transformation of Academic PDFs to Markdown

Changxu Duan^(✉) 

Technische Universität Darmstadt, Darmstadt, Germany
duan@linglit.tu-darmstadt.de

Abstract. Academic documents stored in PDF format can be transformed into plain text structured markup languages to enhance accessibility and enable scalable digital library workflows. Markup languages allow for easier updates and customization, making academic content more adaptable and accessible to diverse usage, such as linguistic corpus compilation. Such documents, typically delivered in PDF format, contain complex elements including mathematical formulas, figures, headers, and tables, as well as densely layouted text. Existing end-to-end decoder transformer models can transform screenshots of documents into markup language. However, these models exhibit significant inefficiencies; their token-by-token decoding from scratch wastes a lot of inference steps in regenerating dense text that could be directly copied from PDF files. To solve this problem, we introduce EditTrans, a hybrid editing-generation model whose features allow identifying a queue of to-be-edited text from a PDF before starting to generate markup language. EditTrans contains a lightweight classifier fine-tuned from a Document Layout Analysis model on 162,127 pages of documents from arXiv. In our evaluations, EditTrans reduced the transformation latency up to 44.5% compared to end-to-end decoder transformer models, while maintaining transformation quality. Our code and reproducible dataset production scripts are open-sourced.

Keywords: PDF-to-Markdown Transformation · Document Layout Analysis · Text-Editing Model · Fill in the Middle

1 Introduction

Transforming academic documents from PDF to markup languages such as HTML or Markdown significantly enhances their accessibility and usability. This conversion not only improves web accessibility but also boosts document interactivity, enhances searchability and indexing, and guarantees compatibility across different platforms [8]. This is particularly relevant for large-scale digital library initiatives and supports FAIR principles by making content machine-actionable [43]. Such documents, typically delivered in PDF format, contain complex elements including mathematical formulas, figures, headers, and tables, as well as

densely layouted text, posing significant challenges for computational document processing [20].

Document understanding involves the automated classification and extraction of information from richly typeset documents [5]. While transformer-based models [12] and recent end-to-end decoders [4, 22, 33] can transform PDFs to Markdown, they suffer from a core inefficiency. They generate text token-by-token from scratch, leading to slow inference times and unnecessary computational overhead, especially for dense academic documents where large portions of text could be directly copied.

Despite their effectiveness, decoder-based models suffer from inefficiencies in processing speed. They generate text token-by-token from scratch, leading to slow inference times, especially for dense academic documents where large portions of text can be directly copied from PDFs. This sequential decoding approach introduces unnecessary computational overhead, making the transformation process significantly slower.

To mitigate these inefficiencies, we draw inspiration from two complementary paradigms. First, an edit-based approach, common in tasks like grammatical error correction [25], is more suitable than generating from scratch. Unlike sequence-to-sequence models, edit-based models identify and modify only necessary portions of a document, significantly improving inference speed. Second, to handle the generation of complex, non-copyable elements like formulas and tables, we leverage the Fill in the Middle (FIM) technique [3]. FIM excels at inserting content while conditioning on both preceding and following context, making it ideal for our hybrid approach.

To address the inefficiencies of existing models, we introduce EditTrans. While our approach can be seen as a workflow optimization, its research novelty lies in the effective synthesis of layout-aware classification, text-editing principles, and FIM generation to directly target the redundancy bottleneck in state-of-the-art document transformers. EditTrans features a lightweight classifier, fine-tuned from a DLA model on 162,127 pages from arXiv, to create a structured edit queue that prioritizes modifications only where necessary. In this paper, we present the following contributions:

- **EditTrans:** A Hybrid Editing-Generation Model that pre-identifies and queues editable text from PDFs before initiating markup generation. This approach significantly reduces redundant text generation and improves efficiency.
- To enhance reproducibility and transparency, we release dataset-making scripts along with the arXiv document identifiers used in our experiments. Our open-source code is available online.¹
- We refine PDF-to-Markup evaluation datasets, offering a fairer and more reliable benchmark for comparing tools aimed at improving scholarly document accessibility.

¹ <https://github.com/Fireblossom/EditTrans>.

2 Related Work

2.1 Academic Documents Transformation

GROBID [14] is a machine learning library for extracting, parsing, and restructuring documents, including PDF into structured XML encoded documents. However, it is not flexible because it converts formulas and tables into images, thus hampering subsequent accessibility. docTR [27] and DocBed [51] first identify the document layout and then extract text content. Donut [15] is a Document Understanding model consisting of a visual encoder and language model decoder without obtaining texts directly from the document. Nougat [4] follows Donut in implementing screenshot-to-Markdown transformation of Academic Documents. LOCR [39] solves the problem of Nougat’s hallucination and repetition using additional location prompts. Kosmos-2.5 [22] and DocOwl-1.5 [10] implement a more generalized screenshot-to-Markdown transformation with Vision-Language methods and larger model size. OlmOCR [33] is fine-tuned from the Qwen2-VL [42], which features text from PDF files and their corresponding coordinates as anchors into the language model’s prompt. Closed-source services such as Llama-Parse² and Mistral OCR³ also provide PDF-to-Markdown transformation. This paper describes EditTrans, which edits Nougat, Kosmos-2.5, and OlmOCR’s input sequence to speed up the transformation.

2.2 Document Layout Analysis

Recent Document Layout Analysis (DLA) models have become increasingly powerful thanks to the availability of large-scale document layout datasets [13, 20, 32, 50]. Computer Vision models have been able to extract layouts in screenshots of documents [19, 44, 47]. Language models have also been applied to recognize layouts. LayoutLM [46] and its variant VILA [35] are transformer encoder models that analyze document layouts from the texts and their 2D coordinates. LayoutLMv2 and 3 [12, 45] additionally attach visual features to the transformer encoder. DiT [18] was trained autoregression from unlabeled data. LILT [41] has migrated English layout pre-training to multiple languages. ERNIE-Layout [31] introduces the task of reading order prediction, enhancing the accuracy of the DLA models. In this work, our Editable Text Identification module is fine-tuned from ERNIE-Layout.

2.3 Text Generation with Text-Editing Models

Transformers decoder models generate outputs token-by-token from scratch, thus making them slow at inference time. Text-editing models provide several benefits over decoder models, including faster inference speed, higher sample efficiency, as well as better control and interpretability of the outputs [25]. LaserTagger [26] implements the Sentence Fusion task with three actions: KEEP, DELETE,

² <https://cloud.llamaindex.ai/parse>.

³ <https://mistral.ai/news/mistral-ocr>.

and REPLACE. FELIX [24] and Edit5 [23] also achieve text reordering. PIE [1], Seq2Edits [37], and GECToR [29] edit the text using the Iterative Refinement approach. There is a blog post⁴ about Google Search correcting user input using Edit5 [23] with low-latency features. Additionally, recent innovations like Locate&Edit [36] represent an energy-based approach to controlled text generation that can work with black-box language models by first identifying spans relevant to specific constraints and then selectively replacing them with more suitable alternatives, thereby preserving the core semantics of the original text while satisfying desired constraints. Our work organizes PDF-sourced texts into edit queues, which mimics the behavior of text-editing models.

2.4 Fill in the Middle

Fill in the Middle (FIM) [3] is a straightforward yet effective approach for training language models to fill in missing text in the middle of documents. Their extensive evaluations demonstrated that training with this transformed data does not compromise the model’s original left-to-right generative capabilities. FIM techniques have been applied to various natural language processing tasks, including Document Completion, Code Editing, and Writing Assistance. Currently, large language model services for programming assistance apply the FIM paradigm, including Gemini Coder⁵, Claude Code⁶, and DeepSeek-Coder [9]. The execution of editing actions in our task inherits the paradigm of FIM.

3 Methodology

EditTrans streamlines academic document transformation into three modules:

1. **Editable Text Identification:** EditTrans begins by classifying spans extracted from PDF pages and identifying which portions of the spans are copyable.
2. **Edit Queue Building:** EditTrans then organizes the classified spans into an edit action queue and delineates a stop criterion for each edit needed.
3. **Edit Actions Execution:** For each span requiring editing, EditTrans utilizes the pre-trained PDF-to-Markdown models to execute the necessary edits with a FIM paradigm.

Figure 1 briefly demonstrates how we can copy text from a PDF and save end-to-end PDF-to-Markdown model’s inference steps and reduce latency. Figure 4 shows an example of EditTrans processing inline formulas.

⁴ <https://research.google/blog/grammar-checking-at-google-search-scale/>.

⁵ <https://geminicoder.org/>.

⁶ <https://docs.anthropic.com/en/docs/agents-and-tools/claude-code/overview>.

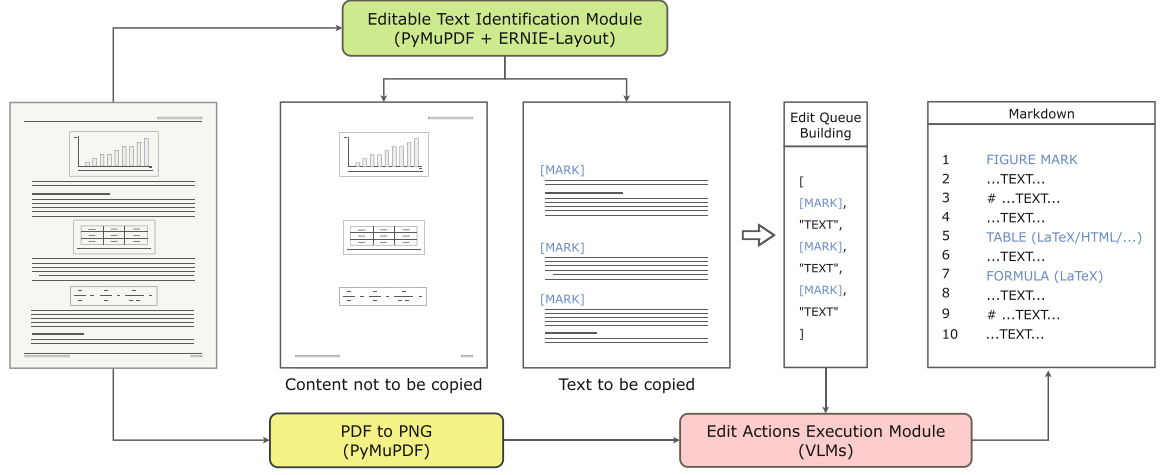


Fig. 1. The EditTrans workflow. The Editable Text Identification module detects whether the span is copyable or not. Edit Queue Building module builds an edit queue, where `[MARK]` initiates generation. Edit Actions Execution module executes edits: the blue part is generated by a backbone VLM with the FIM paradigm.

3.1 Editable Text Identification

Inspired by DLA-related work, we assume that whether the text is copyable or not is highly correlated with its layout information. Specifically, we suggest the following:

1. Dense plain text found in paragraphs should be preserved in its entirety.
2. Page elements such as mathematical formulas, tables, and titles should be modified to align with Markdown formatting standards.
3. Elements that do not convey relevant content, including page headers, footers, and page numbers, should be excluded from the final document.

Following VILA [35], we assume that text editability is homogeneous at the span level. We use PyMuPDF⁷ to extract span-level text and bounding boxes from the PDF. Subsequently, we use PyMuPDF4LLM⁸, a heuristic to detect the reading order of text spans.

We fine-tune the ERNIE layout model for token classification using LoRA [11], omitting global 1D position embeddings to prevent potential biases in layout judgment [40]. This module simplifies the target from the broader range of layout categories typically used in DLA models (e.g., up to 10 categories in DocBank [20]) to 3 categories. We altered ERNIE-Layout’s classification head to predict labels as `KEEP`, `DELETE`, or `INSERT_LEFT`.

- `KEEP` indicates that the span should be included in the Markdown output without editing;
- `DELETE` indicates that the span should be deleted;

⁷ <https://github.com/pymupdf/PyMuPDF>.

⁸ <https://pymupdf.readthedocs.io/en/latest/pymupdf4llm/>.

- `INSERT_LEFT` indicates that a trigger for PDF-to-Markdown model generation should be inserted before this span.

This approach is notably different from already existing text-editing models. Our editing logic in this paper is to delete text that should not be copied and then insert tokens that should be generated by the decoder, e.g., a mathematical formula is deleted from the PDF, and an equivalent expression is generated before the next span of editable text.

As we have fine-tuned the ERNIE layout for token classification, and each span may contain more than one token, a voting classifier [6] is applied to decide the prediction of the spans. Details of the fine-tuning hyperparameters are documented in Appendix B.

3.2 Edit Queue Building

Once we have the span-level edit annotation, we can turn it into an edit queue Q that prompts the pre-trained PDF-to-Markdown model for which portion of the text to edit. Each edit queue Q starts with an edit trigger, followed by a sequential processing of each span. We iterate through each span in the edit annotation.

If the next span is predicted to label `KEEP`, we add span text to Q . Note that if the length of the text characters in this span is very small, we do not add it and instead expect the PDF-to-Markdown model to generate it because a text makes it difficult to match where the PDF-to-Markdown model should stop generating. We then match the first words in the text of this span to sign the PDF-to-Markdown model stop generation and start copy.

If the next span is predicted as `DELETE`, we do not add anything to Q .

If the next span is `INSERT_LEFT`, we will add an edit trigger first and then add this span’s text sequence to Q . Similarly to the `KEEP` span, we will match the first words as an edit stop sign to the PDF-to-Markdown model. Note that if the length of the text characters in this span is very small (e.g. fewer than 5 characters), we do not add it and instead expect the PDF-to-Markdown model to generate it. This heuristic prevents inefficient, fragmented generation where the model would be frequently interrupted to copy very short spans.

At the end, we will add an extra edit trigger to allow the PDF-to-Markdown model to generate end-of-sentence tokens.

3.3 Edit Actions Execution

Algorithm 1 briefly demonstrates the execution of the edit action. In this step, we initialize an empty tokens sequence S and traverse the edit queue Q .

If the next element in Q is an edit trigger, a screenshot of the page and S are fed into the PDF-to-Markdown model. The PDF-to-Markdown model’s generation phase involves Fill Markdown In the Middle task, which predicts the next token in the sequence until a stopping criterion is met. The output of the PDF-to-Markdown model is added to S .

Input: Edit actions queue Q , PDF document D , Nougat model $\mathcal{M}$

Output: Generated Markdown sequence S

Initialize empty token sequence S ;

Extract screenshot of the page from D ;

```

while  $Q$  is not empty do
     $q \leftarrow \text{dequeue}(Q)$ ;
    if  $q$  is an edit trigger then
        Feed screenshot and  $S$  into  $\mathcal{M}$ ;
        while  $\mathcal{M}$  has not reached stopping criteria do
            | Generate next token and append to  $S$ ;
        end
    end
    else if  $q$  is a to-be-copied span then
        | Tokenize text from  $q$  and append to  $S$ ;
    end
end
Detokenize  $S$  into Markdown format;
return  $S$ ;

```

Algorithm 1: Edit actions execution process, the example end-to-end PDF-to-Markdown model in this pseudo code is Nougat.

If the next element in the queue is a to-be-copied span, we tokenize the text of this span and add it to the end of S .

Fill Markdown in the Middle. In this module, the pre-trained PDF model is fed with the current in-progress token sequence and begins to autoregressively decode through the decoding until it departs the stopping criteria. We set the stopping criteria to be the first n tokens of the next-to-be-copied span or end-of-sentence tokens.

Batch Synchronization. For batch inference, since the lengths of the next-to-be-copied spans in the edit queue Q are not equal, the lengths of the sequences in S are not equal after executing each edit action. We perform left padding for each sequence within the batch to ensure that the operations within the batch are synchronized.

Edit Actions Skip. In practice, we have found some deviations in the Edit Action Queue. Specifically, the recognition of Editable Text Identification (ETI) may not be reliable enough, and as a result, it may produce an incorrect edit action queue. In order to minimize the impact of ETI errors, we added a function to skip the current action in the Edit Actions Execution module. If the last $n' > n$ tokens currently generated coincide with the next-to-be-copied span in the edit action queue, we pop the next action from the edit action queue Q and proceed to execute it.

The values for n and n' (empirically set to 3 and 5, respectively) represent a trade-off. Smaller values risk premature termination of generation, while larger values might cause the model to overlook necessary small edits.

In a nutshell, we copy the simple text from the PDF and leave PDF-to-Markdown models in charge of generating the complex parts, such as formulas and tables. Finally, S is outputted and de-tokenized into the Markdown format.

EditTrans’ hyperparameters can be found in Sect. C.

4 Experiments

4.1 PDF-to-Markdown Models

There are currently three open-source weighted PDF-to-Markdown models. We used them as backbones to test the validity of EditTrans.

1. Nougat [4], we use `nougat-base`⁹ which has 349M parameters.
2. Kosmos-2.5¹⁰ [22], with 1.37B parameters.
3. OlmOCR [33], we use `olmOCR-7B-0225-preview`¹¹ with 8.29B parameters.

EditTrans is expected to produce the same output as Nougat, Kosmos-2.5, and OlmOCR (i.e., generate Markdown). EditTrans requires a PDF page as input, while Nougat and Kosmos-2.5 require a screenshot of the PDF page, and OlmOCR additionally requires text anchors in PDF files to build the prompt.

Prompt Lookup Decoding (PLD) [34] is a technique that speeds up LLM generation by identifying and reusing sequences of tokens that appear in the input prompt. Since OlmOCR uses text anchors in the file as part of the prompt, we use it as a comparison. Nougat and Kosmos-2.5 do not apply to PLD because they don’t have text in the prompt.

4.2 Datasets Building

As there is no existing dataset released as full PDFs at this time, we downloaded the L^AT_EX source code bundles for the July and August 2023 papers from arXiv. Then we use a framework [7] that compiles L^AT_EX to PDF, plus annotates for semantic labels, reading order, and L^AT_EX code corresponding to mathematical formulas and tables for each element on a page. A part of the downloaded source code of the papers was not annotated successfully because it was written in a way that the framework could not parse. A total number of 14,320 papers were annotated, yielding 180,146 pages.

Spans are extracted from these pages and are labeled as either KEEP, DELETE, or INSERT_LEFT, based on the results of the semantic annotation of the previous step. We mark the captions of figures and tables as DELETE because they are reordered.

Pages that are empty or challenging to read, such as those containing full-page images, long tables, or bibliographies, are excluded from the dataset. Finally, a dataset was assembled consisting of 180,146 pages, each annotated

⁹ <https://huggingface.co/facebook/nougat-base>.

¹⁰ <https://huggingface.co/microsoft/kosmos-2.5>.

¹¹ <https://huggingface.co/allenai/olmOCR-7B-0225-preview>.

with span-level text copyable labels and their corresponding bounding boxes. Our dataset is rich in layout diversity, including single- and multi-column formats, complex tables, numerous mathematical equations, and embedded figures, allowing robust evaluation beyond simple layouts. We randomly split the training set size to 162,127 and the test set to 18,019. The vast majority of the pages in this dataset are in English.

We then attached a Markdown target for each page, which emulates the PDF-to-Markdown model’s style of inserting mathematics formulas and tables as $\text{\LaTeX}$ code. $\text{\LaTeX}$ is quite flexible because it allows user-defined macros. Therefore, we normalize the formula and table $\text{\LaTeX}$ codes with LaTeXXML¹².

The method in this paper extracts text spans from PDFs, which requires access to the full text of academic papers. As arXiv does not grant permission to repost the full-text¹³, we publish the scripts for creating the datasets plus the dataset’s arXiv numbers to provide reproducibility.

4.3 Model-Specific PDF-to-Markdown Test Dataset Building

In previous work, since neither Nougat nor Kosmos-2.5 released their datasets, transformation quality was evaluated on their private datasets. OlmOCR, on the other hand, implements human evaluations. These are difficult to reproduce.

We randomly selected 128 pages from the test set for the PDF-to-Markdown transformation speed and quality test. In addition, we also randomly selected 128 pages each from papers categorized in Economics and Quantum Physics, respectively, to test the performance of EditTrans on different domains.

In practice, we found that the output formats of the three models are not the same, depending on their training strategies.

1. Nougat transforms all the mathematical formulas in the PDF page to $\text{\LaTeX}$ code. Nougat then takes the table body, as well as the caption of the graph and table, and places it at the end of the Markdown of the page.
2. Kosmos-2.5 converts the table in the PDF page to HTML format¹⁴, and it places the table and its caption at the beginning of the page. Kosmos-2.5 then places the caption of the figure at the tail end of the page.
3. OlmOCR will arrange tables and figures in the order in which they appear on the page and convert the tables with Markdown table syntax.

However, these differences do not lead to worse readability of the transformation results. Based on the fact that the PDF-to-Markdown transformation task is relatively flexible, we matched the behaviour of each model and assembled appropriate standard transformation results for each of them.

This provides fair and reproducible criteria for transformation quality. The scripts for making the test dataset are open-sourced on GitHub in order to provide a fair evaluation for future PDF-to-Markdown models as well.

¹² <https://math.nist.gov/~BMiller/LaTeXML/>.

¹³ https://info.arxiv.org/help/license/reuse.html#full_text.

¹⁴ This observed behavior is different from that described in their paper [22].

4.4 Evaluation Metrics

We performed timings on each module to evaluate the transformation latency reduced by EditTrans. Following Nougat, we use Edit Distance [17], BLEU [30], METEOR [2], precision, recall, and F-1 to evaluate transformation quality.

Edit Distance calculates the minimum number of character edits (insertions, deletions, substitutions) required to convert one text into another, expressed as a ratio of total characters to standardize comparisons.

BLEU is a widely-used evaluation metric in machine translation tasks, BLEU assesses textual similarity by counting overlapping sequences of words (n-grams) between the predicted and reference texts.

METEOR is a translation evaluation metric emphasizing recall. It measures semantic alignment between the predicted text and reference text, considering synonym matches and paraphrased content.

Precision, recall, and F-1 provide a balanced measure of accuracy and completeness in the generated text predictions.

5 Results

5.1 Editable Text Identification Model Accuracy

We evaluate the classifier’s performance using token-level F1 scores with micro-averaging. The achieved token-level F1 score is **0.963**. The classifier demonstrates efficient inference, requiring approximately 0.03 seconds per page.

5.2 PDF-To-Markdown Transformation Latency and Quality

Table 1 shows that EditTrans saves inference steps and thus reduces latency. On average, Kosmos-2.5 gains the greatest advantage from EditTrans, reducing generation steps by up to 43.8% on the Quantum Physics dataset and by at least 36.8% on the arXiv dataset. Conversely, Nougat benefits the least, with step reductions as low as 29.2% on the arXiv dataset. Overall, the reduction in latency closely aligns with the proportion of inference steps saved. Also, we observe that PLD reduces the latency of OlmOCR, but not as efficiently as EditTrans.

Table 2 illustrates that EditTrans generally improves or only minimally affects the quality of PDF-to-Markdown transformation. In terms of Edit Distance, EditTrans typically reduces errors across datasets; however, a minor increase of 0.003 occurs for Kosmos-2.5 on the arXiv dataset. Translation-based metrics (BLEU and METEOR) exhibit negligible differences, with variations remaining below 0.01, indicating a tiny substantial impact from EditTrans. The F1 scores mostly show improvements when EditTrans is applied, except in the cases of Nougat on the arXiv and Economics datasets with 0.005. Overall, these results demonstrate that EditTrans consistently maintains or marginally enhances transformation quality without any significant negative effects. We do not add the PLD comparison to the table as it does not change the generated texts.

Table 1. Generation steps and inference delays for three models on three test datasets. w/o ET represents models directly reading PDF files and generating their Markdown code. w/ ET represents models generated with the help of EditTrans. OlmOCR additionally compares the efficiency of PLD.

Model	Test Dataset	Metrics			
		Generation Steps	Saving Steps	Latency (s)	Reduced latency
Nougat w/o ET	arXiv	926.82	-	6.37	-
Nougat w/ ET		656.28	29.2%	4.47	29.8%
Kosmos-2.5 w/o ET		879.28	-	11.62	-
Kosmos-2.5 w/ ET		555.88	36.8%	7.24	37.7%
OlmOCR w/o ET		1148.47	-	34.47	-
OlmOCR w/ ET		766.69	33.2%	22.91	33.5%
OlmOCR w/ PLD		1148.47	-	27.21	21.1%
Nougat w/o ET	Economics	817.62	-	5.63	-
Nougat w/ ET		562.17	31.2%	3.84	31.7%
Kosmos-2.5 w/o ET		786.66	-	10.38	-
Kosmos-2.5 w/ ET		446.27	43.3%	5.81	44.0%
OlmOCR w/o ET		870.08	-	26.42	-
OlmOCR w/ ET		561.82	35.4%	17.07	35.3%
OlmOCR w/ PLD		870.08	-	21.66	21.8%
Nougat w/o ET	Quantum Physics	967.80	-	6.59	-
Nougat w/ ET		607.63	37.2%	4.12	37.4%
Kosmos-2.5 w/o ET		911.75	-	11.99	-
Kosmos-2.5 w/ ET		511.97	43.8%	6.66	44.5%
OlmOCR w/o ET		1174.38	-	35.73	-
OlmOCR w/ ET		738.45	37.1%	22.32	37.5%
OlmOCR w/ PLD		1174.38	-	28.52	20.1%

5.3 Latency of Each Module in EditTran

We analyze the execution time overhead introduced by each module as an ablation study. As illustrated in Fig. 2, integrating EditTrans incurs a slight latency increase. However, this overhead is negligible when compared to the substantial time savings achieved during the generation step. Importantly, this additional latency remains stable and does not escalate with larger backbone model sizes.

5.4 Error Analysis

Mitigating ETI Error Propagation. A core challenge is that errors from the upstream ETI module could propagate. We designed the *Edit Actions Skip* mechanism (3.3) specifically to address this. This feature acts as a self-correction step. For example, if the ETI module incorrectly labels a span for generation, but the backbone model generates the content of the subsequent KEEP span anyway, our skip mechanism detects this overlap and prevents the redundant copy action.

Table 2. PDF-to-Markdown transformation quality of the three models on the three test datasets. w/o ET represents the model directly reading and generating the screenshots of the PDF file. w/ ET represents the model generated with the help of EditTrans.

Model	Test Dataset	Metrics					
		Edit Dist ↓	BLEU ↑	METEOR ↑	Precision ↑	Recall ↑	F1 ↑
Nougat w/o ET	arXiv	0.343	0.547	0.616	0.822	0.675	0.720
Nougat w/ ET		0.335	0.542	0.619	0.801	0.676	0.715
Kosmos-2.5 w/o ET		0.384	0.506	0.590	0.724	0.634	0.656
Kosmos-2.5 w/ ET		0.387	0.516	0.600	0.737	0.644	0.666
OlmOCR w/o ET		0.311	0.541	0.676	0.671	0.679	0.667
OlmOCR w/ ET		0.294	0.532	0.683	0.698	0.699	0.688
Nougat w/o ET	Economics	0.283	0.645	0.710	0.838	0.759	0.778
Nougat w/ ET		0.278	0.625	0.701	0.819	0.746	0.762
Kosmos-2.5 w/o ET		0.320	0.590	0.699	0.751	0.738	0.726
Kosmos-2.5 w/ ET		0.312	0.604	0.706	0.768	0.742	0.739
OlmOCR w/o ET		0.281	0.594	0.742	0.707	0.737	0.712
OlmOCR w/ ET		0.277	0.584	0.734	0.725	0.760	0.732
Nougat w/o ET	Quantum Physics	0.356	0.528	0.580	0.838	0.667	0.729
Nougat w/ ET		0.349	0.550	0.601	0.863	0.688	0.752
Kosmos-2.5 w/o ET		0.384	0.528	0.600	0.778	0.660	0.696
Kosmos-2.5 w/ ET		0.374	0.545	0.616	0.794	0.671	0.713
OlmOCR w/o ET		0.294	0.573	0.699	0.715	0.707	0.704
OlmOCR w/ ET		0.257	0.571	0.712	0.735	0.712	0.708

This allows the pipeline to gracefully recover from ETI errors, enhancing its robustness.

Garbage in, Garbage out. As Fig. 3 shows, PyMuPDF4LLM, while generally reliable, can struggle with extremely complex PDF layouts. This example is a very rare edge case (<0.03% of our dataset pages), where PyMuPDF4LLM’s spatial heuristics for span extraction misinterpret the true reading order. Our current adoption of PyMuPDF4LLM was a deliberate choice, carefully balancing its generally robust accuracy and high processing efficiency across the vast majority of documents against the challenge posed by such infrequent, complex cases. Consequently, when these rare reading order errors do occur, the edit action queue generated by EditTrans becomes invalid, leading to no reduction in generation steps for those specific pages. We notice that the OlmOCR method of extracting text anchors seems to extract text in the correct reading order. We will turn to this method in future studies.

A Small Vocabulary is Better. During quality assessments, one frequent error observed with Kosmos-2.5 was the incorrect representation of $\text{\LaTeX}$ codes for inline mathematical formulas. For instance, the symbol $\pm$ should be encoded as `\pm` according to $\text{\LaTeX}$ standards, but Kosmos-2.5 instead outputs the Unicode

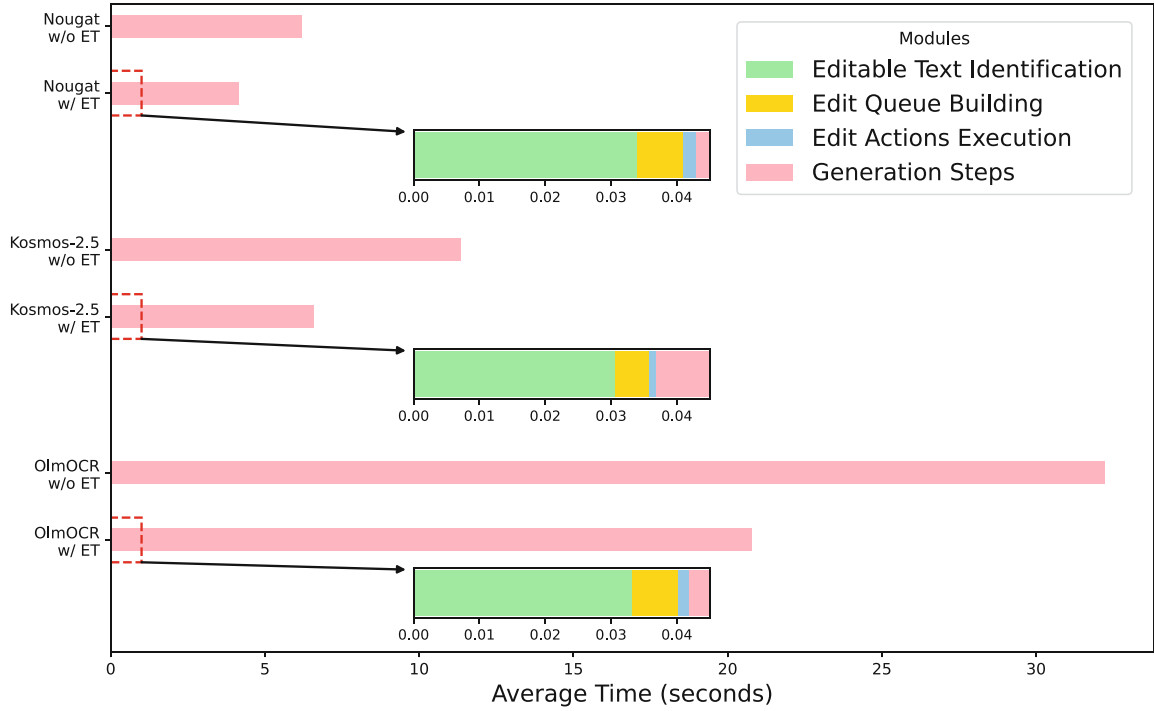


Fig. 2. Execution time overhead for each module. where ET stands for EditTrans. Generation Step in w/ ET represents the Fill Markdown In the Middle step described in Sect. 3.3, and in w/o ET represents the inference step of the model. As can be seen from the figure, the module of EditTrans adds an extra portion of latency (~ 0.04 s), but it is insignificant compared to the time it saves in the Generation Step.

character `\u00b1`. This issue likely arises because Kosmos-2.5 uses an extensive vocabulary, and its pre-training dataset may not have effectively filtered out such specific encoding errors compared to models like Nougat.

1024 Tokens are not Enough. The maximum generation length for Nougat and Kosmos-2.5 is set to 1024, while OlmOCR is set to 2048. This is due to the different maximum lengths of their models and the limitation of GPU RAM. Table 3 illustrates that the maximum sequence length of 1024 tokens is generally inadequate for fully converting a single page of scientific documents. Up to 80.5% of pages encountered truncation due to this limit. Although EditTrans occasionally allows individual outputs to slightly exceed this limit through copy-span operations, it does not fundamentally solve the truncation problem. Even OlmOCR’s increased token limit of 2048 remains insufficient in 10.2% of cases on the arXiv dataset. Consequently, sequence length limitations continue to pose significant challenges in PDF-to-Markdown transformations. Moreover, we observed that tables, followed by complex formulas, consume substantial token resources. Developing a language-model-friendly standard specifically for tables could potentially alleviate this issue.

Specialized Models are more Stable. During our quality assessment of OlmOCR’s outputs, we identified several consistency issues. (1) OlmOCR

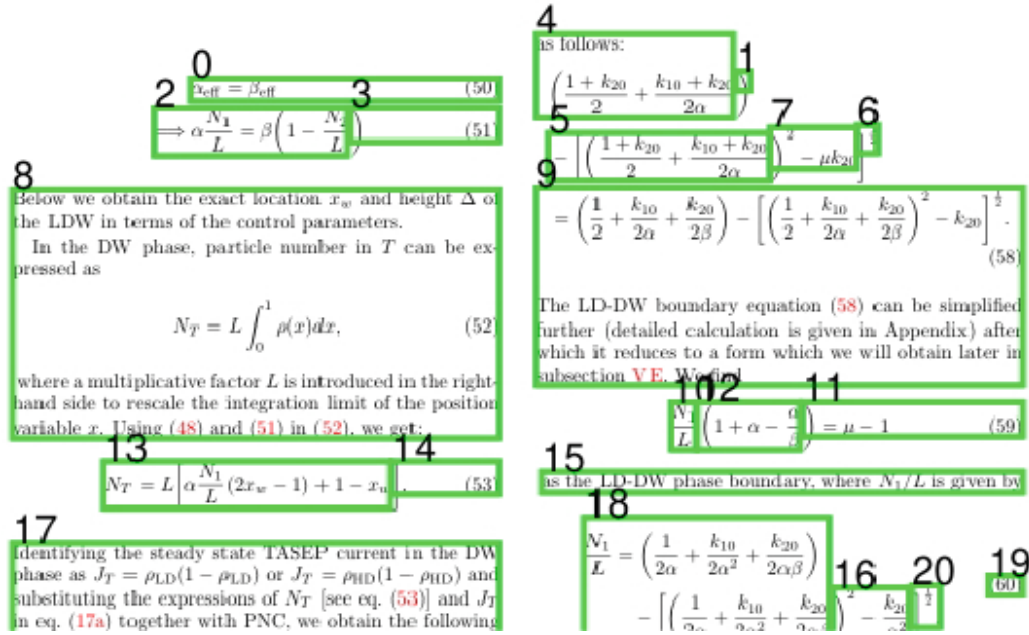


Fig. 3. Edge case example encountered during the construction of the editing action queue. The green boxes represent text spans, and the numbers indicate the reading order predicted by PyMuPDF4LLM. In this example, the text columns on the left should have been read from top to bottom, followed by the columns on the right. But their reading order is disrupted. Causing EditTrans becomes ineffective in this scenario. (Color figure online)

Table 3. Pre-trained PDF-to-Markdown models sometimes produce outputs that reach the maximum sequence length limit. When this occurs, texts at the end of the page is truncated, potentially leading to data loss and incomplete transformations.

	Test Dataset		
Model	arXiv	Economics	Quantum Physics
Nougat	67.2%	43.8%	80.5%
Kosmos-2.5	54.7%	37.5%	62.5%
OlmOCR	10.2%	1.6%	3.9%

occasionally generates redundant or unclear image captions; for example, **! [Figure 5: ...] (image)** is Markdown-compliant but contains semantically meaningless references. (2) OlmOCR inconsistently uses both $\$...\$$ and $\backslash(\dots\backslash)$ notations for inline mathematical expressions. While both formats are valid in $\text{\LaTeX}$, mixing them complicates quality control. These inconsistencies primarily result from OlmOCR’s fine-tuning based on Qwen2-VL, which is a model pre-trained on highly diverse common sense datasets, introducing unwanted variability and potentially affecting performance on specialized tasks.

6 Future Work

We observed that certain documents cannot be fully transformed by existing PDF-to-Markdown models due to issues such as hallucinations and repetitive text. These problems persist even with EditTrans, as it does not directly influence the backbone model’s generation process. LOCR [39] addresses these issues effectively by enhancing Nougat’s output using visual positional guidance, significantly reducing occurrences of hallucination and repetition. Given LOCR’s complementary nature to Nougat, it should integrate smoothly with EditTrans. We are closely monitoring LOCR’s progress and plan to combine it with EditTrans upon its official release.

Another significant challenge is that current PDF-to-Markdown models typically exclude figures from the transformation, whereas ERNIE Layout has proven capable of accurate figure extraction. We plan to explore methods for embedding figures into Markdown outputs effectively. One promising approach is inspired by Mistral OCR, which encodes images as base64 strings directly within Markdown documents.

Additionally, given the success of FIM methods in code generation, Large Language Diffusion Models [28] present an intriguing area for future research. This approach could be particularly suitable because PDF-to-Markdown tasks primarily leverage visual and textual information rather than relying solely on linguistic sequences.

User-centered assessments focusing on the readability and scholarly usefulness of the transformed documents would provide valuable insights into the practical impact of our method. Tree-based edit distance [49] is also an evaluation metric that can be extended to Markdown.

7 Conclusion

In this paper, we try to address the critical inefficiency of end-to-end models for the task of transforming academic PDFs into structured Markdown. We introduced EditTrans, a hybrid editing-generation framework that reframes the problem by decoupling simple text copying from complex, generative tasks. By leveraging a lightweight, layout-aware classifier to build an edit queue and then directing powerful, off-the-shelf models to perform targeted Fill-in-the-Middle generation, EditTrans achieves significant efficiency gains. Our comprehensive experiments demonstrated a latency reduction of up to 44.5% and a saving of over 43% in decoding steps across multiple state-of-the-art backbone models, all while maintaining or slightly improving the quality and fidelity of the final output.

The significance of this work extends beyond mere acceleration. EditTrans offers a practical pathway to scalable document processing for the digital library and archival communities. By significantly reducing the computational cost of conversion without sacrificing quality, it makes high-fidelity digitization of large academic collections a cornerstone of modern research infrastructure, both

economically and temporally feasible. This directly supports the FAIR principles by transforming static, difficult-to-parse PDFs into accessible, searchable, and machine-actionable content, thereby enhancing their value for researchers, preservationists, and the broader academic community.

While we acknowledge the limitations of our current implementation, such as its dependency on upstream reading order tools and its focus on textual content, the EditTrans framework establishes a robust and modular foundation. Its design inherently allows for future enhancements, including the integration of more advanced layout analysis and figure processing modules. Ultimately, EditTrans represents a practical and impactful step towards creating more intelligent, efficient, and scalable workflows for preserving and disseminating scholarly knowledge in the digital age.

Acknowledgments. This work was conducted within the research project InsightsNet (<https://insightsnet.org/>), which is funded by the Federal Ministry of Education and Research (BMBF) under grant no. 01UG2130A. We gratefully acknowledge support from Dr. Wolfgang Stille and the hessian.AI Service Center (funded by the Federal Ministry of Education and Research, BMBF, grant no. 01IS22091) and the hessian.AI Innovation Lab (funded by the Hessian Ministry for Digital Strategy and Innovation, grant no. S-DIW04/0013/003). We want to thank Dr. Sabine Bartsch for supervising this research. We also thank the anonymous reviewers for their insightful comments and suggestions, which greatly improved the manuscript.

A Example

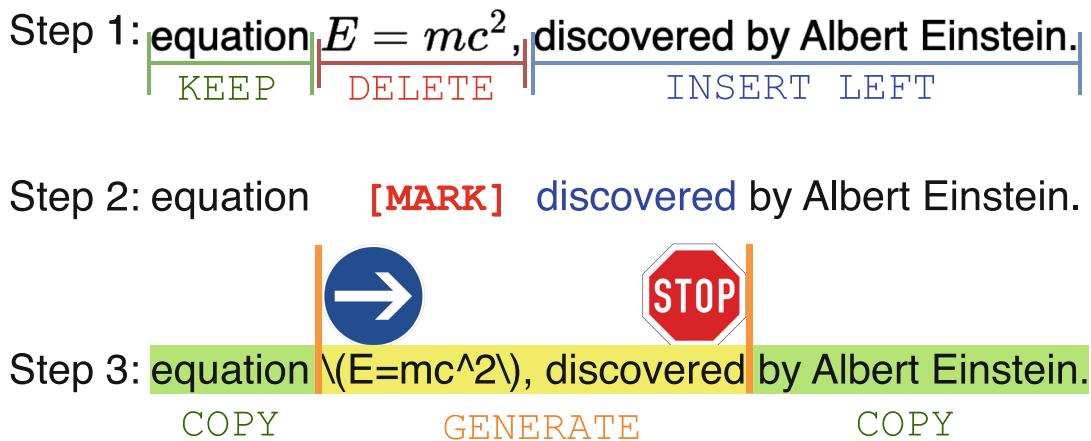


Fig. 4. Example of Edit Trans processing inline formulas. Step 1 detects whether the span is copyable or not. Step 2 builds an edit queue, where [MARK] initiates generation and a blue word serves as a stop signal. Step 3 executes edits: the green part is copied, while the yellow part is generated by a backbone model with the Fill in the Middle paradigm. (Color figure online)

B Details for Fine-Tuning ERNIE Layout

- * Base Model: ERNIE-Layout-PyTorch¹⁵
- * RoPE [38] implementation: <https://github.com/NormXU/ERNIE-Layout-Pytorch>
- * Batch Size: 48
- * Epochs: 10
- * Weight Decay: 1×10^{-5}
- * Dropout rate: 0.1
- * Optimizer: AdamW [21]
 - Learning Rate: 2×10^{-5}
 - ϵ : 1×10^{-6}
- * LoRA:
 - Rank: 256
 - α : 256
- * All Parameters: 291,131,460
- * Trainable Parameters: 9,457,154 (3.25%)
- * The ERNIE-Layout model was fine-tuned on an 1×A100 machine for 12 h.

C Implementation Details

The number of lookup tokens for Prompt Lookup Decoding is set to 3. We used the implementation of the Transformers library for our experiments because it has the simplest implementation. We plan to move to vLLM [16] and SGLang [48] for faster speeds, but this will require more compatibility changes.

D Limitations

Due to the limitations of the ERNIE layout model, our method chunks the edit sequence input to a maximum of 1024 tokens, but we have observed many pages exceeding this token count.

Full-page formulas and tables cannot benefit from our method. EditTrans may be less efficient in batch generation due to synchronization, and we are developing a more flexible approach.

References

1. Awasthi, A., Sarawagi, S., Goyal, R., Ghosh, S., Piratla, V.: Parallel iterative edit models for local sequence transduction. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 4260–4270. Association for Computational Linguistics, Hong Kong, China (2019). <https://doi.org/10.18653/v1/D19-1435>, <https://aclanthology.org/D19-1435/>

¹⁵ <https://huggingface.co/Norm/ERNIE-Layout-Pytorch>.

2. Banerjee, S., Lavie, A.: METEOR: an automatic metric for MT evaluation with improved correlation with human judgments. In: Goldstein, J., Lavie, A., Lin, C.Y., Voss, C. (eds.) *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*, pp. 65–72. Association for Computational Linguistics, Ann Arbor, Michigan (2005). <https://aclanthology.org/W05-0909/>
3. Bavarian, M., et al.: Efficient training of language models to fill in the middle (2022)
4. Blecher, L., Cucurull, G., Scialom, T., Stojnic, R.: Nougat: Neural optical understanding for academic documents (2023)
5. Cui, L., Xu, Y., Lv, T., Wei, F.: Document AI: benchmarks, models and applications (2021)
6. Diem, M., Kleber, F., Sablatnig, R.: Text classification and document layout analysis of paper fragments. In: *2011 International Conference on Document Analysis and Recognition (ICDAR)*, pp. 854–858. IEEE, Piscataway, NJ, USA (2011). <https://doi.org/10.1109/ICDAR.2011.175>
7. Duan, C., Tan, Z., Bartsch, S.: LaTeX rainbow: universal LaTeX to PDF document semantic & layout annotation framework. In: Ghosal, T., Grezes, F., Allen, T., Lockhart, K., Accomazzi, A., Blanco-Cuaresma, S. (eds.) *Proceedings of the Second Workshop on Information Extraction from Scientific Publications*, pp. 56–67. Association for Computational Linguistics, Bali, Indonesia (2023). <https://doi.org/10.18653/v1/2023.wiesp-1.8>, <https://aclanthology.org/2023.wiesp-1.8/>
8. Frankston, C., Godfrey, J., Brinn, S., Hofer, A., Nazzaro, M.: HTML papers on arXiv – Why it is important, and how we made it happen (2024)
9. Guo, D., et al.: DeepSeek-Coder: when the large language model meets programming – The rise of code intelligence (2024)
10. Hu, A., et al.: mPLUG-DocOwl 1.5: unified structure learning for OCR-free document understanding. *arXiv preprint arXiv:2403.12895* (2024)
11. Hu, E.J., et al.: LoRA: Low-rank adaptation of large language models. In: *International Conference on Learning Representations (ICLR)* (2022). <https://openreview.net/forum?id=nZeVKeeFYf9>
12. Huang, Y., Lv, T., Cui, L., Lu, Y., Wei, F.: LayoutLMv3: pre-training for document AI with unified text and image masking. In: *Proceedings of the 30th ACM International Conference on Multimedia* (2022), pp. 4083–4091. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3503161.3548112>
13. Jaume, G., Kemal Ekenel, H., Thiran, J.P.: FUNSD: a dataset for form understanding in noisy scanned documents. In: *2019 International Conference on Document Analysis and Recognition Workshops (ICDARW)*, vol. 2, pp. 1–6. IEEE, Piscataway, NJ, USA (2019). <https://doi.org/10.1109/ICDARW.2019.10029>
14. Kermitt2: GROBID (2008–2024). <https://github.com/kermitt2/grobid>
15. Kim, G., et al.: OCR-Free document understanding transformer. In: Avidan, S., Brostow, G., Cissé, M., Farinella, G.M., Hassner, T. (eds.) *Computer Vision – ECCV 2022: 17th European Conference on Computer Vision*. LNCS, vol. 13661, pp. 466–483. Springer, Cham (2022). https://doi.org/10.1007/978-3-031-19815-1_29
16. Kwon, W., et al.: Efficient memory management for large language model serving with PagedAttention. In: *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles (SOSP 2023)*. Association for Computing Machinery, New York, NY, USA (2023)
17. Levenshtein, V.I.: Binary codes capable of correcting deletions, insertions and reversals. *Soviet Phys. Doklady* **10**(8), 707–710 (1966)

18. Li, J., Xu, Y., Lv, T., Cui, L., Zhang, C., Wei, F.: DiT: self-supervised pre-training for document image transformer. In: Proceedings of the 30th ACM International Conference on Multimedia (MM 2022), pp. 3530–3539. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3503161.3547911>
19. Li, K., et al.: Cross-domain document object detection: benchmark suite and method. In: 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), pp. 12912–12921. IEEE, Piscataway, NJ, USA (2020). <https://doi.org/10.1109/CVPR42600.2020.01293>
20. Li, M., et al.: DocBank: a benchmark dataset for document layout analysis. In: Scott, D., Bel, N., Zong, C. (eds.) Proceedings of the 28th International Conference on Computational Linguistics, pp. 949–960. International Committee on Computational Linguistics, Barcelona, Spain (Online) (2020). <https://doi.org/10.18653/v1/2020.coling-main.82>, <https://aclanthology.org/2020.coling-main.82/>
21. Loshchilov, I., Hutter, F.: Decoupled weight decay regularization. In: International Conference on Learning Representations (ICLR) (2019). <https://openreview.net/forum?id=Bkg6RiCqY7>
22. Lv, T., et al.: Kosmos-2.5: a multimodal literate model. arXiv preprint [arXiv:2309.11419](https://arxiv.org/abs/2309.11419) (2023)
23. Mallinson, J., Adamek, J., Malmi, E., Severyn, A.: EdiT5: semi-autoregressive text editing with t5 warm-start. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2022, pp. 2126–2138. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (2022). <https://doi.org/10.18653/v1/2022.findings-emnlp.156>, <https://aclanthology.org/2022.findings-emnlp.156/>
24. Mallinson, J., Severyn, A., Malmi, E., Garrido, G.: FELIX: flexible text editing through tagging and insertion. In: Cohn, T., He, Y., Liu, Y. (eds.) Findings of the Association for Computational Linguistics: EMNLP 2020, pp. 1244–1255. Association for Computational Linguistics, Online (2020). <https://doi.org/10.18653/v1/2020.findings-emnlp.111>, <https://aclanthology.org/2020.findings-emnlp.111/>
25. Malmi, E., et al.: Text generation with text-editing models. In: Ballesteros, M., Tsvetkov, Y., Alm, C.O. (eds.) Proceedings of the 2022 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Tutorial Abstracts, pp. 1–7. Association for Computational Linguistics, Seattle, United States (2022). <https://doi.org/10.18653/v1/2022.naacl-tutorials.1>, <https://aclanthology.org/2022.naacl-tutorials.1/>
26. Malmi, E., Krause, S., Rothe, S., Mirylenka, D., Severyn, A.: Encode, tag, realize: high-precision text editing. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP), pp. 5054–5065. Association for Computational Linguistics, Hong Kong, China (2019). <https://doi.org/10.18653/v1/D19-1510>, <https://aclanthology.org/D19-1510/>
27. Mindee: docTR: document text recognition (2021). <https://github.com/mindee/doctr>
28. Nie, S., et al.: Large language diffusion models (2025)
29. Omelianchuk, K., Atrasevych, V., Chernodub, A., Skurzshanskyi, O.: GECToR – grammatical error correction: Tag, not rewrite. In: Burstein, J., Kochmar, E., Leacock, C., Madnani, N., Pilán, I., Yannakoudakis, H., Zesch, T. (eds.) Proceedings of the Fifteenth Workshop on Innovative Use of NLP for Building Educational Applications, pp. 163–170. Association for Computational Linguistics, Seattle, WA,

- USA → (2020). <https://doi.org/10.18653/v1/2020.bea-1.16>, <https://aclanthology.org/2020.bea-1.16/>
30. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: Bleu: a method for automatic evaluation of machine translation. In: Isabelle, P., Charniak, E., Lin, D. (eds.) *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*, pp. 311–318. Association for Computational Linguistics, Philadelphia, Pennsylvania, USA (2002). <https://doi.org/10.3115/1073083.1073135>, <https://aclanthology.org/P02-1040/>
 31. Peng, Q., et al.: ERNIE-layout: layout knowledge enhanced pre-training for visually-rich document understanding. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 3744–3756. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (2022). <https://doi.org/10.18653/v1/2022.findings-emnlp.274>, <https://aclanthology.org/2022.findings-emnlp.274/>
 32. Pfitzmann, B., Auer, C., Dolfi, M., Nassar, A.S., Staar, P.: DocLayNet: a large human-annotated dataset for document-layout segmentation. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining (KDD 2022)*, pp. 3743–3751. Association for Computing Machinery, New York, NY, USA (2022). <https://doi.org/10.1145/3534678.3539043>
 33. Poznanski, J., et al.: olmOCR: unlocking trillions of tokens in PDFs with vision language models (2025)
 34. Saxena, A.: Prompt lookup decoding (2023). <https://github.com/apoorvumang/prompt-lookup-decoding/>
 35. Shen, Z., Lo, K., Wang, L.L., Kuehl, B., Weld, D.S., Downey, D.: VILA: improving structured content extraction from scientific PDFs using visual layout groups. *Trans. Assoc. Comput. Linguist.* **10**, 376–392 (2022). https://doi.org/10.1162/tacl_a_00466, <https://aclanthology.org/2022.tacl-1.22/>
 36. Son, H.R., Lee, J.Y.: Locate&Edit: energy-based text editing for efficient, flexible, and faithful controlled text generation (2024)
 37. Stahlberg, F., Kumar, S.: Seq2Edits: sequence transduction using span-level edit operations. In: Webber, B., Cohn, T., He, Y., Liu, Y. (eds.) *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 5147–5159. Association for Computational Linguistics, Online (2020). <https://doi.org/10.18653/v1/2020.emnlp-main.418>, <https://aclanthology.org/2020.emnlp-main.418/>
 38. Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W., Liu, Y.: RoFormer: enhanced transformer with rotary position embedding. *Neurocomputing* **568** (2024). <https://doi.org/10.1016/j.neucom.2023.127063>
 39. Sun, Y., Zhou, D., Lin, C., He, C., Ouyang, W., Zhong, H.S.: LOCR: location-guided transformer for optical character recognition (2024)
 40. Tu, Y., Guo, Y., Chen, H., Tang, J.: LayoutMask: enhance text-layout interaction in multi-modal pre-training for document understanding. In: Rogers, A., Boyd-Graber, J., Okazaki, N. (eds.) *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 15200–15212. Association for Computational Linguistics, Toronto, Canada (2023). <https://doi.org/10.18653/v1/2023.acl-long.847>, <https://aclanthology.org/2023.acl-long.847/>
 41. Wang, J., Jin, L., Ding, K.: LiLT: a simple yet effective language-independent layout transformer for structured document understanding. In: Muresan, S., Nakov, P., Villavicencio, A. (eds.) *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 7747–7757.

- Association for Computational Linguistics, Dublin, Ireland (2022). <https://doi.org/10.18653/v1/2022.acl-long.534>, <https://aclanthology.org/2022.acl-long.534/>
42. Wang, P., et al.: Qwen2-VL: enhancing vision-language model’s perception of the world at any resolution (2024)
 43. Wilkinson, M.D., et al.: The fair guiding principles for scientific data management and stewardship. *Sci. Data* **3**(1) (2016). <https://doi.org/10.1038/sdata.2016.18>, <http://dx.doi.org/10.1038/sdata.2016.18>
 44. Wu, X., Hu, Z., Du, X., Yang, J., He, L.: Document layout analysis via dynamic residual feature fusion. In: 2021 IEEE International Conference on Multimedia and Expo (ICME), pp. 1–6. IEEE, Piscataway, NJ, USA (2021). <https://doi.org/10.1109/ICME51207.2021.9428465>
 45. Xu, Y., et al.: LayoutLMv2: multi-modal pre-training for visually-rich document understanding. In: Zong, C., Xia, F., Li, W., Navigli, R. (eds.) *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 2579–2591. Association for Computational Linguistics, Online (2021). <https://doi.org/10.18653/v1/2021.acl-long.201>, <https://aclanthology.org/2021.acl-long.201/>
 46. Xu, Y., Li, M., Cui, L., Huang, S., Wei, F., Zhou, M.: LayoutLM: pre-training of text and layout for document image understanding. In: *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD 2020)*, pp. 1192–1200. Association for Computing Machinery, New York, NY, USA (2020). <https://doi.org/10.1145/3394486.3403172>
 47. Yang, H., Hsu, W.H.: Vision-based layout detection from scientific literature using recurrent convolutional neural networks. In: 2020 25th International Conference on Pattern Recognition (ICPR), pp. 6455–6462. IEEE, Piscataway, NJ, USA (2021). <https://doi.org/10.1109/ICPR48806.2021.9412557>
 48. Zheng, L., et al.: SGLang: efficient execution of structured language model programs. In: *The Thirty-eighth Annual Conference on Neural Information Processing Systems* (2024). <https://openreview.net/forum?id=VqkAKQibpq>
 49. Zhong, X., ShafieiBavani, E., Jimeno Yepes, A.: Image-based table recognition: data, model, and evaluation. In: *Computer Vision – ECCV 2020: 16th European Conference, Glasgow, UK, August 23–28, 2020, Proceedings, Part XXI*, pp. 564–580. Springer-Verlag, Cham (2020). https://doi.org/10.1007/978-3-030-58589-1_34
 50. Zhong, X., Tang, J., Jimeno Yepes, A.: PubLayNet: largest dataset ever for document layout analysis. In: *2019 International Conference on Document Analysis and Recognition (ICDAR)*, pp. 1015–1022. IEEE, Piscataway, NJ, USA (2019). <https://doi.org/10.1109/ICDAR.2019.00166>
 51. Zhu, W., Sokhandan, N., Yang, G., Martin, S., Sathyanarayana, S.: DocBed: a multi-stage OCR solution for documents with complex layouts. In: *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 36, no. 11, pp. 12643–12649 (2022). <https://doi.org/10.1609/aaai.v36i11.21539>, <https://ojs.aaai.org/index.php/AAAI/article/view/21539>



Semantically Orthogonal Framework for Citation Classification: Disentangling Intent and Content

Changxu Duan¹  and Zhiyin Tan²  

¹ Technische Universität Darmstadt, Darmstadt, Germany
duan@linglit.tu-darmstadt.de

² L3S Research Center, Leibniz University Hannover, Hannover, Germany
zhiyin.tan@l3s.de

Abstract. Understanding the role of citations is essential for research assessment and citation-aware digital libraries. However, existing citation classification frameworks often conflate citation intent (why a work is cited) with cited content type (what part is cited), limiting their effectiveness in auto classification due to a dilemma between fine-grained type distinctions and practical classification reliability. We introduce **SOFT**, a **S**emantically **O**rthogonal **F**ramework with **T**wo dimensions that explicitly separates citation intent from cited content type, drawing inspiration from semantic role theory. We systematically re-annotate the ACL-ARC dataset using SOFT and release a cross-disciplinary test set sampled from ACT2. Evaluation with both zero-shot and fine-tuned Large Language Models (LLMs) demonstrates that SOFT enables higher agreement between human annotators and LLMs, and supports stronger classification performance and robust cross-domain generalization compared to ACL-ARC and SciCite annotation frameworks. These results confirm SOFT’s value as a clear, reusable annotation standard, improving clarity, consistency, and generalizability for digital libraries and scholarly communication infrastructures. All code and data are publicly available on GitHub (<https://github.com/zhiyintan/SOFT>).

Keywords: Citation Intent Classification · Annotation Framework · Cross-Domain Generalization

1 Introduction

Citations are central to scholarly communication, shaping how knowledge is credited, organized, and reused across research communities. However, traditional citation metrics, such as counts or h-index, offer only coarse measures of scientific impact, failing to capture the nuanced ways in which works are cited, interpreted, or contested. To address this gap, citation classification has

C. Duan and Z. Tan—The authors contributed equally and are listed alphabetically.

© The Author(s), under exclusive license to Springer Nature Switzerland AG 2026
W.-T. Balke et al. (Eds.): TPD 2025, LNCS 16097, pp. 183–206, 2026.
https://doi.org/10.1007/978-3-032-05409-8_12

emerged as a critical task in computational bibliometrics and information science. It moves beyond simple quantitative counts to unlock the meaning within scholarly dialogue by analyzing the textual context to understand why a work was cited. The aim is to distinguish whether a prior work is being used as a foundational method, critiqued for its findings, or merely mentioned as background. This functional understanding provides the technological backbone for a new generation of digital libraries and intelligent research assistants, enabling advanced services such as automated literature reviews [3, 15, 27, 28], the detection of scientific controversies [41], and impact-aware scientific search [35, 63].

Recent advances in Natural Language Processing (NLP) have enabled large-scale citation classification to be deployed in digital library infrastructures. Platforms such as Web of Science [13], Scite [41], and Semantic Scholar [2] incorporate functionalities aimed at detecting aspects of citation intent to improve retrieval, mapping, and assessment. By automatically labeling background, method, result, or support, these platforms enhance transparency, accelerate literature review, and surface influential or controversial contributions [17].

Despite growing adoption, current citation classification practices face persistent limitations. Existing annotation frameworks often conflate distinct citation intents and content types and lack consistent semantic definitions. These ambiguities hinder downstream applications such as citation-based retrieval [7] and impact analysis [37].

In this paper, we introduce **SOFT**: a **S**emantically **O**rthogonal **F**ramework with **T**wo dimensions for citation annotation. SOFT separates what is being cited (**Cited Content Type**) from why it is being cited (**Citation Intent**), enabling clearer definitions, improved annotation consistency, and stronger support for LLM-based classification. Unlike prior one-dimensional frameworks, SOFT is grounded in the principles of semantic roles, offering conceptual alignment with how authors structure argumentation by performing actions (intent) on objects (content).

We validate SOFT through a comprehensive re-annotation of the ACL-ARC dataset [28] and a cross-domain evaluation on ACT2 [40], covering 19 scientific fields. Results show that SOFT yields higher inter-model and human-LLM agreement, improves classification accuracy for both zero-shot and fine-tuned models, and generalizes more robustly across domains than existing frameworks. These findings position SOFT as a reusable annotation standard for supporting citation-aware digital libraries and transparent research assessment. Our contributions are as follows:

- We propose SOFT, a two-dimensional, semantically grounded citation annotation framework that explicitly distinguishes the predicative intent of a citation from its cited content type.
- We conduct a systematic re-annotation of the ACL-ARC dataset and release a cross-domain test set from ACT2 under the SOFT framework.
- We benchmark SOFT against ACL-ARC and SciCite using zero-shot and fine-tuned LLMs, demonstrating gains in agreement, learnability, and generalizability.

2 Related Work

Research on citation classification spans several dimensions, including the development of annotation frameworks, the design of automated classification models, and the evaluation of their performance across different datasets and scientific domains. Section 2.1 reviews existing citation intent classification frameworks, most of which rely on single-layer labels that conflate the predicative function of the citation with the type of content being referenced. Section 2.2 surveys computational approaches for automating citation intent classification, ranging from early feature-based models to recent prompt-driven large language models. Section 2.3 highlights the limited exploration of cross-domain applicability, particularly across different scientific disciplines, within existing framework designs. Finally, Sect. 2.4 discusses related efforts in re-annotating benchmark datasets, which provide the foundation for our re-annotation of ACL-ARC and ACT2 using a two-dimensional framework.

2.1 Citation Intent Classification Frameworks

The functional purpose behind why an author cites a particular work has been described using various terms, including *Citation Motivation* [55, 59], *Citation Purpose* [1], *Citation Type* [45], *Citation Function* [28, 60], and *Citation Intent* [14]. For conceptual clarity and consistency, we adopt **Citation Intent** throughout this paper.

Research on citation classification has evolved from bibliometric traditions to computational frameworks tailored for large-scale modeling. One of the earliest and most fine-grained typologies was proposed by [55], which defined thirteen citation roles spanning rhetorical, epistemic, and contextual functions. The Citation Function Corpus (CFC) [60] was one of the first computationally oriented citation resources. Drawing on Spiegel’s typology [55], it defined twelve function labels grouped into four superclasses (WEAKNESS, CONTRAST, POSITIVE, and NEUTRAL). The Citation Typing Ontology (CiTO) [52] introduced an ontology-based framework with over thirty citation relations, intended for semantic web interoperability. Although formal and expressive, CiTO remains relatively under-used in NLP research.

Other frameworks were designed for domain-specific annotation and model training. The six-label framework (USE, CRITICIZING, COMPARISON, SUBSTANTIATING, BASIS, and NEUTRAL) proposed by [1] emphasized purpose-driven types developed through the annotation of NLP publications. ACL-ARC [28] defined a six-type framework (BACKGROUND, COMPARISON OR CONTRAST, MOTIVATION, USES, EXTENSION, and FUTURE) oriented toward citation influence modeling. SciCite [14] condensed this six-type framework into a three-type framework (BACKGROUND, METHOD, and RESULTCOMPARISON), motivated by the observation that types such as MOTIVATION, EXTENSION, and FUTURE often function as background context, to better support machine reading and research navigation in both computer science and biomedical domains.

ACT and ACT2 [40, 46] extended citation annotation to additional scientific fields using a six-type framework similar to ACL-ARC (replacing COMPARISON OR CONTRAST with COMPARESCONTRASTS and extending it to capture not only differences but also similarities and disagreements between citations). In addition to intent, ACT and ACT2 also annotated influence using two binary labels (INFLUENTIAL and INCIDENTAL) with an added emphasis on author influence and comparability.

Together, these frameworks have shaped the landscape of citation intent classification, offering valuable insights into scholarly discourse. However, many adopt single-layer label structures that implicitly combine the author’s action, the citation’s function, with its semantic argument, the type of content being cited. While effective for some tasks, this design can limit semantic clarity and hinder flexible reuse across diverse domains. We return to these structural challenges in Sect. 3, which motivates the design of a new, semantically disentangled framework.

2.2 Citation Intent Classification (CIC) Models

Computational approaches to CIC have evolved significantly. Early models relied on hand-engineered features [36, 47], but the release of benchmark datasets like ACL-ARC and SciCite catalyzed a shift towards deep learning. Transformer-based encoders, particularly the domain-specific SciBERT [10], quickly became the standard [18, 50]. More recent advances explore diverse learning paradigms, including graph-based models [11], multi-task learning [22, 53], and prompt-based methods [30, 32]. Following the latest trend, recent work has also focused on fine-tuning LLMs for this task [29].

In our experiments, we re-annotated the ACL-ARC dataset using our proposed two-dimensional framework and used it for model training and evaluation. To assess cross-domain generalizability, we additionally re-annotated a sample from the ACT2 dataset. Results from these evaluations are presented in Sect. 5, where we compare the performance of CitePrompt and CitationIntentOpenLLM under our framework.

2.3 Cross-Domain Generalizability

A persistent challenge in CIC is the limited ability of models to generalize across different scientific domains or datasets [33, 53, 64]. To rigorously assess true generalization capabilities, models must be subjected to cross-domain testing: training on a source dataset (e.g., ACL-ARC) and evaluating on a distinct target (e.g., ACT2) [12].

Cross-domain or cross-dataset evaluation offers a principled framework for testing generalization capacity. In this setting, models are trained on one source dataset and evaluated on an independent target dataset [12]. This setup is widely used across NLP tasks to assess whether models learn domain-agnostic features or overfit to dataset-specific patterns. For instance, prior work has investigated transfer performance in question answering across biomedical, Wikipedia, and

web dataset [24], and in relation extraction across distinct subdomains such as news and politics [8]. Another example is the divide in machine translation Generalizability between resource-rich and low-resource languages [4].

This lack of robustness is particularly evident even when datasets ostensibly share the same annotation framework. For example, ACT2 adopted the ACL-ARC label set for annotations across multiple disciplines. Despite this shared taxonomy, prior work demonstrated a dramatic drop in performance when models effective on ACL-ARC were applied to ACT2 [30]. This highlights that surface-level label consistency does not guarantee semantic equivalence across domains or robust model transfer.

We interpret these findings as indicative of deeper structural limitations in existing citation classification frameworks. In particular, many current frameworks conflate citation’s function with its semantic arguments (the type of cited content), leading to annotation ambiguities that are further amplified in heterogeneous, multi-domain settings. In our experiments, we evaluate this by training models on our re-annotated ACL-ARC dataset and testing their performance on the ACT2 dataset using our revised framework annotations.

2.4 The Applicability of Re-annotating Datasets

The performance of supervised machine learning models is fundamentally tied to the quality of their training data [65]. However, real-world annotations are often noisy, with errors arising from ambiguous guidelines, subjective interpretations, limitations in annotator expertise, or simple human mistakes [25, 43]. Such errors can undermine model training, compromise evaluation reliability, and reduce the robustness of downstream applications.

Recognizing these limitations, recent work has emphasized systematic identification and correction of label errors in widely used benchmarks. For example, studies of the COCO dataset [34] have revealed deficiencies in object mask annotations, prompting initiatives such as COCO-ReM [54] and Sama-COCO [67], both of which introduced revised annotations that improved downstream model performance. Similarly, the CleanCoNLL [51] project conducted a comprehensive relabeling of the CoNLL-03 Named Entity Recognition dataset [61], correcting 7% of entity labels and demonstrating measurable gains in benchmark reliability.

In parallel, automated and semi-automated approaches have been developed to assist error detection and correction. Model-driven methods such as Confident Learning estimate the joint distribution of observed and true labels to identify likely misannotations [42], while domain-specific tools like ObjectLab apply trained object detectors to flag anomalies in annotation quality [62]. Recently, LLMs have also been explored as annotation agents [23, 56], although careful quality control remains essential in such settings.

In this work, we conduct a manual re-annotation of the ACL-ARC dataset [28] using our proposed two-dimensional framework. For comparison and internal validation, we also task a set of open-source LLMs with performing the same re-annotation under controlled prompting. To evaluate label quality and framework clarity, we compute agreement between human annotators and

LLMs. These experiments allow us to assess the reliability of our framework and explore the potential role of LLMs in future annotation workflows.

3 SOFT: A Semantically Orthogonal Framework with Two Dimensions

To address the growing need for semantically robust, interpretable, and transferable citation annotations, we propose a new framework grounded in discourse intent and cited content structure. Before introducing its design, we first examine the limitations of existing classification frameworks that motivate our departure from conventional approaches. These limitations arise from structural entanglement, under-specified functional roles, and ambiguous referential targets, all of which hinder consistent annotation and downstream model performance.

3.1 Limitations of Existing Citation Classification Frameworks

Despite the progress enabled by widely used frameworks such as CiTO [52], ACL-ARC [28], SciCite [14], and ACT/ACT2 [40, 45], several structural limitations persist. These challenges reduce annotation clarity and consistency, hinder model generalization, and complicate the semantic interpretation of citation acts. We identify three interrelated problems:

(1) Dimensional Entanglement: Conflating Cited Content Type and Citation Intent. Many previous citation classification frameworks failed to separate the cited content type, such as method, data, or finding, from citation intent or function, such as use, and instead combined both dimensions into a single label. For example, in CiTO, types like “uses method in”, “uses data from”, and “uses conclusions from” explicitly bind the act of using with the specific object being used, resulting in a conflation of content type and citation intent. This design leads to an explosion in the number of types while still missing important boundary cases, which increases the cognitive burden for annotators and makes modeling more challenging due to both type proliferation and unclassifiable instances. SciCite, in contrast, presents a related limitation differently: its type BACKGROUND information is broadly defined and absorbs all citation acts that do not fit into the method or result types. As a result, many citations with specific functional intent are ultimately subsumed under the background type, which obscures their actual scholarly role.

(2) Perspective Ambiguity: Authorial Commitment and Discourse Voice. Additionally, inconsistent perspective across types introduces further confusion for annotators. For example, in the ACL-ARC framework, types such as BACKGROUND, MOTIVATION, and FUTURE use the cited work as the subject of the action, while types such as USES, EXTENSION, and COMPARISON OR CONTRAST use the citing work as the acting subject. This shift in perspective causes cognitive inconsistency and makes the annotation process more confusing.

(3) Lack of a Functionally Grounded Intention Framework. Moreover, the underlying citation intent is not always clearly specified. For example, a citation marked as `COMPARISON` OR `CONTRAST` might align findings, methods, or conceptual framing, but whether the comparison serves to justify a design, critique a precedent, or merely provide context depends entirely on the citing author’s functional goal. Existing frameworks offer no mechanism for disambiguating these divergent functions, leaving the predicative action of the citation act ambiguous. Without an explicit model separating the author’s action from the object of that action, annotation relies on inference and subjective interpretation, which reduces reproducibility and cross-annotator agreement.

3.2 The SOFT Citation Annotation Framework

We propose **SOFT**, a two-dimensional citation annotation framework designed to overcome core limitations of previous citation classification systems. SOFT directly addresses three major problems: (1) entanglement of content type and citation function, (2) inconsistent annotation perspectives, and (3) ambiguous or underspecified citation intent.

First, SOFT explicitly disentangles **Cited Content Type** (what specific content is referenced from the cited work) from **Citation Intent** (why the citing work refers to that content), thus resolving the long-standing problem of dimensional entanglement, ensuring each dimension is orthogonal and semantically bounded, greatly reducing ambiguity and cognitive load.

Second, SOFT maintains a unified authorial perspective by consistently modeling the action of the citing work. This approach is grounded in semantic role theory [6, 20, 21, 44], which decomposes events into predicates (actions) and arguments (participants or entities involved). In SOFT, citation intent always corresponds to the action or communicative function performed by the citing work, such as “use X from [cited work]”, “modify X from [cited work]”, or “evaluate against X from [cited work]”, where X is the specific cited content. These intents, like “use” or “modify”, align with classic event predicates in semantic role theory. Other intents in SOFT, such as “contextualize using X from [cited work]”, “signal a gap using X from [cited work]”, “highlight a limitation in X from [cited work]”, or “justify a design choice based on X from [cited work]”, capture broader discourse actions in scientific writing. While these latter cases extend beyond simple event predicates, all SOFT intent types are defined to specify what the citing work is doing, ensuring each annotation remains grounded in a clear and consistent authorial perspective.

Third, SOFT provides explicit, semantically precise definitions for all citation intent types, ensuring the underlying authorial purpose is clearly specified in every case. For instance, we rephrase `BACKGROUND` (used in ACL-ARC, Sci-Cite, and ACT/ACT2) as `CONTEXTUALIZE` to mark deliberate authorial moves, and rephrase `EXTENSION` as `MODIFY` to capture not only extension but also reduction, replacement, or novel combination. We replace the citation function `MOTIVATION` (present in ACL-ARC and ACT/ACT2) with `SIGNALGAP` and

HIGHLIGHTLIMITATION, which allow for a fine-grained distinction between identifying a research gap and highlighting a limitation. To further reduce ambiguity, we eliminate the type **COMPARISON** OR **CONTRAST** (in ACL-ARC), which is often confused in practice, and instead introduce **JUSTIFYDESIGNCHOICE** for decision justification and **EVALUATEAGAINST** for direct empirical comparison.

The **SOFT** type inventory was empirically derived and iteratively refined using diverse datasets and existing frameworks (including SciCite, ACL-ARC, and CiTO), systematically resolving label ambiguity and mixed semantic scope. As a result, **SOFT** yields a more interpretable and robust annotation framework, improving both human agreement and model generalizability for tasks such as discourse analysis and citation influence detection. Below, we detail the two core dimensions of the framework.

Dimension 1: Cited Content Type captures the ontological status of the contribution being referenced in the cited work. We distinguish three types:

(1) **PERFORMEDWORK**: The citing work references what the cited work *did* (e.g., experimental process, pipeline design), without isolating specific outcomes or reusable resources. *Example: “[cited work] developed an NLP pipeline.”*

(2) **DISCOVERY**: The citing work references observations, findings, or theoretical conclusions made by the cited work. *Example: “[cited work] observed that dropout improves stability.”*

(3) **PRODUCEDRESOURCE**: The citing work references reusable outputs such as datasets, algorithms, models, tools, metrics, standardized settings, etc. *Example: “We use the parser from [cited work].”*

These distinctions support fine-grained analysis of how different forms of scientific contribution are referenced, reused, or framed in scholarly discourse.

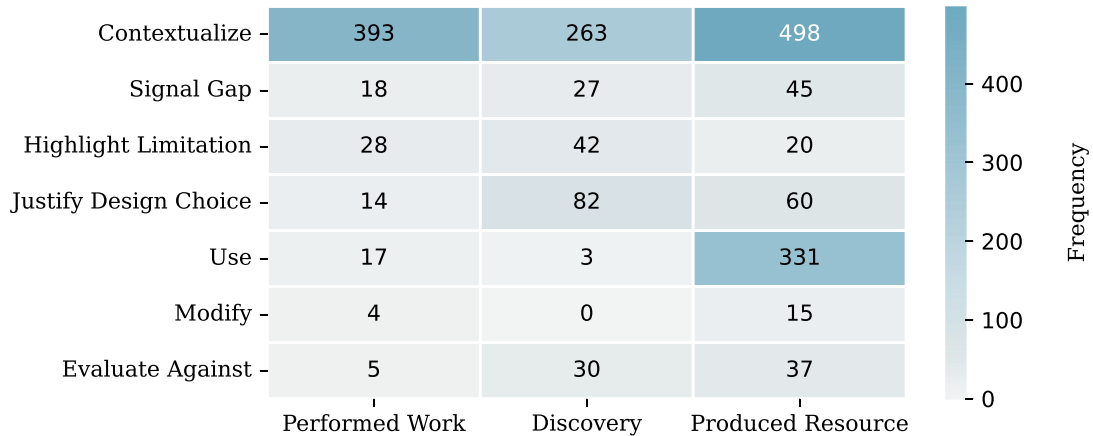


Fig. 1. Type Statistics for the re-annotated ACL-ARC Dataset.

Dimension 2: Citation Intent characterizes the predicative function of the citation, representing the action performed by the citing author. Each label is

defined as a predicate that takes the cited content as one of its core arguments. We frame each label from the perspective of the citing author as the agent of the citation action.

(1) **CONTEXTUALIZE**: The cited work is mentioned to provide background, illustrate prior research, or describe related contributions. No design decision or reuse is involved. *Example*: “[cited work] introduced hierarchical attention for sentiment analysis.”

(2) **SIGNALGAP**: The citation highlights an unresolved problem or open question. The gap may be identified by the cited work or the citing work, and no commitment to solving it is required. *Example*: “[cited work] noted that tuning remains underexplored.”

(3) **HIGHLIGHTLIMITATION**: The citing work or cited work identifies a flaw, constraint, or drawback in the cited method or result. This type is used only when the cited contribution is explicitly critiqued. *Example*: “[cited work] fails to capture domain drift in user behavior.”

(4) **JUSTIFYDESIGNCHOICE**: The citing work supports a design or methodological decision by referencing the cited work. No direct reuse is required, but the author must commit to a choice. *Example*: “We follow [cited work]’s feature grouping to ensure consistency.” This type does not apply if the action is merely hypothetical (e.g., “we could follow ...”).

(5) **USE**: The citing work directly applies a reusable contribution (e.g., model, process, settings, definitions) from the cited work. The citing author must be the actor, meaning statements like “[cited work] used...” do not qualify. Only past or present-tense application counts; plans or hypothetical uses are not ‘Use’. *Example*: “We use [cited work]’s BERT-based classifier.”

(6) **MODIFY**: The citing work alters or extends a reusable contribution from the cited work, such as by adapting configurations, changing algorithms, or integrating with a new pipeline. *Example*: “We adapt [cited work]’s encoder for multi-domain inputs.”

(7) **EVALUATEAGAINST**: The citing work explicitly compares its own findings and results with those of the cited work, typically to establish effectiveness. *Example*: “Our model outperforms [cited work]’s method on F_1 score.”

These citation intent types are designed to be mutually exclusive, which can be combined with content types for multi-dimensional interpretation.

Design Rationale and Applications. By structurally separating citation intent from cited content type, SOFT resolves ambiguities common in one-dimensional frameworks. For example, citing a model without applying it is annotated as **CONTEXTUALIZE:PRODUCEDRESOURCE**, while referring to an unresolved issue is **SIGNALGAP:DISCOVERY**. Only when the citing author explicitly acts, e.g., reuses, modifies, justifies, or critiques, is an active intent assigned. This orthogonality supports consistent labeling across citation contexts and

improves annotation reliability. SOFT’s two-dimensional design also enhances downstream applications. Separating gap signaling from design justification enables clearer modeling of research motivation. Distinguishing between using a tool (USE:PRODUCEDRESOURCE) and referencing a methodological trend (CONTEXTUALIZE:PERFORMEDWORK) supports finer-grained tracking of scientific influence, method lineage, and intent-aware scientific recommendation. As demonstrated in Sect. 5, the SOFT framework yields higher human–LLM agreement, better classification performance, and improved generalizability across scientific domains. We next describe our annotation guidelines and procedures, including the re-annotation of ACL-ARC and a cross-domain sample from ACT2.

4 Experiments

4.1 Datasets and Re-annotation Procedure

Datasets Overview. The ACL-ARC dataset [28] is a standard CIC benchmark, yet its annotation framework exhibits the conceptual conflation and boundary issues discussed in Sect. 3.1. To resolve these shortcomings, we re-annotate the entire dataset using our two-dimensional SOFT framework, thereby providing a refined and semantically coherent resource.

For our experiments, we adopt the context-expanded version of ACL-ARC provided by [39], which enhances the original dataset by adding broader citation contexts and structured metadata, such as the citing and cited paper titles and the section in which the citation appears. Before re-annotation, we manually cleaned the dataset to correct systemic text extraction errors, such as misaligned page breaks and disrupted reading orders, and incorrectly sequenced footnotes. Additionally, we recovered 533 missing section headings from the original paper PDFs to improve contextual understanding. The ACL-ARC dataset contains 1,931 citation instances. Following the original data split in [28], we adopt the same partitioning: 1,647 instances are used for training and 284 for testing.

To evaluate cross-domain generalizability, we also re-annotated a subset (264 examples) of the ACT2 test set [40]. ACT2 spans 19 top-level scientific domains from Microsoft Academic Graph (MAG), with dominant fields including Psychology (21.99%), Medicine (13.48%), Biology (10.91%), and Computer Science (10.27%). These examples serve as our out-of-distribution test data, enabling strict evaluation of generalization performance. The division of training and test sets ensures that citation instances from the same source document do not appear in both sets, preventing data leakage and preserving evaluation integrity.

Re-Annotation Procedure. The re-annotation process was conducted by a team of graduate students in computational linguistics. To ensure reliability and account for interpretive ambiguity, each citation instance was independently labeled by at least three annotators using our revised two-dimensional framework. Disagreements were flagged and subsequently resolved through structured group discussion, in which annotators reviewed framework definitions and deliberated to reach a consensus type for each case. This multi-pass strategy helped

enforce type consistency and refine interpretation standards across annotators. Types of statistics for the re-annotated ACL-ARC are reported in Fig. 1.

4.2 Evaluating Framework Interpretability and Annotation Reliability

A core indicator of annotation framework quality is Inter-Annotator Agreement (IAA), reflecting how clearly and consistently its types are defined [5, 9]. However, IAA was not reported in the original ACL-ARC release [28], leaving the interpretability of its framework unquantified. In this study, we also do not report human IAA due to the dynamic annotation process: some annotators departed and new ones joined partway through the task, so not all instances were annotated by the same set of individuals, and the number of annotations per instance varied. This evolving annotator pool resembles the crowdsourcing approach used for the SciCite dataset [14], where calculating a dataset-wide human IAA score also presents challenges. Instead, we evaluate framework interpretability and reliability through LLM-based agreement studies.

To assess the clarity and reliability of our framework, we conducted an agreement experiment using open-source LLMs as proxies for human annotators, offering a scalable and replicable assessment of annotation feasibility. Specifically, we deployed four distinct LLMs: *Qwen-2.5-72B-Instruct* (Qwen) [48], *Mistral-Small-24B-Instruct-2501* (Mistral) [38], *Llama-3.3-70B-Instruct* (Llama) [58], and *Gemma-3-27B-it* (Gemma) [57], to independently annotate the ACL-ARC citation contexts according to our framework and guidelines. For each context, every LLM generated a rationale and a classification decision, repeated three times, with the final type determined by majority vote. We then computed pairwise Cohen’s κ scores [16] under two settings: (1) between model pairs (LLM-LLM IAA), and (2) between each model and the human consensus annotations described in Sect. 4.1 (Human-LLM IAA). The latter measures how well the framework enables LLMs to approximate expert human judgments.

As a comparative baseline, we repeated the process with both the six-type ACL-ARC framework and the three-type SciCite framework [14], instructing the same LLMs to annotate ACL-ARC contexts. For ACL-ARC framework, annotations were made on its original dataset. For SciCite, we mapped ACL-ARC types onto SciCite’s framework by aligning USES with METHOD, COMPARISONCONTRAST with RESULTCOMPARISON, and grouping BACKGROUND, EXTENSION, MOTIVATION, and FUTURE under BACKGROUND.

This LLM-based simulation serves as a practical proxy for human agreement and enables direct empirical comparison across frameworks. The resulting IAA scores provide a quantifiable measure of boundary clarity, interpretability, and annotation feasibility under the SOFT framework.

4.3 Testing Orthogonality and Interpretive Clarity via Classification

To empirically evaluate whether our revised annotation framework offers clearer semantic boundaries than prior frameworks, we conducted a comparative clas-

sification experiment. This design is grounded in the hypothesis, motivated by prior work in framework comparison [26, 49], that if a framework defines types with more precise and less ambiguous boundaries, then machine learning models trained on such data should achieve higher classification performance.

Classifiers. We employed two open-source, state-of-the-art CIC models with distinct architectures:

- **CitePrompt** [32]: A prompt-based fine-tuning framework built on SciBERT [10], implemented via the OpenPrompt toolkit [19].
- **CitationIntentOpenLLM** [29]: A LLM classifier based on **Qwen-2.5-14B-Instruct** (Qwen-Small), fine-tuned using LLaMA-Factory toolkit [66].

In addition to these fine-tuned models, we also evaluated the performance of several zero-shot classifiers using the LLMs introduced in Sect. 4.1.

The selection of classifiers in this study was guided by a deliberate and strong preference for open-source models over closed-source alternatives. This commitment is crucial for several reasons fundamental to rigorous scientific inquiry. Firstly, utilizing publicly available models like SciBERT and various instruction-tuned LLMs ensures the transparency and reproducibility of our research. This allows other researchers to independently verify our findings and build upon our work without the “black box” nature or access restrictions often associated with proprietary systems. Secondly, open-source models grant the critical flexibility required for in-depth analysis and custom fine-tuning, as demonstrated by our use of LLaMA-Factory for adapting Qwen-Small. Such granular control over model architecture and training processes, often unavailable with closed-source APIs, is paramount for rigorously evaluating new annotation frameworks like SOFT and understanding nuanced model behavior. Finally, relying on open-source solutions promotes broader accessibility and fosters a collaborative research environment, which is essential for advancing the field of computational bibliometrics, unhindered by commercial licensing or opaque model mechanics.

Evaluation Metrics. Following prior CIC evaluation protocols [29, 32], we report both **Accuracy** and **Macro-averaged F_1 score**. While Accuracy provides an overall measure of correct predictions, Macro F_1 computes the average F_1 score across classes, mitigating the effects of label imbalance and better reflecting type-level learnability.

4.4 Evaluating Cross-Domain Generalizability on ACT2

To assess whether our revised framework supports generalizable citation intent classification across disciplinary contexts, we conducted a transfer evaluation using the test subset of ACT2 [40], a multi-domain benchmark that spans biology, medicine, economics, and computer science. As discussed in Sect. 2.1, ACT2 adopts a flat framework derived from ACL-ARC. To adapt it for our evaluation,

we selected a representative subset of ACT2 as cross-domain test dataset and applied the same re-annotation protocol described in Sect. 4.1 using our two-dimensional framework. For a controlled comparison, we reused the same classification models (CitePrompt, CitationIntentOpenLLM, and zero-shot LLMs) and evaluation metrics (Accuracy and Macro F_1) from Sect. 4.3. This alignment allows us to directly assess whether our framework enhances cross-domain robustness and preserves semantic clarity under domain shift.

5 Results and Analysis

We evaluate the SOFT framework along three axes: interpretability, in-domain learnability, and cross-domain generalizability. Results are compared against two existing citation frameworks, ACL-ARC and SciCite.

Table 1. Pairwise Cohen’s κ between LLMs and human annotators.

(a) ACL-ARC Framework (6 types)					(b) SciCite Framework (3 types)				
	Llama	Mistral	Gemma	Qwen		Llama	Mistral	Gemma	Qwen
Mistral	0.5833				Mistral	0.6363			
Gemma	0.4504	0.3860			Gemma	0.6855	0.6423		
Qwen	0.5776	0.6013	0.3600		Qwen	0.6762	0.7151	0.6928	
HUMAN	0.3956	0.4048	0.4302	0.3809	HUMAN	0.4016	0.3850	0.4618	0.4116

(c) SOFT: Cited Content Type (3 types)					(d) SOFT: Citation Intent (7 types)				
	Llama	Mistral	Gemma	Qwen		Llama	Mistral	Gemma	Qwen
Mistral	0.5396				Mistral	0.6154			
Gemma	0.5751	0.5657			Gemma	0.5941	0.5949		
Qwen	0.5496	0.6169	0.5717		Qwen	0.6282	0.5887	0.6175	
HUMAN	0.5193	0.4932	0.5901	0.5596	HUMAN	0.6620	0.6070	0.6232	0.6918

Interpretability and Agreement. We assess framework clarity using pairwise Cohen’s κ across four pre-trained LLMs (LLM-LLM) and between LLM predictions and human consensus labels (Human-LLM). As expected, SciCite achieves the highest LLM-LLM agreement due to its coarse granularity. However, as shown in Table 1, SOFT offers significantly higher Human-LLM agreement, especially for the **Citation Intent** dimension ($\kappa = 0.662$ for Llama, 0.692 for Qwen). Despite being conceptually more complex, **Cited Content Type** also outperforms ACL-ARC and SciCite in Human-LLM agreement, indicating that both dimensions are interpretable and consistently applicable.

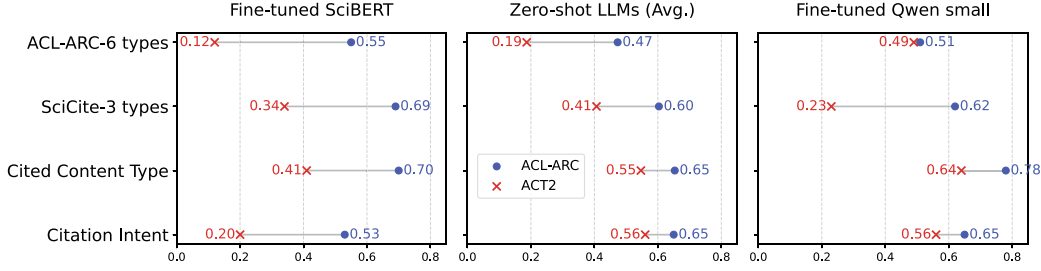


Fig. 2. Comparison of in-domain and cross-domain macro F_1 scores across four annotation frameworks. Each subplot corresponds to a model type, with blue circles denoting in-domain and red crosses indicating cross-domain performance.

In-Domain Learnability. We evaluate the classification performance of both fine-tuned and zero-shot models on the ACL-ARC test set using three annotation frameworks: ACL-ARC, SciCite, and SOFT. In the SOFT framework, **Cited Content Type** and **Citation Intent** are treated as two independent classification tasks. Table 2 and Fig. 3 in Appendix B report accuracy and macro F_1 scores across all models. Despite its conceptual specificity, **Cited Content Type** classification yields strong in-domain performance. Fine-tuned Qwen-Small achieves 0.78 macro F_1 , while zero-shot models average 0.66. Although no existing framework provides a comparable baseline, these results demonstrate that content type distinctions are not only human-interpretable but also model-learnable across architectures. SOFT also outperforms prior frameworks on **Citation Intent** classification. For example, fine-tuned Qwen-Small achieves 0.65 macro F_1 under SOFT, compared to 0.51 on ACL-ARC and 0.62 on SciCite. Notably, this gain occurs even though SOFT-Intent has more types (7) than ACL-ARC (6), refuting the assumption that performance is merely a function of label count. This suggests the framework’s semantic clarity and well-defined boundaries are more critical to model performance than raw granularity. The average zero-shot F_1 under SOFT reinforces this point, reaching 0.69, substantially above ACL-ARC (0.49) and SciCite (0.61), demonstrating that SOFT’s functional types align more closely with both pre-trained and fine-tuned model representations.

We note that our reported scores are lower than those in prior work, including CitePrompt [32], which may be attributed to differences in dataset partitioning, as discussed in Sect. 4.1.

Cross-Domain Generalizability. To assess the robustness of different frameworks under domain shift, we evaluate models trained on ACL-ARC directly on a re-annotated subset of ACT2. No further fine-tuning or domain adaptation is performed. Table 2 and Fig. 3 in Appendix B report both accuracy and macro

F_1 scores, highlighting how well each framework generalizes beyond the original training domain. Despite its conceptual complexity, **Cited Content Type** classification generalizes well. Zero-shot LLMs maintain solid performance (average F_1 drop 17%), and fine-tuned Qwen-Small drops moderately (0.78 to 0.64), suggesting that these content distinctions remain stable across domain shifts. SOFT-trained models exhibit superior generalizability on **Citation Intent** classification. Fine-tuned Qwen-Small drops from 0.65 to 0.56 (SOFT), outperforming ACL-ARC (0.51 to 0.49) and SciCite (0.62 to 0.23). Zero-shot performance mirrors this trend (Fig. 2). SOFT’s average F_1 drop is only 15.8% (vs. 30–65% for other frameworks), suggesting its discourse-grounded intent types generalize more robustly.

Dimension-Wise Comparison. A dimension-wise analysis reveals distinct model behaviors. In-domain, zero-shot LLMs perform better on **Citation Intent** (avg. F_1 : 0.69) than **Cited Content Type** (0.66), suggesting these functions align well with LLM priors. With fine-tuning, however, **Cited Content Type** becomes more learnable (Qwen-Small F_1 : 0.78 vs. 0.65 for intent), indicating its distinctions benefit more from supervision. Cross-domain generalization is competitive for both dimensions, with fine-tuned Qwen showing only modest performance drops on both content (Qwen-Small: 18%; SciBERT: 41%) and intent (Qwen-Small: 14%; SciBERT: 62%). This highlights SOFT’s stability under domain shift and the superior transferability of fine-tuned Qwen over SciBERT.

Overall Analysis. Across all experiments, the SOFT framework consistently outperforms ACL-ARC and SciCite in interpretability, learnability, and cross-domain robustness. Fine-tuned Qwen-Small achieves the best results on **Cited Content Type**, while Qwen zero-shot achieves top performance on **Citation Intent**. These findings establish SOFT as a semantically orthogonal and generalizable alternative for citation annotation.

6 Conclusion

This paper introduces the SOFT framework, a semantically orthogonal framework with two dimensions for citation annotation that disentangles *cited content type* from *citation intent*. By addressing the conceptual conflation and boundary ambiguity in existing frameworks, SOFT provides clearer interpretive definitions and better supports both human annotation and model-based

classification. Empirical evaluations across LLM agreement, in-domain classification, and cross-domain generalization confirm SOFT’s advantages. The *citation intent* dimension demonstrates high zero-shot and fine-tuned performance, while the *cited content type* dimension, despite its semantic complexity, proves learnable and transferable. Compared to ACL-ARC and SciCite, SOFT consistently achieves higher human-LLM agreement, stronger in-domain macro F_1 , and more stable cross-domain generalization. These gains are especially pronounced for instruction-tuned LLMs, which preserve broader generalization after fine-tuning.

Future work will explore extending SOFT to support multi-label citation contexts, integrating finer-grained functional types, and applying the framework to large-scale citation graphs for discourse-level functional modeling, influence tracking, and knowledge extraction.

Acknowledgment. Changxu Duan was funded by the Federal Ministry of Education and Research (BMBF) under grant no. 01UG2130A, as part of the InsightsNet research project (insightsnet.org). Zhiyin Tan was funded by the “HybrInt - Hybrid Intelligence through Interpretable AI in Machine Perception and Interaction” project (Zukunft Nds, Niedersächsisches Ministerium für Wissenschaft, Grant ID: ZN4219). We thank Dr. Sabine Bartsch for proofreading. We also thank the anonymous reviewers for their insightful comments and suggestions.

Appendix

A Overall Model Performance

Table 2 presents the overall performance of the evaluated models across the ACL-ARC, SciCite, and our proposed SOFT frameworks, for both in-domain (ACL-ARC test set) and cross-domain (re-annotated ACT2 test set) scenarios. The performance is measured by accuracy and macro- F_1 score.

B Per-Type Analysis Across Frameworks and Models

To gain a more granular understanding of model behavior and framework characteristics, we visualize the per-class F_1 -scores for both in-domain and cross-domain settings using radar plots, as shown in Fig. 3. Each plot depicts the performance of a specific model across the classes of a given framework, with blue representing in-domain and red representing cross-domain F_1 Scores. These visualizations allow for a direct comparison of how well individual classes are learned and how performance on these classes transfers across domains.

Across ACL-ARC and SciCite, only a few types achieve consistently learnable performance. In ACL-ARC, USES ($F_1 = 0.60\text{--}0.72$) and FUTURE ($F_1 = 0.70\text{--}0.80$) are reliably detected across models, while types like MOTIVATION ($F_1 = 0.06\text{--}0.27$) and extends ($F_1 = 0.17\text{--}0.24$) exhibit uniformly poor results, regardless of model type. In SciCite, background and method achieve moderate F_1 (up to 0.83 and 0.69), but RESULTCOMPARISON remains consistently weak ($F_1 \leq 0.47$, recall < 0.5). These patterns suggest semantic overlap and label fuzziness in legacy frameworks, especially for tasks that require disambiguating subtle intent.

The SOFT framework, in contrast, features multiple types that are both conceptually precise and computationally learnable. Zero-shot LLMs perform well on USE ($F_1 = 0.79\text{--}0.90$), CONTEXTUALIZE (0.78–0.86), and PRODUCE-RESOURCE (0.76–0.82), while the fine-tuned LLM (Qwen-small) outperforms all other models on discovery and PRODUCEDRESOURCE (both $F_1 = 0.82$). More difficult types like PERFORMEDWORK show a large performance gap in F_1 score (LLMs: 0.37–0.57, Qwen-small: 0.69, SciBERT: 0.79), indicating that domain-adaptive fine-tuning is crucial when lexical variation is high. For JUSTIFYDESIGNCHOICE, zero-shot LLM Llama ($F_1 = 0.72$) outperforms both fine-tuned models (Qwen-small: $F_1 = 0.45$, SciBERT: $F_1 = 0.42$), suggesting that general discourse modeling, rather than label-specific exposure, governs success.

Precision-recall asymmetries further differentiate framework quality. In SciCite, the method exhibits high recall (e.g., zero-shot LLMs: 0.86–0.94) but low precision (e.g., zero-shot LLMs: 0.4–0.53), indicating label overgeneralization. In SOFT, several types, including JUSTIFYDESIGNCHOICE and performed work, exhibit the opposite: high precision (e.g., Qwen: 0.79) but low recall (e.g., Mistral: 0.28), showing that models can detect prototypical cases but struggle with broader coverage. Notably, SIGNALGAP (Qwen: precision 0.81, recall 0.68) and METHOD (SciBERT: precision 0.64, recall 0.76) achieve better balance. These contrasts affirm that our SOFT framework not only yields more distinguishable and learnable labels but also makes model errors more interpretable, enabling both zero-shot application and targeted fine-tuning.

C Computational Resources and Software

We adopted the original hyperparameter settings from CitePrompt [32] and CitationIntentOpenLLM [29]. All experiments were conducted on a single H100 GPU, including both fine-tuning and inference. For LLM inference, we used the vLLM library [31]. Fine-tuning required approximately 5 GPU-hours, and inference consumed an additional 4 GPU-hours in total.

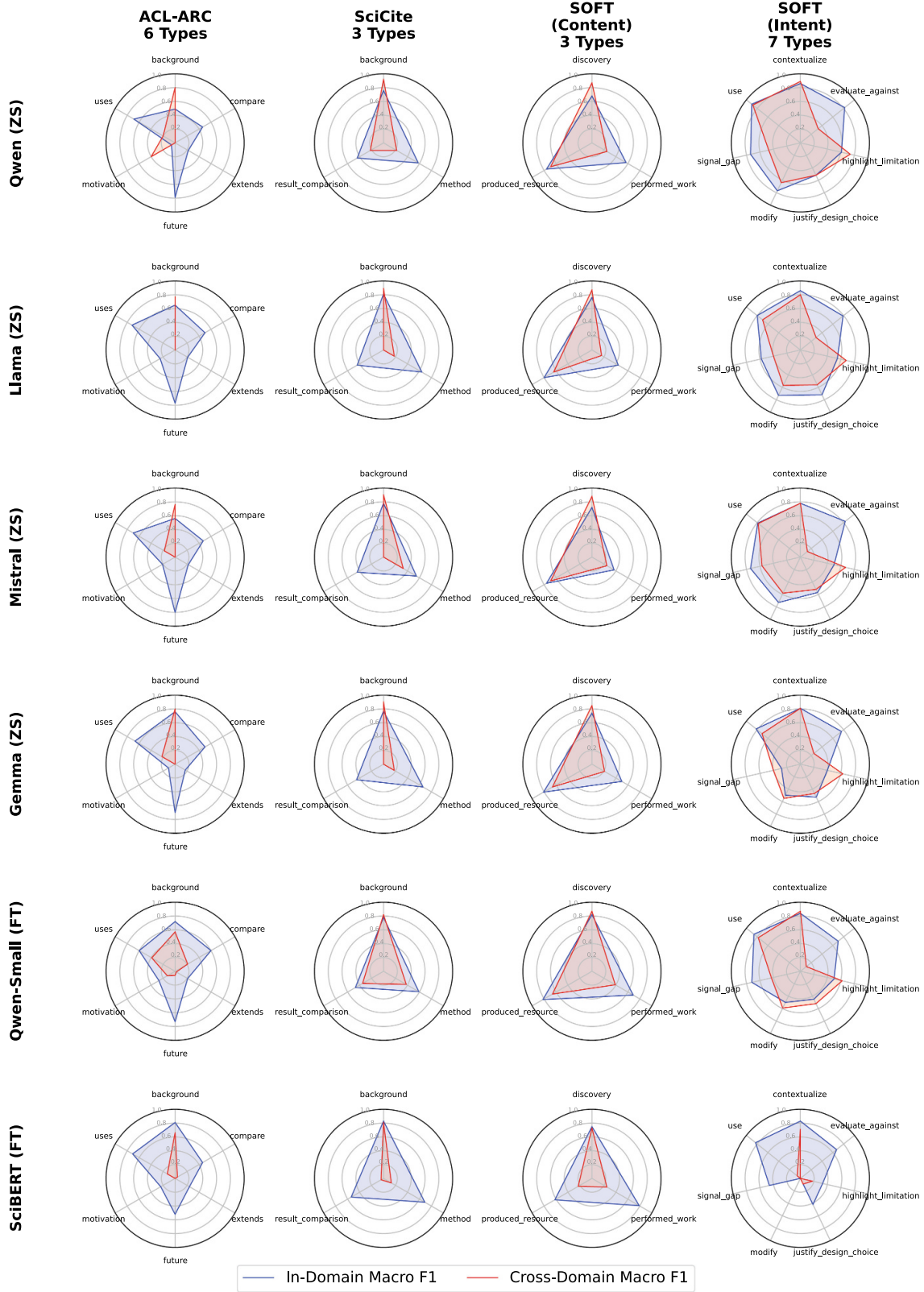


Fig. 3. Grid of radar plots (4 frameworks $\times$ 6 models) showing per-class F1-scores. Blue lines/areas represent In-Domain F1 scores, and red lines/areas represent Cross-Domain F1 scores. Each axis corresponds to a class label within the respective framework. F1-scores range from 0 (center) to 1 (outer edge).

Table 2. In-domain and cross-domain performance of citation classification models (zero-shot: “ZS”, fine-tuned: “FT”) across frameworks.

Framework Classifier		In-Domain (ACL-ARC)		Cross-Domain (ACT2)	
		Accuracy	Macro F_1	Accuracy	Macro F_1
ACL-ARC (6 types)	ZS Llama	0.58	0.52	0.60	0.13 (75%↓)
	ZS Mistral	0.54	0.49	0.59	0.16 (67%↓)
	ZS Gemma	0.65	0.48	0.62	0.17 (65%↓)
	ZS Qwen	0.48	0.45	0.63	0.23 (49%↓)
	FT SciBERT	0.66	0.55	0.47	0.12 (78%↓)
	FT Qwen-Small	0.65	0.51	0.58	0.49 (4%↓)
SciCite (3 types)	ZS Llama	0.72	0.63	0.80	0.37 (54%↓)
	ZS Mistral	0.67	0.59	0.81	0.41 (31%↓)
	ZS Gemma	0.70	0.63	0.80	0.36 (43%↓)
	ZS Qwen	0.67	0.59	0.84	0.45 (24%↓)
	FT SciBERT	0.77	0.69	0.66	0.34 (51%↓)
	FT Qwen-Small	0.69	0.61	0.40	0.23 (63%↓)
SOFT: Cited Content Type (3 types)	ZS Llama	0.73	0.67	0.75	0.55 (18%↓)
	ZS Mistral	0.67	0.62	0.78	0.46 (26%↓)
	ZS Gemma	0.74	0.68	0.73	0.58 (15%↓)
	ZS Qwen	0.70	0.67	0.77	0.60 (10%↓)
	FT SciBERT	0.70	0.70	0.58	0.41 (41%↓)
	FT Qwen-Small	0.79	0.78	0.75	0.64 (18%↓)
SOFT: Citation Intent (7 types)	ZS Llama	0.79	0.72	0.72	0.57 (21%↓)
	ZS Mistral	0.74	0.71	0.69	0.57 (20%↓)
	ZS Gemma	0.71	0.59	0.71	0.55 (7%↓)
	ZS Qwen	0.81	0.75	0.82	0.64 (15%↓)
	FT SciBERT	0.72	0.53	0.52	0.20 (62%↓)
	FT Qwen-Small	0.77	0.65	0.77	0.56 (14%↓)

Color Legend: ≤20% drop , 21–40% drop , >40% drop in cross-domain Macro F_1 .

References

1. Abu-Jbara, A., Ezra, J., Radev, D.: Purpose and polarity of citation: towards NLP-based bibliometrics. In: Vanderwende, L., Daumé III, H., Kirchhoff, K. (eds.) Proceedings of the 2013 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, pp. 596–606. Association for Computational Linguistics, Atlanta, Georgia (2013). <https://aclanthology.org/N13-1067/>
2. Allen Institute for AI: Semantic Scholar (2025). <https://www.semanticscholar.org/>

3. Anderson, M.H., Lemken, R.K.: Citation context analysis as a method for conducting rigorous and impactful literature reviews. *Organ. Res. Methods* **26**(1), 77–106 (2020). <https://doi.org/10.1177/1094428120969905>
4. Araabi, A., Niculae, V., Monz, C.: Joint dropout: improving generalizability in low-resource neural machine translation through phrase pair variables. In: Utiyama, M., Wang, R. (eds.) *Proceedings of Machine Translation Summit XIX, Vol. 1: Research Track*, pp. 12–25. Asia-Pacific Association for Machine Translation, Macau SAR, China (2023). <https://aclanthology.org/2023.mtsummit-research.2/>
5. Artstein, R., Poesio, M.: Survey article: inter-coder agreement for computational linguistics. *Comput. Linguist.* **34**(4), 555–596 (2008). <https://doi.org/10.1162/coli.07-034-R2>. <https://aclanthology.org/J08-4004/>
6. Baker, C.F., Fillmore, C.J., Lowe, J.B.: The Berkeley FrameNet project. In: 36th Annual Meeting of the Association for Computational Linguistics and 17th International Conference on Computational Linguistics, Volume 1, pp. 86–90. Association for Computational Linguistics, Montreal, Quebec, Canada (1998). <https://doi.org/10.3115/980845.980860>. <https://aclanthology.org/P98-1013/>
7. Bascur, J.P., Verberne, S., van Eck, N.J., Waltman, L.: Academic information retrieval using citation clusters: in-depth evaluation based on systematic reviews. *Scientometrics* **128**(5), 2895–2921 (2023). <https://doi.org/10.1007/s11192-023-04681-x>
8. Bassignana, E., Plank, B.: CrossRE: a cross-domain dataset for relation extraction. In: Goldberg, Y., Kozareva, Z., Zhang, Y. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2022*, pp. 3592–3604. Association for Computational Linguistics, Abu Dhabi, United Arab Emirates (2022). <https://doi.org/10.18653/v1/2022.findings-emnlp.263>. <https://aclanthology.org/2022.findings-emnlp.263/>
9. Bayerl, P.S., Paul, K.I.: What determines inter-coder agreement in manual annotations? A meta-analytic investigation. *Comput. Linguist.* **37**(4), 699–725 (2011). https://doi.org/10.1162/COLI_a_00074. <https://aclanthology.org/J11-4004/>
10. Beltagy, I., Lo, K., Cohan, A.: SciBERT: a pretrained language model for scientific text. In: Inui, K., Jiang, J., Ng, V., Wan, X. (eds.) *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pp. 3615–3620. Association for Computational Linguistics, Hong Kong, China (2019). <https://doi.org/10.18653/v1/D19-1371>. <https://aclanthology.org/D19-1371/>
11. Berrebbi, D., Huynh, N., Balalau, O.: Graphcite: citation intent classification in scientific publications via graph embeddings, pp. 779–783. *WWW 2022: The ACM Web Conference 2022*. ACM (2022). <https://doi.org/10.1145/3487553.3524657>
12. Chen, Y., et al.: CDEvalSumm: an empirical study of cross-dataset evaluation for neural summarization systems. In: Cohn, T., He, Y., Liu, Y. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 3679–3691. Association for Computational Linguistics, Online (2020). <https://doi.org/10.18653/v1/2020.findings-emnlp.329>. <https://aclanthology.org/2020.findings-emnlp.329/>
13. Clarivate Analytics: Web of Science (2025). <https://www.webofscience.com/>
14. Cohan, A., Ammar, W., van Zuylen, M., Cady, F.: Structural scaffolds for citation intent classification in scientific publications. In: Burstein, J., Doran, C., Solorio, T. (eds.) *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 3586–3596. Association for Computational Linguistics, Minneapolis, Minnesota (2019). <https://doi.org/10.18653/v1/N19-1361>. <https://aclanthology.org/N19-1361/>

15. Cohen, A.M., Hersh, W.R., Peterson, K., Yen, P.Y.: Reducing workload in systematic review preparation using automated citation classification. *J. Am. Med. Inform. Assoc.* **13**(2), 206–219 (2006). <https://doi.org/10.1197/jamia.m1929>
16. Cohen, J.: A coefficient of agreement for nominal scales. *Educ. Psychol. Measur.* **20**(1), 37–46 (1960). <https://doi.org/10.1177/001316446002000104>
17. Dardas, L.A., Sallam, M., Woodward, A., Sweis, N., Sweis, N., Sawair, F.A.: Evaluating research impact based on semantic scholar highly influential citations, total citations, and altmetric attention scores: the quest for refined measures remains illusive. *Publications* **11**(1), 5 (2023). <https://doi.org/10.3390/publications11010005>
18. Devlin, J., Chang, M.W., Lee, K., Toutanova, K.: BERT: pre-training of deep bidirectional transformers for language understanding. In: Burstein, J., Doran, C., Solorio, T. (eds.) *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pp. 4171–4186. Association for Computational Linguistics, Minneapolis, Minnesota (2019). <https://doi.org/10.18653/v1/N19-1423>. <https://aclanthology.org/N19-1423/>
19. Ding, N., et al.: OpenPrompt: an open-source framework for prompt-learning. In: Basile, V., Kozareva, Z., Stajner, S. (eds.) *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics: System Demonstrations*, pp. 105–113. Association for Computational Linguistics, Dublin, Ireland (2022). <https://doi.org/10.18653/v1/2022.acl-demo.10>. <https://aclanthology.org/2022.acl-demo.10/>
20. Fillmore, C.J.: The case FRO case. *Universals in Linguistic Theory* (1968)
21. Fillmore, C.J., et al.: Frame semantics. *Cogn. Linguist. Basic Read.* **34**, 373–400 (2006)
22. Ghosal, T., Varanasi, K.K., Kordoni, V.: A deep multi-tasking approach leveraging on cited-citing paper relationship for citation intent classification. *Scientometrics* **129**(2), 767–783 (2023). <https://doi.org/10.1007/s11192-023-04811-5>
23. Gilardi, F., Alizadeh, M., Kubli, M.: Chatgpt outperforms crowd workers for text-annotation tasks. *Proc. Natl. Acad. Sci.* **120**(30) (2023). <https://doi.org/10.1073/pnas.2305016120>
24. Guo, M., Yang, Y., Cer, D., Shen, Q., Constant, N.: MultiReQA: a cross-domain evaluation for retrieval question answering models. In: Ben-David, E., et al. (eds.) *Proceedings of the Second Workshop on Domain Adaptation for NLP*, pp. 94–104. Association for Computational Linguistics, Kyiv, Ukraine (2021). <https://aclanthology.org/2021.adaptnlp-1.10/>
25. Hettiachchi, D., et al.: Investigating and mitigating biases in crowdsourced data, pp. 331–334. *CSCW 2021: Computer Supported Cooperative Work and Social Computing*. ACM (2021). <https://doi.org/10.1145/3462204.3481729>
26. Ivanova, R.V., Kirrane, S., van Erp, M.: Comparing annotated datasets for named entity recognition in English literature. In: Calzolari, N., et al. (eds.) *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pp. 3788–3797. European Language Resources Association, Marseille, France (2022). <https://aclanthology.org/2022.lrec-1.404/>
27. Jha, R., Jbara, A.A., Qazvinian, V., Radev, D.R.: NLP-driven citation analysis for scientometrics. *Nat. Lang. Eng.* **23**(1), 93–130 (2016). <https://doi.org/10.1017/s1351324915000443>
28. Jurgens, D., Kumar, S., Hoover, R., McFarland, D., Jurafsky, D.: Measuring the evolution of a scientific field through citation frames. *Trans. Assoc. Com-*

- put. *Linguist.* **6**, 391–406 (2018). https://doi.org/10.1162/tacl_a_00028. <https://aclanthology.org/Q18-1028/>
29. Koloveas, P., Chatzopoulos, S., Vergoulis, T., Tryfonopoulos, C.: Can LLMs predict citation intent? An experimental analysis of in-context learning and fine-tuning on open LLMs (2025). <https://arxiv.org/abs/2502.14561>
 30. Kunnath, S.N., Pride, D., Knoth, P.: Prompting strategies for citation classification. In: *Proceedings of the 32nd ACM International Conference on Information and Knowledge Management, CIKM 2023*, pp. 1127–1137. Association for Computing Machinery, New York (2023). <https://doi.org/10.1145/3583780.3615018>
 31. Kwon, W., et al.: Efficient memory management for large language model serving with pagedattention. In: *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles* (2023)
 32. Lahiri, A., Sanyal, D.K., Mukherjee, I.: Citeprompt: using prompts to identify citation intent in scientific papers, pp. 51–55. *2023 ACM/IEEE Joint Conference on Digital Libraries (JCDL)*. IEEE (2023). <https://doi.org/10.1109/jcdl57899.2023.00017>
 33. Li, K., et al.: Probing out-of-distribution generalization in machine learning for materials (2024). <https://arxiv.org/abs/2406.06489>
 34. Lin, T.-Y., et al.: Microsoft COCO: common objects in context. In: Fleet, D., Pajdla, T., Schiele, B., Tuytelaars, T. (eds.) *ECCV 2014*. LNCS, vol. 8693, pp. 740–755. Springer, Cham (2014). https://doi.org/10.1007/978-3-319-10602-1_48
 35. Lo, K., Wang, L.L., Neumann, M., Kinney, R., Weld, D.: S2ORC: the semantic scholar open research corpus. In: Jurafsky, D., Chai, J., Schluter, N., Tetreault, J. (eds.) *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, pp. 4969–4983. Association for Computational Linguistics, Online (2020). <https://doi.org/10.18653/v1/2020.acl-main.447>. <https://aclanthology.org/2020.acl-main.447/>
 36. Meng, R., Lu, W., Huan Chi, Y., Han, S.: Automatic classification of citation function by new linguistic features (2017). <http://memray.me/uploads/iconference17-citation-function.pdf>
 37. Mercier, D., Rizvi, S., Rajashekar, V., Dengel, A., Ahmed, S.: Impactcite: An xlnet-based solution enabling qualitative citation impact analysis utilizing sentiment and intent. In: *Proceedings of the 13th International Conference on Agents and Artificial Intelligence*, pp. 159–168. SCITEPRESS - Science and Technology Publications (2021). <https://doi.org/10.5220/0010235201590168>
 38. Mistral AI Team: Mistral Small 3 (2025). <https://mistral.ai/news/mistral-small-3/>
 39. Nambanoor Kunnath, S., Pride, D., Knoth, P.: Dynamic context extraction for citation classification. In: He, Y., Ji, H., Li, S., Liu, Y., Chang, C.H. (eds.) *Proceedings of the 2nd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics and the 12th International Joint Conference on Natural Language Processing (Volume 1: Long Papers)*, pp. 539–549. Association for Computational Linguistics, Online only (2022). <https://doi.org/10.18653/v1/2022.aacl-main.41>. <https://aclanthology.org/2022.aacl-main.41/>
 40. Nambanoor Kunnath, S., Stauber, V., Wu, R., Pride, D., Botev, V., Knoth, P.: ACT2: a multi-disciplinary semi-structured dataset for importance and purpose classification of citations. In: Calzolari, N., et al. (eds.) *Proceedings of the Thirteenth Language Resources and Evaluation Conference*, pp. 3398–3406. European Language Resources Association, Marseille, France (2022). <https://aclanthology.org/2022.lrec-1.363/>

41. Nicholson, J.M., et al.: scite: a smart citation index that displays the context of citations and classifies their intent using deep learning. *Quant. Sci. Stud.* **2**(3), 882–898 (2021). https://doi.org/10.1162/qss_a_00146
42. Northcutt, C., Jiang, L., Chuang, I.: Confident learning: estimating uncertainty in dataset labels. *J. Artif. Intell. Res.* **70**, 1373–1411 (2021). <https://doi.org/10.1613/jair.1.12125>
43. Northcutt, C.G., Athalye, A., Mueller, J.: Pervasive label errors in test sets destabilize machine learning benchmarks. In: *Thirty-Fifth Conference on Neural Information Processing Systems Datasets and Benchmarks Track (Round 1)* (2021). <https://openreview.net/forum?id=XccDXrDNLek>
44. Palmer, M., Gildea, D., Kingsbury, P.: The proposition bank: an annotated corpus of semantic roles. *Comput. Linguist.* **31**(1), 71–106 (2005). <https://doi.org/10.1162/0891201053630264>. <https://aclanthology.org/J05-1004/>
45. Pride, D., Knoth, P.: An authoritative approach to citation classification. In: *Proceedings of the ACM/IEEE Joint Conference on Digital Libraries in 2020, JCDL 2020*, pp. 337–340. Association for Computing Machinery, New York (2020). <https://doi.org/10.1145/3383583.3398617>
46. Pride, D., Knoth, P.: An authoritative approach to citation classification. *JCDL 2020: The ACM/IEEE Joint Conference on Digital Libraries in 2020*. ACM (2020). <https://doi.org/10.1145/3383583.3398617>
47. Qazvinian, V., Radev, D.R.: Scientific paper summarization using citation summary networks. In: Scott, D., Uszkoreit, H. (eds.) *Proceedings of the 22nd International Conference on Computational Linguistics (Coling 2008)*, pp. 689–696. Coling 2008 Organizing Committee, Manchester, UK (2008). <https://aclanthology.org/C08-1087/>
48. Qwen: Qwen2.5 technical report (2025). <https://arxiv.org/abs/2412.15115>
49. Riabi, A., Moulleron, V., Mahamdi, M., Antoun, W., Seddah, D.: Beyond dataset creation: critical view of annotation variation and bias probing of a dataset for online radical content detection. In: Rambow, O., Wanner, L., Apidianaki, M., Al-Khalifa, H., Eugenio, B.D., Schockaert, S. (eds.) *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 8640–8663. Association for Computational Linguistics, Abu Dhabi, UAE (2025). <https://aclanthology.org/2025.coling-main.578/>
50. Roman, M., Shahid, A., Khan, S., Koubaa, A., Yu, L.: Citation intent classification using word embedding. *IEEE Access* **9**, 9982–9995 (2021). <https://doi.org/10.1109/access.2021.3050547>
51. Rücker, S., Akbik, A.: CleanCoNLL: a nearly noise-free named entity recognition dataset. In: Bouamor, H., Pino, J., Bali, K. (eds.) *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 8628–8645. Association for Computational Linguistics, Singapore (2023). <https://doi.org/10.18653/v1/2023.emnlp-main.533>. <https://aclanthology.org/2023.emnlp-main.533/>
52. Shotton, D.: CiTO, the citation typing ontology. *J. Biomed. Semant.* **1**(Suppl. 1), S6 (2010). <https://doi.org/10.1186/2041-1480-1-s1-s6>
53. Shui, Z., et al.: Fine-tuning language models on multiple datasets for citation intention classification. In: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (eds.) *Findings of the Association for Computational Linguistics: EMNLP 2024*, pp. 16718–16732. Association for Computational Linguistics, Miami, Florida, USA (2024). <https://doi.org/10.18653/v1/2024.findings-emnlp.974>. <https://aclanthology.org/2024.findings-emnlp.974/>

54. Singh, S., Yadav, A., Jain, J., Shi, H., Johnson, J., Desai, K.: Benchmarking object detectors with coco: a new path forward. In: Leonardis, A., Ricci, E., Roth, S., Russakovsky, O., Sattler, T., Varol, G. (eds.) ECCV 2024, pp. 279–295. Springer, Cham (2025). https://doi.org/10.1007/978-3-031-72784-9_16
55. Spiegel-Rosing, I.: Science studies: bibliometric and content analysis. *Soc. Stud. Sci.* **7**(1), 97–113 (2 1977). <https://doi.org/10.1177/030631277700700111>
56. Tan, Z., et al.: Large language models for data annotation and synthesis: a survey. In: Al-Onaizan, Y., Bansal, M., Chen, Y.N. (eds.) Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing, pp. 930–957. Association for Computational Linguistics, Miami, Florida, USA (2024). <https://doi.org/10.18653/v1/2024.emnlp-main.54>. <https://aclanthology.org/2024.emnlp-main.54/>
57. Team, G.: Gemma 3 technical report (2025). <https://arxiv.org/abs/2503.19786>
58. Team, L.: The llama 3 herd of models (2024). <https://arxiv.org/abs/2407.21783>
59. Teufel, S.: Argumentative zoning: information extraction from scientific text (1999). <https://www.cl.cam.ac.uk/~sht25/thesis/t1.pdf>
60. Teufel, S., Siddharthan, A., Tidhar, D.: Automatic classification of citation function. In: Jurafsky, D., Gaussier, E. (eds.) Proceedings of the 2006 Conference on Empirical Methods in Natural Language Processing, pp. 103–110. Association for Computational Linguistics, Sydney, Australia (2006). <https://aclanthology.org/W06-1613/>
61. Tjong Kim Sang, E.F., De Meulder, F.: Introduction to the CoNLL-2003 shared task: language-independent named entity recognition. In: Proceedings of the Seventh Conference on Natural Language Learning at HLT-NAACL 2003, pp. 142–147 (2003). <https://aclanthology.org/W03-0419/>
62. Tkachenko, U., Thyagarajan, A., Mueller, J.: Objectlab: automated diagnosis of mislabeled images in object detection data (2023). <https://doi.org/10.48550/ARXIV.2309.00832>. <https://arxiv.org/abs/2309.00832>
63. Valenzuela, M., Ha, V., Etzioni, O.: Identifying meaningful citations. In: AAAI Workshop: Scholarly Big Data, vol. 15, p. 13 (2015)
64. Yu, H., Liu, J., Zhang, X., Wu, J., Cui, P.: A survey on evaluation of out-of-distribution generalization (2024). <https://arxiv.org/abs/2403.01874>
65. Zha, D., et al.: Data-centric artificial intelligence: a survey. *ACM Comput. Surv.* **57**(5) (2025). <https://doi.org/10.1145/3711118>
66. Zheng, Y., Zhang, R., Zhang, J., Ye, Y., Luo, Z.: LlamaFactory: unified efficient fine-tuning of 100+ language models. In: Cao, Y., Feng, Y., Xiong, D. (eds.) Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 3: System Demonstrations), pp. 400–410. Association for Computational Linguistics, Bangkok, Thailand (2024). <https://doi.org/10.18653/v1/2024.acl-demos.38>. <https://aclanthology.org/2024.acl-demos.38/>
67. Zimmermann, E., Szeto, J., Pasquero, J., Ratle, F.: Benchmarking a benchmark: how reliable is MS-coco? (2023). <https://arxiv.org/abs/2311.02709>